

The Landscape of Agentic Time Series Systems: Architectures, Reliability, and Frontiers

Yifan Hu^{†,✉}, Jie Yang[†], Xilin Dai[†], Wanxu Cai^{*}, Kuiye Ding^{*}, Yuante Li^{*}, Qinghua Liu^{*}, Enze Ma^{*}, Zhiyuan Qu^{*}, Yixin Wang^{*}, Binyan Xu^{*}, Kexin Zhang^{*}, Peiyuan Liu, Zhijian Xu, Guibin Zhang, Yujin Tang, Yanwei Yue, Kening Zheng, Chengze Li, Hanrong Zhang, Haoyan Xu, Naiqi Li, Tao Dai, Dawei Cheng, John Paparrizos, Kaize Ding, Tian Zhou, Qiang Xu[‡], Shu-tao Xia[‡], Shirui Pan, Philip S. Yu[‡]

[†]Co-first authors. ^{*}Core contributors (listed alphabetically)  Project Organizer. [‡]Core Supervisors.

Affiliations: Tsinghua University, University of Illinois Chicago, The Chinese University of Hong Kong, University of Technology Sydney, Carnegie Mellon University, The Ohio State University, University of Southern California, National University of Singapore, Dartmouth College, Peking University, Shenzhen University, Tongji University, Northwestern University, Griffith University

Time series analysis is increasingly moving beyond the classical goal of producing accurate forecasts. In many real-world domains, including finance, healthcare, transportation, climate, energy, industry, and scientific discovery, time series data must be interpreted, contextualized, verified, and often transformed into decisions whose consequences unfold over time. Recent time series foundation models improve cross-domain generalization through reusable temporal representations, while Large Language Models for Time Series (LLM4TS) methods expand time series modeling by incorporating language interfaces, multimodal alignment, and explicit temporal reasoning. However, most existing systems are still largely model-centric, mapping observations to predictions, explanations, or answers without establishing a closed loop involving evidence acquisition, tool or action selection, feedback, and state updates. This survey provides a systematic landscape of *agentic time series systems*, an emerging paradigm that treats time series analysis as closed-loop interaction with temporal environments. We clarify the progression from time series foundation models to LLM-based translators, temporal reasoners, and closed-loop time series agents. We define a time series agent as an operational system that observes temporal evidence, selects tools or actions based on evolving states, receives feedback, and updates subsequent behavior. To organize the literature, we propose a five-layer capability map covering time series perception, time series reasoning, planning and action, memory and knowledge, and temporal world models. This map is not a mandatory checklist for every prototype, but rather a structured view of how time series agents progressively acquire stronger capabilities for interpretation, adaptation, decision-making, and environment modeling. In parallel, we treat benchmarks as an evolving evaluative infrastructure reflecting the field’s shift toward multimodal understanding, reasoning faithfulness, tool-use reliability, and decision-centric assessment. Furthermore, we position reliability and trustworthiness as cross-cutting evaluative dimensions that constrain all capability layers, covering forecasting quality, reasoning faithfulness, tool-use reliability, hallucination and grounding, robustness, decision safety, human alignment, auditability and reproducibility. Building on an extensive literature corpus, we synthesize representative methods, benchmarks, applications, and frontiers, highlighting current capabilities, fragmented evaluations, and open problems for trustworthy deployment. We argue that the next phase of time series research should involve developing reliable, auditable, and adaptive agents that can analyze, decide, learn, and act under dynamic uncertainty.

✉ **Main Contact:** huyf25@mails.tsinghua.edu.cn, jyang265@uic.edu, 1155279830@link.cuhk.edu.hk

🔗 **Github:** <https://github.com/TROUBADOUR000/Awesome-Agentic-Time-Series>

Note: If you identify your own or other papers relevant to this survey that have not been discussed (we apologize for any such omissions due to the rapidly expanding literature), please feel free to contact us via email or raise an issue on [GitHub](#).

Contents

1	Introduction	4
2	Preliminaries: Formalizing Agentic Time Series Systems	7
2.1	Four Paradigms of LLM-Based Time Series Systems	8
2.2	Formalizing the Time Series Agent	9
2.3	Time Series Agents $\neq$ NLP Agents	10
2.4	The Five-Layer Architecture	13
3	Time Series Perception: Observing and Structuring Temporal Environments	14
3.1	Raw Numerical Perception	15
3.2	Diagnostic Tool-based Perception	16
3.3	Symbolic Perception	17
3.4	Structural Perception	18
3.5	Multimodal Perception	19
3.6	Evidence Map and Design Implications	20
4	Time Series Reasoning: Inferring Temporal Dynamics and Causality	21
4.1	Pattern and Structural Reasoning	22
4.2	Numerical and Statistic-Grounded Reasoning	23
4.3	Causal and Compositional Reasoning	24
4.4	Reflective and Metacognitive Reasoning	24
4.5	Slow-Thinking and Reinforcement-Enhanced Reasoning	25
5	Planning and Action: From Forecasting to Autonomous Decision-Making	27
5.1	Workflow Planning: Making Forecasting Pipelines Explicit	27
5.2	Tool Routing and Evidence Acquisition	29
5.3	Model, Data, and Code Orchestration	30
5.4	Multi-Agent Coordination: Separating Incompatible Responsibilities	30
5.5	Sequential Decision-Making and External Action	31
5.6	Evidence Map and Design Implications	32
6	Memory and Knowledge: Accumulating Temporal Experience	32
6.1	Beyond Conversation History: Why Time Series Agents Need Structured Memory	33
6.2	Context and Working Memory	35
6.3	Episodic and Case-Based Memory	36
6.4	Long-Horizon Temporal and Regime Memory	37
6.5	Knowledge Memory	38
6.6	Procedural and Reflective Memory	38
6.7	Failure and Confidence Memory	40
6.8	Memory Dynamics: Formation, Retrieval, and Evolution	40
6.9	Evidence Map and Open Challenges	41
7	Time Series World Models: Simulating Dynamic Temporal Environments	43
7.1	Environment Understanding: Structuring Temporal Dependencies	44
7.2	Temporal Simulation: From Forecasting to Controlled Generation	45
7.3	Counterfactual Simulation: Alternative Futures Under Changed Conditions	45
7.4	Executable Grounding: Tool-Exposed World-Model Interfaces	46
7.5	Synthesis and Outlook	46
8	Reliability and Trustworthiness in Agentic Time Series Systems	47
8.1	From Predictive Accuracy to a Reliability Stack	48
8.2	Robustness Under Temporal Shift and Extreme Events	49
8.3	Hallucination, Grounding, and Faithful Explanations	50

8.4	Safety, Security, and Adversarial Threats	50
8.5	Decision Safety and Risk Control	51
8.6	Human Alignment and Auditability	51
8.7	Evaluation Protocols for Trustworthy Deployment	52
9	Benchmarks and Evaluation Protocols	52
9.1	Series-Only Foundation Benchmarks	53
9.2	Heterogeneous-Augmented End-to-End Benchmarks	54
9.3	From Label to Text: Time Series Understanding and Reasoning	55
9.4	Beyond Static Exam: Working on System-Level Agentic Tasks	56
9.5	Evaluation Boundaries: Mapping Benchmarks to the Layers	56
10	Applications of Agentic Time Series Systems Across Domains	57
10.1	General-Purpose Time Series Analysis	58
10.2	Finance and Trading	59
10.3	Traffic, Urban Systems, and Transportation	60
10.4	Weather, Energy, and Buildings	60
10.5	Healthcare	61
10.6	Industrial IoT, Observability, and Root Cause Analysis	62
10.7	Agriculture, Autonomous Systems, Retail, and Supply Chain	63
11	Positions and Frontiers	63
11.1	Verifiable Temporal Reasoning	64
11.2	End-to-End Agentic Training	65
11.3	Memory and Lifelong Adaptation	65
11.4	Temporal World Models	66
11.5	Decision-Centric Evaluation	66
11.6	Standardized Temporal Tooling	66
11.7	Position: Time Series Agents as Temporal Decision Systems	67
12	Conclusion	67

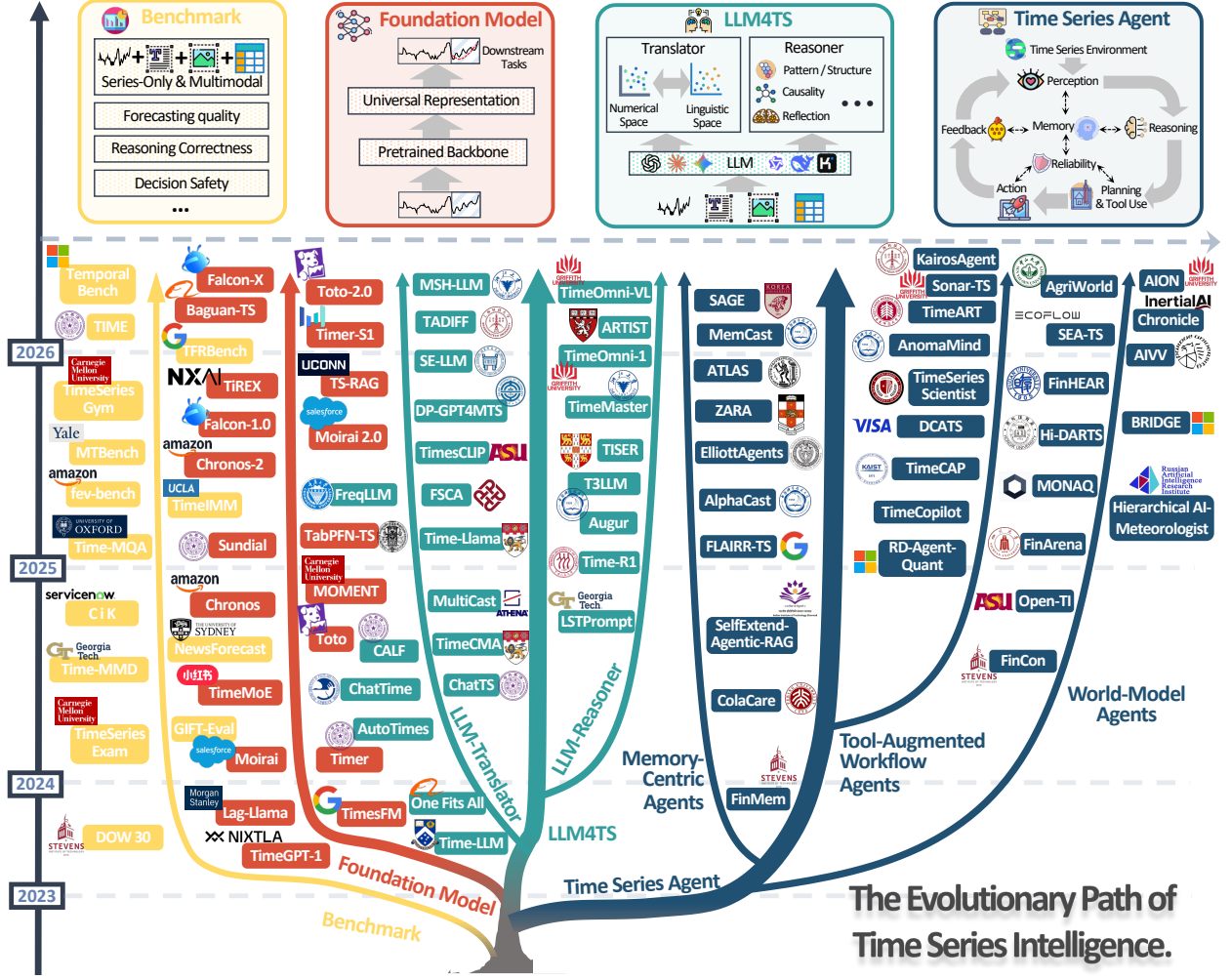


Figure 1 The evolutionary path of time series intelligence across four major research streams: benchmarks, foundation models, LLM4TS, and time series agents. The field is progressing from model-centric representation learning and evaluation toward language-mediated reasoning and, ultimately, closed-loop time series agents that can perceive, reason, plan, use tools, update state and act based on feedback. It further maps representative works into this taxonomy to provide a consolidated landscape.

1 Introduction

Time series data constitute one of the most pervasive interfaces through which computational systems observe the real world (Wang et al., 2026a; Kottapalli et al., 2025; Wen et al., 2022; Zhang et al., 2024; Chang et al., 2025). Financial markets evolve through prices, volumes, order books, and news-driven events (Zhu et al., 2024; Hu et al., 2025b; Cheng et al., 2022); healthcare systems continuously record physiological signals, laboratory measurements, and electronic health records (Soyiri and Reidpath, 2013; Vargas-Santiago et al., 2025; Patel and Rahimi, 2026); transportation networks generate traffic flows, mobility traces, and incident logs (Zhang et al., 2026c; Liu et al., 2023); industrial platforms collect sensor streams from machines, factories, and cyber-physical systems (Fatima and Rahimi, 2024; Hu and Man, 2023); and environmental sciences rely on weather, climate, energy, and ecological time series (Niu et al., 2025; Bi et al., 2022). Across these domains, the practical goal is rarely limited to predicting the next value of a sequence. Analysts and deployed systems must also inspect data quality, recognize temporal patterns, detect anomalies, relate observations to external events, select appropriate tools and models, assess uncertainty, explain results, and, in many cases, take actions whose consequences unfold over time. Time series analysis is therefore not only a predictive modeling problem, but also a sequential process of evidence acquisition, reasoning, decision-making, and adaptation.

The methodological history of time series research reflects a gradual expansion of what a system is expected to do. Classical statistical models and modern deep architectures (Zhou et al., 2021; Wu et al., 2021; Zhou et al., 2022; Zeng et al., 2023; Hu et al., 2025d, 2026, 2025c; Liu et al., 2024c) mainly focus on learning temporal dependencies for forecasting, classification, imputation, and anomaly detection. More recently, time series foundation models have introduced a stronger model-centric paradigm: large pretrained models (Liu et al., 2026c, 2025a; Das et al., 2023; Ansari et al., 2025; Khwaja et al., 2026) aim to provide reusable temporal representations, broad cross-domain transfer, and zero or few-shot generalization across heterogeneous datasets. This line of work substantially expands the scale, transferability, and general-purpose modeling capacity of time series systems. Nevertheless, its primary abstraction remains model-centric: the system learns a reusable temporal representation or predictive distribution, and then applies it to downstream tasks.

Large language models have further changed the design space of time series systems. A first family of methods uses LLMs as translators between numerical time series and linguistic or semantic spaces. Representative systems align time series with prompts, text prototypes, symbolic tokens, synthetic descriptions, visualizations, or multimodal representations, enabling numerical sequences to be interpreted through language interfaces (Zhou et al., 2023; Jin et al., 2024; Cao et al., 2023; Liu et al., 2024d, 2025b; Xie et al., 2024; Jiang et al., 2026). These approaches improve interaction, contextualization, and reuse of linguistic priors, but they usually remain centered on transforming inputs into predictions or explanations. A second family moves from translation to explicit reasoning. Recent work studies whether language models can reason about trends, periodicity, anomalies, similarity, temporal order, cross-channel dependencies, and causal associations; other systems introduce chain-of-thought (Cai et al., 2024; Yu et al., 2025a), slow-thinking (Cheng et al., 2026b), reinforcement learning (Luo et al., 2025b; Guan et al., 2025), process-verifiable reasoning (Messica et al., 2026), or reviewer-style correction (Lu et al., 2026) for time series tasks. This line marks an important transition: time series are no longer treated merely as numerical arrays to be encoded, but as temporal evidence over which systems must infer, justify, and sometimes revise conclusions.

However, translation and reasoning alone do not yet constitute agentic time series intelligence. A model could align time series with text, answer questions about a plot, or generate an explanation in a single pass, while still lacking the ability to decide what evidence should be acquired, which tool should be invoked, whether an intermediate result should be trusted, what should be remembered, and how future behavior should change after feedback (Cheng et al., 2026a; Kong et al., 2025b). This distinction becomes critical in realistic temporal environments. Time series are often non-stationary (Liu et al., 2025c; Ryan et al., 2025), feedback is frequently delayed (Long et al., 2024), and actions are sometimes costly or irreversible (Hu et al., 2025a; Li et al., 2026d). Furthermore, uncertainty accumulates across forecasting horizons and decision steps (Shi et al., 2024), while external events can abruptly invalidate historical patterns (Ge et al., 2025; Wang et al., 2025a). Under such conditions, a static input-output model, however strong, is insufficient as a general solution for temporal analysis and decision support.

This observation motivates the emerging paradigm of **agentic time series systems**. In this survey, we use this term to refer to closed-loop systems that operate in temporal environments by observing data and context, structuring temporal evidence, selecting tools or actions based on evolving states, receiving feedback, and updating subsequent behavior. More sophisticated systems further incorporate explicit reasoning, memory, world models, and verification under reliability constraints (Luo et al., 2025a; Wang et al., 2024a). *Recent systems expose these requirements through different capability layers.* At the perception layer, multimodal and context-aware agents transform raw series, plots, text, events, and domain context into structured temporal evidence (Ruan and Liang, 2026; Trirat et al., 2026; Lee et al., 2025; Sukhorukov et al., 2026). At the reasoning layer, tool-grounded and reflective systems infer temporal patterns, anomalies, causal hypotheses, and uncertainty from such evidence (Wu et al., 2026b; Liu et al., 2025d; Tao et al., 2026c,b). At the planning and action layer, workflow agents decompose temporal goals into executable pipelines, route tools or models, orchestrate data and code, and in some domains take external decisions (Zhao et al., 2025; Yeh et al., 2025; Li et al., 2026d; Xiao et al., 2024). At the memory and knowledge layer, retrieval- and experience-conditioned systems store historical patterns, prior cases, procedural traces, and failure signals for future adaptation (Yu et al., 2025b; Jalori et al., 2025; Zhang et al., 2026a; Tao et al., 2026a; Pan et al., 2026). At the world-model layer, emerging systems connect agents with simulation, world-tool protocols, counterfactual generation, and domain-constrained environments (Zhang et al., 2026b; Quinlan et al., 2026; Zhang et al., 2025b). Although these works vary in maturity and scope, they collectively suggest a shift from *model-centric prediction* to

closed-loop temporal intelligence. Crucially, they show that time series agency is not defined by any single mechanism such as tool use, RAG, multi-agent collaboration, or reinforcement learning, but by the integration of perception, reasoning, planning, memory, world modeling, and verification into an adaptive loop.

In parallel with the architectural evolution, benchmarks and datasets have developed as an independent but tightly coupled evaluation infrastructure. Early and broad forecasting benchmarks primarily assess predictive quality, generalization, calibration, and robustness of model-centric systems (Aksu et al., 2024; Shchur et al., 2025; Qiao et al., 2026). Multimodal, question-answering, and reasoning-oriented benchmarks evaluate whether LLM-enhanced systems can understand temporal patterns, align external context, and produce faithful explanations (Liu et al., 2024b; Kong et al., 2025a; Cai et al., 2024; Chen et al., 2025a; Weng et al., 2026; Ahamed et al., 2026b). More recent engineering, tool-use, and decision-centric benchmarks further examine whether agentic systems can construct workflows, invoke tools correctly, manage artifacts, reason under delayed feedback, and support safe decisions (Cai et al., 2025b; Nechepurenko and Shuvalov, 2026; Mostafa et al., 2026; Karger et al., 2025). Thus, benchmarks should not be viewed as a substage of time series models (Wang et al., 2026a). Rather, they form a diagnostic layer that evolves with the field: as systems move from forecasting models to LLM-for-time-series (LLM4TS) and then to time series agents, evaluation must also move from output accuracy toward reasoning correctness, tool-use reliability, decision safety, auditability, and deployment trustworthiness.

Taken together, these developments expose a conceptual gap in the current literature. Existing surveys on LLMs for time series have organized the field around prompting, tokenization, representation learning, alignment, forecasting performance, or multimodal interfaces (Zhang et al., 2025c, 2024; Shi et al., 2025a). Recent surveys and position papers have begun to emphasize reasoning and agentic forecasting (Kong et al., 2025b; Ning et al., 2025a; Chang et al., 2025; Cheng et al., 2026a). **Yet several boundaries remain insufficiently clarified.** ① Not every LLM-enhanced time series model is an agent. A backbone model, a translator, and a reasoner may all be essential building blocks, but they differ from systems that maintain state, use tools, update memory, and act under feedback. ② Existing categorizations often mix architectural capabilities with implementation mechanisms such as RAG, tool use, multi-agent collaboration, or reinforcement learning. These mechanisms are important, but they do not by themselves define the cognitive structure of a time series agent. ③ Evaluation is still fragmented: forecasting benchmarks, reasoning benchmarks, engineering benchmarks, and decision benchmarks typically stress different parts of the system, while deployed agents fail through interactions among perception, reasoning, tools, memory, and actions. Thus, a survey of agentic time series systems requires not only a list of methods, but a framework that explains *what capabilities are required, how they are connected, and how their failures propagate*.

Motivated by the above, this survey strives to establish a systematic framework for agentic time series systems that reconciles existing definitions across time series foundation models, LLM4TS methods, time series reasoners, and closed-loop agents. It also bridges emerging trends from multimodal alignment and tool-augmented reasoning to memory-centric and world-model-based agents. Furthermore, it elucidates the architectural and reliability principles required for trustworthy autonomous temporal intelligence. Specifically, we aim to address the following key questions:

Key Questions

- ① **Paradigm Boundary.** What distinguishes agentic time series systems from time series foundation models, LLM-based translators, and time series reasoners?
- ② **Agentic Architecture.** What core capabilities are necessary for a time series system to operate as a closed-loop agent in temporal environments?
- ③ **Reliability and Trustworthiness.** How should reliability be defined and evaluated for systems that perceive temporal evidence, reason over dynamics, use tools, update memory, and take actions?
- ④ **Evaluation and Applications.** What do existing benchmarks and real-world applications reveal about the current maturity, limitations, and deployment readiness of agentic time series systems?
- ⑤ **Future Frontiers.** What research directions are most critical for moving from agentic prototypes toward trustworthy autonomous temporal intelligence?

To answer these questions, we structure the survey as a sequence of conceptual steps. For question ❶, we separate four related but distinct paradigms: *LLM as Backbone*, *LLM as Translator*, *LLM as Reasoner*, and *Time Series Agent*, thereby clarifying when an LLM-enhanced time series model becomes a genuinely closed-loop agent. Question ❷ motivates our five-layer agentic architecture, moving from observing temporal evidence, to inferring dynamics, to acting through tools and workflows, to accumulating experience, and eventually to simulating environments. For question ❸, we position reliability and trustworthiness as a cross-cutting evaluative dimension that constrains every layer of the architecture, covering forecasting quality, reasoning correctness, tool-use reliability, hallucination and grounding, decision safety, human alignment, efficiency, auditability, and reproducibility. Question ❹ is examined by synthesizing existing benchmarks and applications through capability layers, evaluation signals, and deployment contexts, so as to identify which aspects of current systems are measurable and where evaluation remains fragmented. Building upon this assessment, question ❺ is explored through several emerging yet underdeveloped research frontiers, including reliable temporal reasoning, memory-centric adaptation, decision-centric evaluation, temporal world models, standardized tool protocols, and data-agent-based temporal data engineering.

Contributions This survey makes four main contributions. (1) We clarify the evolution from time series foundation models and LLM4TS methods to agentic time series systems, while treating benchmarks as a parallel evaluative infrastructure. This distinction separates backbone models, translator models, time series reasoners, and closed-loop agents. (2) We propose a five-layer architecture for time series agency, covering perception, reasoning, planning and action, memory and knowledge, and world models. (3) We position reliability and trustworthiness as a cross-cutting evaluative dimension, analyzing how failures may enter through temporal evidence, propagate through reasoning and tool use, affect actions, and be reinforced by memory or feedback. (4) Based on more than 200 papers, we synthesize representative methods, benchmarks, applications, and emerging frontiers, highlighting key gaps in evaluation, deployment, data-agent-based temporal data engineering, and temporal world modeling.

Outline of the Survey The remainder of this survey is organized as follows. [Sec. 2](#) formalizes agentic time series systems and distinguishes them from backbone, translator, and reasoner paradigms. [Sec. 3](#) discusses time series perception, including numerical, symbolic, structural, and multimodal evidence construction. [Sec. 4](#) reviews temporal reasoning, covering pattern, numerical, causal, reflective, and slow-thinking reasoning. [Sec. 5](#) examines planning and action, including workflow decomposition, tool routing, model/data/code orchestration, multi-agent coordination, and external decision-making. [Sec. 6](#) studies memory and knowledge mechanisms for accumulating temporal experience. [Sec. 7](#) discusses temporal world models, simulation, intervention, and counterfactual reasoning. [Sec. 8](#) develops a reliability and trustworthiness framework. [Sec. 9](#) surveys benchmarks and evaluation protocols. [Sec. 10](#) reviews applications across domains. [Sec. 11](#) discusses positions and future directions, and [Sec. 12](#) concludes the survey.

2 Preliminaries: Formalizing Agentic Time Series Systems

Recent surveys often group a wide range of systems under the broad label of “large language models for time series” ([Chang et al., 2025](#); [Zhang et al., 2024](#)). While this grouping is useful for capturing the influence of LLMs on time series research, it can obscure the specific characteristics of recent systems. A pretrained forecaster, a model that translates numerical sequences into language, a model that reasons over temporal patterns, and a system that runs an analytical workflow autonomously are not variants of the same design. They differ in terms of whether they maintain state, acquire evidence, invoke tools or actions, receive feedback, and update subsequent behavior. Richer systems could also incorporate explicit reasoning, memory, world models, and verification.

This section establishes the conceptual and mathematical principles that underpin the entire survey. First, we separate four paradigms according to whether the system operates as a closed loop ([Sec. 2.1](#)). We then define a time series agent as a closed-loop decision system situated in a temporal environment ([Sec. 2.2](#)). Next, we explain why this definition cannot be reduced to standard natural language processing (NLP)-agent templates, since time series agents operate under non-stationarity, delayed feedback, multi-scale dependency, numerical grounding requirements, and sometimes irreversible actions ([Sec. 2.3](#)). Finally, we map the formal

Table 1 Four paradigms of LLM-based time series systems. The key distinction lies in whether the system operates in a closed loop involving temporal evidence, tool/action selection, feedback, and state updates. Explicit reasoning, memory, world modeling, and verification are treated as progressively richer capabilities.

Paradigm	Closed loop	Core role	Representative systems	Core limitation
Pretrained Backbone	No	Learns temporal representations or predictive distributions directly from numerical series.	Chronos (Ansari et al., 2024), MOMENT (Goswami et al., 2024), TimesFM (Das et al., 2023)	Strong general-purpose forecasters, but no explicit tool use, memory, or action loop.
LLM as Translator	No	Maps time series into text, tokens, prompts, images, or semantic representations for an LLM.	Time-LLM (Jin et al., 2024), TEST (Sun et al., 2024), ChatTS (Xie et al., 2024)	Natural-language interfaces improve usability, but alignment errors can propagate downstream.
LLM as Reasoner	Partial	Performs explicit inference over temporal evidence, often through chain-of-thought, review, or reinforced reasoning.	TimeSeriesExam (Cai et al., 2024), PATRA (Lu et al., 2026), ARTIST (Messica et al., 2026)	Reasoning is visible, but the system often lacks persistent memory, tool-grounded action, and feedback-driven adaptation.
Time Series Agent	Yes	Maintains a closed-loop policy over temporal evidence, tool/action selection, feedback, and state update; may incorporate explicit reasoning, memory, world models, and verification.	TimeART (Wu et al., 2026b), Cast-R1 (Tao et al., 2026b), MemCast (Tao et al., 2026a), KairosAgent (Feng et al., 2026), TimeSeriesScientist (Zhao et al., 2025)	More autonomous and auditable, but more costly and harder to evaluate and reproduce.

object onto a five-layer agentic architecture, introducing reliability and trustworthiness as a vertical evaluative dimension cutting through all layers (Sec. 2.4). This provides the conceptual bridge from the paradigm evolution discussed in the introduction to the layer-wise taxonomy developed in the rest of the survey.

2.1 Four Paradigms of LLM-Based Time Series Systems

We distinguish between four related yet fundamentally different paradigms by asking a simple question: does the system merely produce an output, or does it maintain a closed loop involving evidence, action, feedback, and adaptation? Tab. 1 and Fig. 1 summarize this distinction.

The first paradigm uses an LLM-style multi-layer Transformer architecture as its *backbone*. Systems such as Chronos (Ansari et al., 2024), MOMENT (Goswami et al., 2024), and TimesFM (Das et al., 2023) tokenize, patch, scale, or otherwise encode raw numerical sequences and learn general-purpose temporal representations through large-scale pretraining. While they provide strong baselines for zero-shot and few-shot forecasting, the system typically makes a prediction in a single pass and then stops. The second paradigm uses the LLM as a *translator* between numerical, linguistic, and multimodal spaces, as demonstrated by Time-LLM (Jin et al., 2024), TEST (Sun et al., 2024), and ChatTS (Xie et al., 2024). They align series with text prototypes, soft prompts, symbolic tokens, or synthetic descriptions, enabling a frozen or fine-tuned language model to interpret them. This enlarges the interaction interface of time series modeling, but the system remains essentially an input-output mapper unless the translated evidence is embedded in a broader loop. The third paradigm uses the LLM as a *reasoner* that performs explicit inference over temporal evidence. It asks the model to infer temporal patterns, anomalies, similarities, causal hypotheses, or future dynamics, sometimes with explicit reasoning traces, as probed by TimeSeriesExam (Cai et al., 2024), and introduces chain-of-thought, review-and-correct, segment selection, or reinforcement-trained deliberation (Su et al., 2025; Messica et al., 2026; Parker et al., 2025). While reasoning is a genuine advance over translation, it is not sufficient for agency. A reasoner might still justify an answer without deciding what evidence to acquire next, which tool to use, what information to retain, or how to adjust its behaviour in response to feedback.

The fourth paradigm is the *time series agent* (Zhao et al., 2025; Wu et al., 2026b; Tao et al., 2026b,a; Ruan and Liang, 2026). Rather than being defined by the use of a particular mechanism, such as tool use, RAG, multi-agent collaboration or reinforcement learning, it is defined by the composition of multiple capabilities into a closed loop. A time series agent observes a temporal environment, constructs operational evidence, reasons over that evidence, selects tools or actions, observes feedback, verifies outcomes, and updates its memory in preparation for the next decision. The boundary between a temporal reasoner and a time series agent is therefore architectural rather than terminological.

2.2 Formalizing the Time Series Agent

Having separated the four paradigms, we now present the subject of study for the remainder of this survey. A time series agent cannot be defined merely by the presence of an LLM, a reasoning trace, or a tool call. Rather, it is a closed-loop system operating in a temporal environment, where the system maintains an evolving state, acquires temporal evidence, selects tools or actions, observes feedback, and updates its subsequent behavior. Importantly, this definition does not require every agentic time series system to implement all advanced capabilities. Some agents may rely on implicit policies rather than explicit reasoning, some may use only short-term context rather than persistent memory. Currently, only a small subset approximates temporal world modeling. We therefore formalize a time series agent as a functional closed-loop system:

$$\mathcal{A}_{TS} = \langle \mathcal{P}, \mathcal{C}; \mathcal{R}, \mathcal{M}, \mathcal{W} \rangle_{\mathcal{E}_{TS}}, \quad (1)$$

where $\mathcal{P}$ and $\mathcal{C}$ denote the minimal operational core of an agent in a temporal environment: perception and planning/action. The remaining components, $\mathcal{R}$, $\mathcal{M}$, and $\mathcal{W}$, denote optional but increasingly important capability layers: explicit time series reasoning, memory and knowledge, and time series world modeling. The subscript $\mathcal{E}_{TS}$ emphasizes that the agent is defined relative to a temporal environment with evolving observations, feedback, and constraints. The above five components are the architectural layers examined in Secs. 3 to 7. Verification is deliberately excluded from the tuple: rather than being a required component of the agent, it is treated as a cross-cutting reliability mechanism, denoted by $\mathcal{V}$ when needed, and studied systematically in Sec. 8.

This formulation separates the minimal definition of agency from the richer capability architecture used to organize the literature. The perception component $\mathcal{P}$ maps raw numerical series, aligned context, plots, tables, and other multimodal observations into structured, queryable evidence (Sec. 3). The planning-and-action component $\mathcal{C}$ maps the current temporal state and evidence into tool calls, model invocations, data or code operations, workflow decisions, or external actions (Sec. 5). Explicit reasoning $\mathcal{R}$, when present, infers patterns, numerical relations, anomalies, causal hypotheses, uncertainty, or explanations from the evidence (Sec. 4). Memory and knowledge $\mathcal{M}$, when present, store and retrieve cases, regimes, procedures, failures, and domain knowledge across time scales (Sec. 6). Temporal world modeling $\mathcal{W}$, when present, represents environment mechanisms so that the agent can simulate possible futures, evaluate interventions, and answer counterfactual questions rather than only extrapolate observed histories (Sec. 7). These capabilities are not homogeneous software modules; they are functional roles that may be implemented explicitly, implicitly, or only partially in current systems.

Concretely, the agent maintains a temporal information state

$$s_t = \langle x_{1:t}, c_{1:t}, z_t, \eta_t, b_t, \Omega_t \rangle, \quad (2)$$

where $x_{1:t}$ denotes the observed series, $c_{1:t}$ the aligned temporal context or external events, z_t an inferred regime or latent environment state, η_t the agent’s internal state, b_t its belief or uncertainty state, and Ω_t the available tool and action space. The internal state η_t may include short-term context, persistent memory, workflow traces, retrieved knowledge, or policy state, depending on the system design. One generic closed-loop step can then be written as

$$e_t = \mathcal{P}(x_{1:t}, c_{1:t}; s_t), \quad (3)$$

$$r_t = \mathcal{R}_{\text{opt}}(e_t, \eta_t, \mathcal{W}_{\text{opt}}), \quad (4)$$

$$(u_t, a_t) \sim \mathcal{C}(\cdot \mid s_t, e_t, r_t, \Omega_t), \quad y_{t+\Delta} = \text{Env}(a_t, u_t), \quad (5)$$

$$\eta_{t+1} = \mathcal{U}(\eta_t, \langle e_t, r_t, u_t, a_t, y_{t+\Delta} \rangle), \quad s_{t+1} = \mathcal{T}(s_t, a_t, u_t, y_{t+\Delta}). \quad (6)$$

Here e_t is structured temporal evidence, r_t is an optional explicit reasoning state, u_t is a tool invocation, a_t is an analytic or external action, and $y_{t+\Delta}$ is the possibly delayed feedback returned by the temporal environment. The notation $\mathcal{R}_{\text{opt}}$ and $\mathcal{W}_{\text{opt}}$ emphasizes that explicit reasoning and world modeling need not be present in every time series agent. Similarly, if an explicit memory module is implemented, the update of η_t may include a memory update such as

$$m_{t+1} = \mathcal{M}(m_t, \langle e_t, r_t, u_t, a_t, y_{t+\Delta} \rangle), \quad (7)$$

but externalized memory is not required for the minimal definition of an agent. What is required is that the system’s next decision can depend on the evolving temporal state rather than only on the original input.

Verification and reliability enter as constraints or evaluators over the trajectory, not as an additional layer:

$$v_t = \mathcal{V}(s_t, e_t, r_t, u_t, a_t, y_{t+\Delta}). \quad (8)$$

The verifier may be an internal self-check, an external evaluator, a statistical diagnostic, a human review process, or a safety constraint. Its role is to assess whether evidence is grounded, reasoning is faithful, tools are used correctly, actions are safe, memory updates are auditable, and simulations are valid. This is why [Sec. 8](#) treats reliability and trustworthiness as a system-level dimension rather than as a component that every time series agent must explicitly implement.

This formalization yields a more flexible boundary between the paradigms in [Sec. 2.1](#). A backbone model, translator, or temporal reasoner may become a component inside a time series agent, but it is not itself an agent unless it participates in a closed-loop policy over temporal evidence, tools or actions, feedback, and state update. Conversely, a practical time series agent need not already possess full persistent memory or a mature world model. Such capabilities represent higher-order extensions of the agentic stack. Representative systems already instantiate different subsets of this abstraction, including tool-grounded reasoning in TimeART ([Wu et al., 2026b](#)) and TS-Agent ([Liu et al., 2025d](#)), workflow planning in TimeSeriesScientist ([Zhao et al., 2025](#)), sequential decision policies in Cast-R1 ([Tao et al., 2026b](#)), memory-augmented forecasting in MemCast ([Tao et al., 2026a](#)), and executable world-tool environments in AgriWorld ([Zhang et al., 2026b](#)). The purpose of the five-layer architecture is therefore not to impose a necessary checklist, but to provide a capability map for understanding how current systems approach closed-loop temporal intelligence.

2.3 Time Series Agents $\neq$ NLP Agents

Time series agents and NLP agents share a common closed-loop abstraction: both can maintain histories, invoke tools, observe intermediate results, and update subsequent actions. The key distinction lies in the environments. Time series agents instantiate the same broad agentic abstraction in environments with more demanding temporal dynamics: observations are numerical and multi-scale, distributions drift over time, feedback can be delayed, actions may be costly or irreversible, and claims must often be grounded in statistical or causal evidence. Thus, the difference is that the evolving temporal environments place pressure on different parts of the agent loop, including observation modeling, transition dynamics, reward or feedback structure, memory update, planning constraints, and reliability evaluation.

This distinction can be expressed through a shared closed-loop template. A general tool-using agent can be written abstractly as

$$(u_t, a_t) \sim \pi(u, a \mid \eta_t, o_t, \Omega_t), \quad \eta_{t+1} = \mathcal{T}_{\mathcal{E}}(\eta_t, o_t, u_t, a_t, y_{t+\Delta}), \quad (9)$$

where η_t is the agent’s history or internal state, o_t is the current observation, Ω_t is the available tool and action space, u_t is a tool invocation, a_t is an action, and $y_{t+\Delta}$ is feedback from the environment. This abstraction can describe both NLP agents and time series agents. The distinction arises from the structure of o_t , the transition $\mathcal{T}_{\mathcal{E}}$, the delay Δ , the nature of the feedback $y_{t+\Delta}$, and the constraints imposed on actions. For an NLP agent, observations are often symbolic artifacts such as retrieved passages, code outputs, web pages, API responses, or dialogue turns. For a time series agent, the observation state includes evolving numerical series, aligned events, regimes, uncertainty, temporal context, and tool outputs:

$$s_t = \langle x_{1:t}, c_{1:t}, z_t, \eta_t, b_t, \Omega_t \rangle. \quad (10)$$

Table 2 How temporal environments change the requirements of closed-loop agents. The differences are best understood as pressures on the observation, transition, feedback, policy, memory, and reliability components of a shared agentic abstraction.

Temporal property	Where it affects the agent loop	Requirement for time series agents	Representative systems / benchmarks
Non-stationarity	Observation and transition dynamics change over time.	Detect regime shifts, update retrieval and memory, and route tools or models under drift.	MemCast (Tao et al., 2026a); FinMem (Yu et al., 2025b); TemporalBench (Weng et al., 2026)
Delayed feedback	Reward, error, or outcome signals arrive after the environment evolves.	Maintain trajectories, assign delayed credit, and verify decisions after outcomes are observed.	Cast-R1 (Tao et al., 2026b); TradingAgents (Xiao et al., 2024); CryptoTrade (Li et al., 2024)
Costly or irreversible action	The action space is constrained by risk, cost, and safety.	Constrain policies by confidence, risk budgets, human oversight, and pre-execution checks.	FinArena (Xu et al., 2025c); Hubble (Shi et al., 2026a); AIVV (Kwon et al., 2026)
Uncertainty accumulation	Beliefs and errors propagate across horizons, tools, and decisions.	Track calibration, propagate uncertainty, and trigger replanning, abstention, or fallback policies.	Cast-R1 (Tao et al., 2026b); TimeCopilot (Garza and Rosillo, 2025); Foresight Arena (Nechepurenko and Shuvalov, 2026)
Multi-scale and multimodal evidence	The observation model must align heterogeneous signals across time scales.	Align series, events, logs, charts, tables, and domain signals across windows, frequencies, and regimes.	AI-Meteorologist (Sukhorukov et al., 2026); AGCD (Wu et al., 2026a); HEARTS (Li et al., 2026c)
Numerical and statistical grounding	Reliability depends on executable checks rather than textual plausibility alone.	Ground claims about trends, lags, thresholds, anomalies, and causality in diagnostics and raw series.	TimeART (Wu et al., 2026b); TS-Agent (Liu et al., 2025d); TS-Reasoner (Ye et al., 2026)

This state is not merely a larger prompt; it encodes the temporal structure of the environment. The agent must reason about what changes, when it changes, at which scale, under which regime, and with what delayed consequence.

The literature reflects this difference in where the closed-loop pressure appears. Systems aligned with the translator and reasoner paradigms of [Sec. 2.1](#), such as Time-LLM (Jin et al., 2024), TEST (Sun et al., 2024), ChatTS (Xie et al., 2024), PATRA (Lu et al., 2026), and TimeSeriesExam (Cai et al., 2024), improve representation alignment or explicit temporal reasoning, but they do not necessarily model feedback, action constraints, or temporal state evolution. In contrast, systems such as TimeSeriesScientist (Zhao et al., 2025), TimeART (Wu et al., 2026b), Cast-R1 (Tao et al., 2026b), TS-Agent (Liu et al., 2025d), TS-Reasoner (Ye et al., 2026), TimeCopilot (Garza and Rosillo, 2025), MemCast (Tao et al., 2026a), MAS4TS (Ruan and Liang, 2026), FinMem (Yu et al., 2025b), TradingAgents (Xiao et al., 2024), and AgriWorld (Zhang et al., 2026b) instantiate different parts of the temporal agent loop, including task-time perception, tool use, planning, memory, verification, environment interaction, and policy learning. The distinction is therefore not whether a system calls itself an agent, but where it sits in the closed-loop interaction between temporal evidence, tools, actions, feedback, and state update.

Table 2 summarizes how temporal environments alter the standard agent loop. The comparison is not meant to imply that NLP agents are simple, or that all time series agents already solve these challenges. Rather, it identifies where temporal dynamics impose additional requirements on otherwise familiar agentic components.

These properties change the role of reasoning and memory within the loop. In many NLP agents, reasoning decomposes a query, synthesizes retrieved evidence, or plans symbolic tool use. In time series agents, reasoning must often be coupled to temporal operators such as decomposition, autocorrelation analysis, change-point detection, frequency analysis, cross-correlation, causal-lag discovery, backtesting, and rolling validation. Because numerical claims about trends, thresholds, lags, anomalies, or causal delays can be checked against

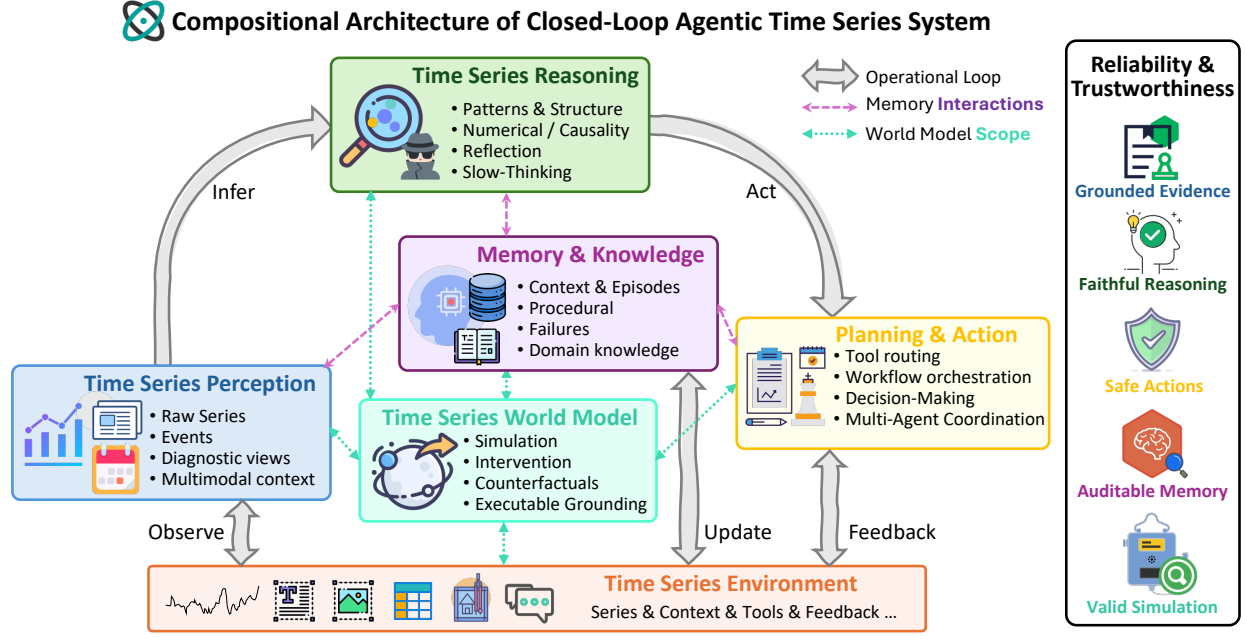


Figure 2 The compositional architecture of the closed-loop agentic time series system. The system coordinates time-series perception, reasoning, planning and action, memory and knowledge, and temporal world modeling around interaction with the temporal environment. Reliability and trustworthiness provide cross-cutting constraints on evidence grounding, reasoning faithfulness, action safety, memory auditability, and simulation validity.

the raw series, tool grounding is not optional for high-stakes temporal decisions. TimeART (Wu et al., 2026b), TS-Agent (Liu et al., 2025d), and TS-Reasoner (Ye et al., 2026) bind reasoning steps to executed observations rather than relying only on free-form textual rationales, while Cast-R1 (Tao et al., 2026b) reformulates forecasting as a sequential reasoning and decision process. The same logic extends to multimodal temporal context, where prices must be read together with news, sensor streams with logs, and electronic health records with clinical knowledge, as in TimeCAP (Lee et al., 2025) and ColaCare (Wang et al., 2025b).

The deployment contrast further illustrates the environmental source of the difference. Financial agents must combine market series, news, filings, and delayed portfolio feedback, which is why FinMem (Yu et al., 2025b), FinCon (Yu et al., 2024), TradingAgents (Xiao et al., 2024), ATLAS (Papadakis et al., 2025), CryptoTrade (Li et al., 2024), Hubble (Shi et al., 2026a), and FinArena (Xu et al., 2025c) introduce memory, debate, risk control, or human intervention rather than emitting a single number. In healthcare, ColaCare (Wang et al., 2025b) grounds electronic-health-record reasoning in retrieved clinical knowledge; in traffic and energy, Open-TI (Da et al., 2024) turns forecasts and context into operational policies; and in agriculture, AgriWorld (Zhang et al., 2026b) embeds the agent in an executable world-tools environment where conclusions can be checked against simulators. These domains are examined in Sec. 10; here they demonstrate that the challenge is not simply to adapt LLM agents to numerical inputs, but to design agents whose policies, transitions, memory updates, and reliability checks are appropriate for temporal environments.

Taken together, these differences motivate the definition adopted throughout this survey: a time series agent is a closed-loop agent operating in a temporal environment, rather than an LLM wrapper for time series inputs. The minimal operational requirements are task-time perception of temporal evidence and planning or action selection over an evolving state. Explicit reasoning, persistent memory, temporal world modeling, and verification are not mandatory for every prototype, but they are the capability directions that make time series agents progressively more autonomous, adaptive, and trustworthy. Systems that lack a closed-loop policy over temporal evidence, tools or actions, feedback, and state update may still be valuable backbone models, translators, or temporal reasoners in the sense of Sec. 2.1, but they should not be conflated with agentic time series systems. The layered architecture developed from Sec. 3 to Sec. 7 is organized around these capabilities, while Sec. 8 studies how their failures should be detected, constrained, and made auditable.

2.4 The Five-Layer Architecture

The formal object $\mathcal{A}_{TS}$ organizes the remainder of the survey into five capability layers and one cross-cutting reliability dimension. These layers should be viewed as a capability map, rather than as a rigid checklist. Perception and planning/action constitute the minimal operational core of a time series agent, since the agent must observe temporal evidence and select tools or actions in response to an evolving state. Explicit reasoning, memory and knowledge, and temporal world modeling further enrich this core by enabling the agent to infer temporal dynamics, accumulate reusable experience, and simulate or reason about alternative futures. The value of existing systems should not be judged by whether they implement all layers, but by how effectively their implemented components support closed-loop temporal interaction. Implementation mechanisms such as tool use, RAG, multi-agent collaboration, reinforcement learning, code execution, and procedural memory may appear across different layers. They are mechanisms for realizing capabilities, not substitutes for the capability layers themselves. Fig. 2 visualizes this concept as a compositional architecture, in which the five components coordinate around a closed-loop interaction with the temporal environment. Meanwhile, reliability and trustworthiness provide cross-cutting constraints on evidence grounding, reasoning faithfulness, action safety, memory auditability, and simulation validity.

Layer A: Time Series Perception (Sec. 3) is the process by which an agent renders a temporal environment observable. It creates operational evidence from raw numerical values, symbolic descriptions, structural relations, diagnostic views, plots, tables, documents, events, and other multimodal observations. Unlike conventional representation learning, perception in an agentic system is task-conditioned. The agent must decide which signals are relevant, what temporal resolution is appropriate, which modalities should be aligned, which details should be preserved or abstracted, and which observations are reliable enough to support later reasoning. This layer answers the question of *what should become visible to the agent, and in what form*.

Layer B: Time Series Reasoning (Sec. 4) transforms structured evidence into statements concerning temporal patterns, numerical relationships, anomalies, causal hypotheses, uncertainty, and potential futures. It includes pattern and structural reasoning, statistics-grounded reasoning, causal and compositional reasoning, reflective reasoning, and slow-thinking or reinforcement-enhanced reasoning. Unlike generic language reasoning, the conclusions of reasoning layer must remain grounded in temporal order, scale, lag, frequency, regime, and numerical magnitude. A time series agent should not only produce plausible explanations, it must also generate claims that can be verified against observations, diagnostic tools, retrieved events, and downstream feedback. This layer addresses the question of *what can be inferred from temporal evidence, and how the inference can be justified*.

Layer C: Planning and Action (Sec. 5) converts goals and reasoning states into executable, revisable, and auditable workflows. It includes task decomposition, tool routing, evidence acquisition, model selection, data and code orchestration, multi-agent coordination, sequential decision-making, and external actions in domains such as finance, traffic, healthcare, energy, and industrial control. Such layer most clearly distinguishes an agent from a passive reasoner because the system not only infers what is happening, but also decides what to do next, when to gather more evidence, when to switch tools, when to request human intervention, and when to act on the environment. This layer answers the question of *what the agent should do next, with which tool or policy, and under what constraints*.

Layer D: Memory and Knowledge (Sec. 6) determines whether a time series agent accumulates experience or merely reacts to the current inputs. It stores and retrieves contextual evidence, episodic cases, long-term temporal regimes, domain knowledge, procedural traces, failure patterns, and confidence signals. The purpose of this layer is not to keep a raw transcript of previous interactions, but to preserve reusable temporal experience, including information such as which regimes resemble the current one, which tools worked under similar conditions, which reasoning paths failed, which domain rules constrain the task, and when old experience should be updated or forgotten. This layer determines *what the agent should remember, reuse, revise, or discard over time*.

Layer E: Time Series World Models (Sec. 7) represent the frontier at which time series agents transition from extrapolating future values to modeling the mechanisms of dynamic environments. A world model should support simulation, intervention analysis, counterfactual reasoning, and interaction with the environment. This layer is not required by all current systems, and existing work only begins to approximate it in domain-specific or partially simulatable settings. Nevertheless, it is essential for agents that must compare possible futures,

reason about actions never taken, test interventions before deployment, or explain why regimes change. This layer addresses *what would happen if the environment, evidence, or action were different*.

Reliability and Trustworthiness (Sec. 8) instantiate the verification operator $\mathcal{V}$ introduced in Sec. 2.2. In Fig. 2, they are depicted as a vertical evaluative dimension, as each layer introduces distinct failure modes. Evidence can be distorted by perception, reasoning can create false patterns or causal relations, planning can select inappropriate tools or unsafe actions, memory can retrieve outdated or misleading information, and world models can simulate implausible futures. Therefore, reliability cannot be reduced to a final forecasting metric. At every layer, it must ask whether evidence is grounded, reasoning is faithful, tools are used correctly, actions satisfy constraints, memory is auditable, simulations are valid, and the overall trajectory is reproducible. The following sections develop the five layers in turn, beginning with perception. Meanwhile, Sec. 8 returns to the reliability dimension as a system-level requirement for trustworthy deployment.

3 Time Series Perception: Observing and Structuring Temporal Environments

Time series perception is the layer through which an agent makes a temporal environment observable. Unlike conventional time series representation learning, whose primary goal is to encode historical values into features for a predefined predictive task, perception in an agentic time series system concerns a broader and more operational question: **what aspects of a temporal environment should become visible, through which observation interface, and with what level of fidelity, context, and reliability** (Goswami et al., 2024; Ansari et al., 2024)? This distinction is central because a time series agent rarely observes a clean, isolated, and regularly sampled sequence. Instead, it is situated in a temporal environment composed of raw numerical trajectories, irregular events, contextual documents, tabular records, visual charts, sensor streams, weather reports, electronic health records, logs, and domain constraints (Xie et al., 2024; Chen et al., 2026). To make such an environment usable, the agent must first transform heterogeneous observations into a structured evidence state that records not only what is observed, but also how it is observed, aligned, summarized, and qualified.

We therefore view time series perception as *task-conditioned evidence construction*. At a given task moment, the agent must determine which signals are relevant, which temporal resolution is appropriate, which observation tools should be applied, how different modalities should be aligned, which observations are reliable, and which details should be preserved or abstracted (Zhao et al., 2025; Liu et al., 2025d). For example, the same traffic sequence may be perceived as a raw numerical trajectory, a recurring daily pattern, an abnormal interval, a weather-conditioned event, or a spatially propagated congestion state depending on the task context. Similarly, a sudden change in a physiological signal may be perceived as noise, a patient-specific baseline shift, an abnormal episode, or a context-dependent clinical observation depending on the available measurements, diagnostic views, and patient history. Thus, perception is not a passive preprocessing step; it is the process that determines the observable state of a temporal environment.

This perspective also clarifies why perception has become a foundational layer in current agentic time series systems. Many existing systems begin by bridging time series with language, tools, structures, vision, tables, or external context (Xie et al., 2024). However, much of the current literature still treats this problem primarily as modality bridging: converting time series into tokens, patches, text, images, embeddings, or prompts that can be consumed by foundation models (Jin et al., 2024; Goswami et al., 2024). While such bridging is necessary, it is not sufficient for agentic perception. An agent requires not only a representation, but an evidence state that is inspectable, temporally grounded, uncertainty-aware, tool-checkable, and robust to missingness, noise, distribution shift, and conflicting context (Wu et al., 2026b).

Formally, let $\mathcal{O}_t$ denote the set of raw observations available to the agent at task step t :

$$\mathcal{O}_t = \{X_t, U_t, D_t, S_t, B_t, \Gamma_t\}, \quad (11)$$

where X_t denotes raw numerical time series signals, U_t denotes external contextual inputs such as news, logs, weather reports, clinical notes, or domain documents, D_t denotes diagnostic observations such as decomposition results, autocorrelation statistics, missingness profiles, anomaly candidates, or frequency-domain summaries, S_t denotes structural observations among variables, entities, sensors, or temporal segments, B_t denotes visual or multimodal artifacts such as plots, tables, dashboards, images, or electronic health records, and Γ_t

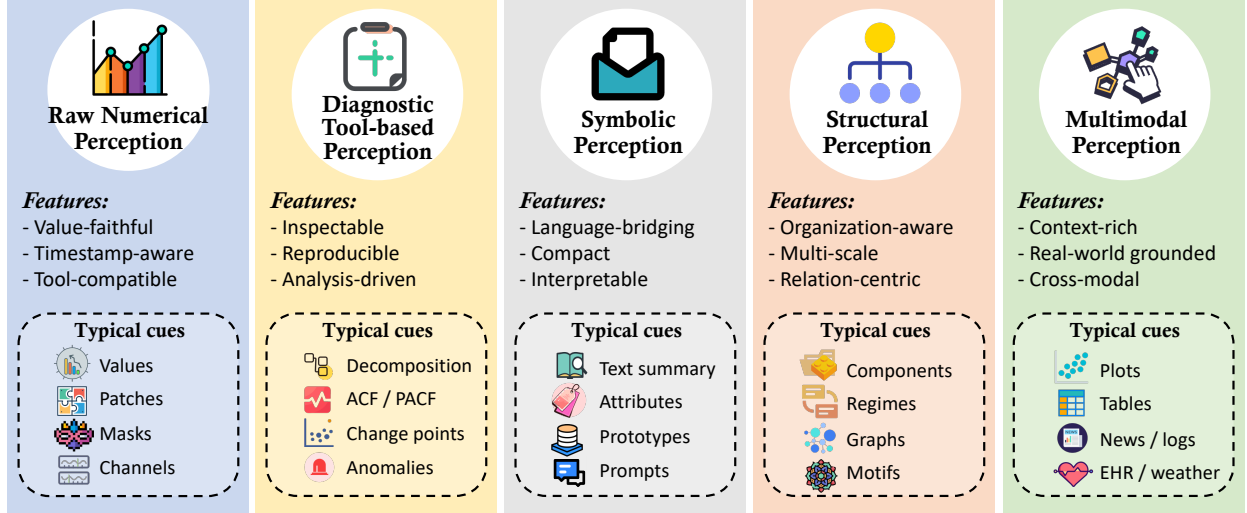


Figure 3 Taxonomy of time series perception in agentic systems. Perception can be categorized as raw numerical, diagnostic tool-based, symbolic, structural, and multimodal forms, each exposing different cues such as values, decompositions, text summaries, regimes, graphs, motifs, plots, tables, and contextual records. These cues transform heterogeneous temporal observations into structured evidence for the follow-up process.

denotes task constraints or domain priors. The perception layer maps these heterogeneous observations into a structured evidence state:

$$E_t = \Phi_{\theta}(\mathcal{O}_t \mid g, m_t, \eta_{<t}), \quad (12)$$

where g is the task goal, m_t is memory, and $\eta_{<t}$ is the prior interaction trace. The evidence state E_t may contain normalized sequences, temporal patches, diagnostic summaries, symbolic descriptions, prototypes, attribute labels, decomposed temporal components, candidate relational or topological structures, visual interpretations, table schemas, multimodal alignments, uncertainty estimates, and links to supporting sources. Rather than being a single embedding, E_t is a structured perceptual record: it specifies which aspects of the temporal environment have been observed, which abstractions have been introduced, and what reliability signals accompany them.

The rest of this section examines five complementary perceptual interfaces as in [Fig. 3](#). **Raw numerical perception** preserves values and local temporal variation with minimal semantic transformation. **Diagnostic tool-based perception** exposes intermediate analytical views such as decomposition, autocorrelation, missingness, and anomaly localization. **Symbolic perception** converts selected temporal properties into textual, semantic, or prototype-based forms. **Structural perception** makes components, regimes, motifs, relations, and graph-like organization explicit. **Multimodal perception** aligns numerical series with external artifacts such as charts, tables, logs, weather, remote sensing data, or medical records. These interfaces are not stages in a fixed pipeline; they are alternative ways of making different parts of the temporal environment observable.

3.1 Raw Numerical Perception

Raw numerical perception preserves the time series as a numerical object. It is the most faithful perceptual interface because the evidence exposed to the agent remains close to the original observations: values, timestamps, channels, windows, patches, masks, and discretized tokens ([Ansari et al., 2024](#); [Goswami et al., 2024](#)). For a univariate sequence, this evidence may simply be a normalized trajectory or a set of local windows. For a multivariate sequence, it may additionally include channel identities, sampling intervals, missing-value masks, and local cross-channel co-variation ([Woo et al., 2024](#)). The central question is therefore not how to interpret the signal semantically, but how much of the original numerical structure can be retained while making the sequence usable by a model or agent ([Das et al., 2023](#); [Liu et al., 2024e](#)).

Recent time series foundation models instantiate this interface through tokenization, patching, quantization, masked encoding, and large-scale temporal pretraining. Although these models are not agents, they provide

raw numerical perceptual substrates that preserve temporal values, patches, and token-level evidence for downstream agentic workflows. Chronos converts real-valued time series into discrete tokens through scaling and quantization, allowing forecasting to be cast as a sequence modeling problem over tokenized numerical observations (Ansari et al., 2024). MOMENT uses masked time series modeling to learn general-purpose representations across tasks and domains, exposing the signal through masked reconstruction and temporal embeddings (Goswami et al., 2024). TimesFM, Moirai, Lag-Llama, and Timer similarly represent time series through patches, windows, decoder-style sequence modeling, or universal forecasting architectures (Das et al., 2023; Woo et al., 2024; Rasul et al., 2023; Liu et al., 2024e). Although these systems are not necessarily agents by themselves, they provide an important perceptual substrate for agentic systems: they turn raw temporal values into reusable numerical evidence that can be queried, forecasted, masked, reconstructed, or compared.

For an agent, raw numerical perception is useful because it preserves quantities that later claims can be checked against. A trend description, anomaly label, or forecast explanation remains auditable only if the underlying values, windows, and timestamps can still be recovered (Liu et al., 2025d). This makes raw numerical perception particularly important in settings where magnitude, timing, and calibration matter, such as threshold-based monitoring, medical signals, market prices, energy loads, and industrial sensors. It also supports tool compatibility: statistical diagnostics, forecasting models, anomaly detectors, and backtesting procedures usually require numerical inputs rather than purely textual descriptions.

The limitation is that numerical fidelity does not imply semantic observability. A sequence of values may accurately preserve the signal while still hiding what the agent needs to know: whether a local change is meaningful, whether a pattern is seasonal, whether two variables interact, or whether an event explains a deviation. Tokenization and patching can also introduce their own perceptual distortions (Ansari et al., 2024; Nie et al., 2022). Quantization may blur small but important changes; fixed-length patches may cut across meaningful events; normalization may remove level information; masked reconstruction objectives may favor generic temporal smoothness over task-specific evidence (Goswami et al., 2024). Thus, raw numerical perception is necessary but insufficient: it gives the agent a faithful view of the signal, but not necessarily an interpretable view of the temporal environment.

3.2 Diagnostic Tool-based Perception

Diagnostic tool-based perception exposes properties of the time series that are not directly visible from raw values alone. Instead of asking the agent to inspect the sequence in its original form, this interface invokes analytical procedures that transform the signal into intermediate diagnostic evidence (Wu et al., 2026b; Liu et al., 2025d). Such evidence may include trend and seasonal components, autocorrelation profiles, partial autocorrelation, missingness patterns, change points, anomaly intervals, frequency-domain summaries, stationarity indicators, or data-quality reports (Cleveland et al., 1990). The perceptual object is therefore no longer just the observed sequence, but a set of tool-produced views that reveal latent temporal properties and possible defects in the data.

Classical time series diagnostics provide the basic vocabulary for this interface. Decomposition methods such as STL separate trend, seasonal, and residual components, making it easier to observe whether a series changes through long-term drift, periodic variation, or irregular disturbance (Cleveland et al., 1990). ACF and PACF expose correlation structures across lags, which are useful for identifying periodicity, persistence, or autoregressive behavior (Box et al., 2015). Change-point detection localizes regime shifts or distributional breaks (Killick et al., 2012). Missingness profiling reveals whether gaps are random, systematic, or concentrated in critical intervals (Little and Rubin, 2019; Rubin, 1976). Anomaly localization identifies point-level, interval-level, or contextual deviations (Chandola et al., 2009). Frequency analysis exposes periodic components that may be obscure in the time domain (Brillinger, 2001). When these tools are invoked inside an agentic workflow, they function as perception operators: they do not by themselves produce final conclusions, but they make otherwise hidden temporal evidence observable.

Agentic systems increasingly adopt such tool-based perception. TimeSeriesScientist decomposes time series analysis into stages such as data curation, planning, forecasting, and reporting, where intermediate diagnostics help determine how the series should be processed (Zhao et al., 2025). TimeART and TS-Agent ground temporal analysis in executable tool calls and intermediate observations, allowing the system to obtain statistical or diagnostic evidence before producing an answer (Wu et al., 2026b; Liu et al., 2025d). In these

systems, diagnostic tools act as perceptual instruments: they determine what aspects of the signal become visible to the agent at task time.

The strength of diagnostic perception is that it is interpretable and often auditable. A decomposition plot, a change-point score, or a missingness profile is easier to inspect than a latent embedding (Cleveland et al., 1990). It also supports reproducibility because the same tool can be rerun on the same window, with explicit parameters and outputs. This makes diagnostic perception well suited for data cleaning, model selection, anomaly triage, quality control, and exploratory time series analysis. It also helps separate perceptual evidence from generated explanation: an agent can report that a seasonal component was extracted by a specific tool rather than merely asserting that the series “looks seasonal.”

However, tool-based perception inherits the assumptions and biases of the tools it invokes. STL assumes a particular decomposition structure; ACF/PACF can be misleading under nonstationarity or strong nonlinear effects; change-point detectors depend on window size, penalty choice, and distributional assumptions; anomaly detectors can confuse rare but valid events with faults; missing-value handling can alter downstream diagnostics (Cleveland et al., 1990; Box et al., 2015; Aminikhanghahi and Cook, 2017). Preprocessing choices such as resampling, smoothing, normalization, and interpolation may also change what the agent perceives. Thus, diagnostic perception should be treated as a view of the temporal environment rather than the environment itself. Its outputs are valuable because they are explicit, but they remain contingent on tool choice, parameterization, and data quality.

3.3 Symbolic Perception

Symbolic perception converts temporal patterns into language-compatible or discrete semantic forms. The evidence exposed to the agent is no longer only a sequence, patch, or diagnostic score, but a symbolic object such as a textual summary, attribute label, prototype, trend descriptor, event tag, prompt token, or natural-language description (Xue and Salim, 2023; Gruver et al., 2023). This interface is especially important for LLM-based time series systems because language models operate most naturally over symbolic sequences (Jin et al., 2024; Sun et al., 2024). Symbolic perception therefore serves as a bridge between numerical temporal signals and human- or language-readable descriptions (Xie et al., 2024).

Several mechanisms instantiate symbolic perception. Textualization converts time series observations into natural-language statements, such as descriptions of trend direction, volatility, periodicity, outliers, or local changes. Prototype-based methods align time series segments with representative temporal shapes or semantic categories. Attribute synthesis extracts descriptors such as “increasing,” “periodic,” “volatile,” “flat,” “spiky,” or “regime-changing.” Prompt-based and soft-prompt approaches map temporal information into prompt tokens or continuous embeddings that can condition an LLM. Systems such as Time-LLM, TEST, PromptCast, LLMLTime, ChatTS, and DualSG explore different ways of making time series information accessible to language models through reprogramming, prompts, textual descriptions, time series captions, or time-series-to-language interfaces (Jin et al., 2024; Sun et al., 2024; Xue and Salim, 2023; Gruver et al., 2023; Xie et al., 2024; Ding et al., 2025).

The perceptual value of symbolic evidence is that it exposes selected temporal properties in a compact and communicable form. A symbolic description can highlight trends, anomalies, or similarities between temporal windows (Gruver et al., 2023). Such evidence is useful when the agent must interact with users, retrieve related cases, compare patterns across domains, or maintain an interpretable trace of what has been observed. It also enables time series data to enter language-centric infrastructures such as prompt templates, retrieval systems, memory stores, and dialogue interfaces (Xue and Salim, 2023). In this sense, symbolic perception increases the accessibility of time series evidence.

The limitation is that symbolic perception is lossy. A textual summary may preserve the direction of change while omitting magnitude, uncertainty, exact timestamps, sampling frequency, or local variation. A prototype may capture shape but ignore scale. An attribute label may simplify a complex interval into a single word. A prompt embedding may encode useful information but remain difficult to inspect (Jin et al., 2024; Sun et al., 2024). Moreover, symbolic descriptions can introduce hallucinated or overconfident interpretations if they are generated without grounding in raw values or tool outputs. For example, describing a series as “sharply increasing” is ambiguous unless the magnitude, window, baseline, and threshold are specified. Thus, symbolic

perception should not replace numerical or diagnostic evidence. It is best understood as an abstraction layer that makes selected aspects of the signal explicit, while requiring links back to the observations from which those symbols were derived.

3.4 Structural Perception

Structural perception makes explicit the organization of a temporal environment. Rather than exposing only values or descriptions, it represents the series in terms of components, regimes, motifs, relations, graphs, causal candidates, or multi-scale temporal structures (Sun et al., 2025; Cui et al., 2025). The perceptual target is therefore the structure underlying the observations: how a signal decomposes into trend and seasonality, where regimes begin and end, which local motifs recur, how variables relate to one another, or which temporal segments may be causally or topologically connected (Lu et al., 2026; Huang et al., 2024).

This interface is important because many time series environments are not well described by independent sequences. A multivariate industrial system contains interacting sensors; a traffic network contains spatially connected roads; a financial market contains correlated assets and lead-lag effects; a healthcare record contains variables whose meanings depend on patient state and intervention history. Structural perception attempts to expose these relations as explicit objects. Graph construction and topology modeling represent variables, sensors, entities, or locations as nodes and their dependencies as edges. Regime segmentation identifies intervals with different dynamics. Motif discovery exposes repeated local shapes. Decomposition separates component-level views. Causal edge extraction or causal association modeling proposes directed temporal relations under lag, confounding, or domain constraints (Sun et al., 2025; Cui et al., 2025).

Recent LLM-related time series systems provide increasingly direct examples of this structural interface. Instead of aligning time series and language only at the token or embedding level, SGCMA (Sun et al., 2025) introduces structure-guided cross-modal alignment by exploiting state-transition graph structures shared by temporal patches and linguistic representations, thereby making sequence-level structural consistency part of the perceptual interface. LeRet (Huang et al., 2024) further illustrates a language-empowered form of structural modeling: inspired by causal extraction in large language models, it introduces a causal dependence learner to capture sequential causal evolution across patches. Compared with ordinary numerical encoding, these methods expose higher-order temporal organization—state transitions, structural consistency, and causal dependence—as perceptual substrates that can be passed to downstream reasoning or forecasting. Systems such as PATRA (Lu et al., 2026) and Augur (Cui et al., 2025) further illustrate this tendency by treating temporal components, pattern structures, or causal associations as objects that can be aligned, refined, or reused.

The strength of structural perception is that it exposes organization that is otherwise implicit. A raw multivariate sequence may show many channels changing together, but a structural view can indicate whether the changes are synchronized, lagged, localized, or propagated through a topology (Sun et al., 2025; Cui et al., 2025). A long trajectory may appear noisy, while a regime view reveals stable intervals separated by transitions. A set of recurrent local windows may become visible only after motif-level abstraction. Such structures can also serve as a compact evidence state: instead of storing every point, the agent may store components, intervals, edges, motifs, state-transition patterns, or causal-dependence candidates that summarize how the environment is organized (Lu et al., 2026; Huang et al., 2024).

Structural perception is nevertheless assumption-heavy. Estimated graphs may reflect correlation rather than stable dependency. Causal candidates may be confounded by shared seasonality, external events, or missing variables. Regime boundaries may shift with the segmentation algorithm. Motifs may depend on distance metrics and normalization. Decomposition may impose additive or multiplicative assumptions that do not match the system. State-transition structures or causal-dependence learners may also encode modeling assumptions that are not directly verifiable from the raw sequence (Sun et al., 2025; Huang et al., 2024). In nonstationary environments, a structure that is valid in one period may fail in another. Therefore, structural evidence should be treated as provisional. Its value lies in making temporal organization inspectable, but its reliability depends on how the structure was estimated, how stable it is across time, and whether alternative structures are plausible (Cui et al., 2025).

Table 3 Perception targets in the time series perception layer, organized by the evidence each target makes observable and the core limitation of that perceptual interface.

Perception target	Evidence made observable	Perceptual function	Representative systems	Core limitation
Raw numerical evidence	Values, normalized windows, temporal patches, discrete tokens, masked or quantized sequence fragments.	Preserves the original temporal signal with minimal semantic transformation and exposes numerical patterns at point, window, or token level.	Chronos (Ansari et al., 2024), MOMENT (Goswami et al., 2024), TimesFM (Das et al., 2023), Moirai (Woo et al., 2024), Lag-Llama (Rasul et al., 2023), Timer (Liu et al., 2024e).	Numerical fidelity does not by itself make the signal semantically interpretable.
Diagnostic evidence	Trend, seasonality, autocorrelation, missingness, change points, abnormal intervals, frequency patterns, and data-quality profiles.	Extracts intermediate diagnostic views that expose latent temporal properties, data defects, and localized irregularities.	TimeSeriesScientist (Zhao et al., 2025), TimeART (Wu et al., 2026b), TS-Agent (Liu et al., 2025d).	Diagnostic views can be brittle when tool assumptions or preprocessing choices are inappropriate.
Symbolic evidence	Textual summaries, semantic tokens, temporal attributes, prototypes, labels, or prompt-compatible abstractions.	Converts numerical or visual temporal patterns into compact symbolic forms that expose human-readable semantics.	Time-LLM (Jin et al., 2024), TEST (Sun et al., 2024), PromptCast (Xue and Salim, 2023), LLMTime (Gruver et al., 2023), ChatTS (Xie et al., 2024).	Symbolic abstraction can omit magnitudes, timestamps, or uncertainty.
Structural evidence	Decomposed components, regimes, motifs, causal edges, variable relations, graph topology, or multi-scale temporal structures.	Reorganizes observations into explicit temporal, relational, or component-level structures that expose dependencies and repeated forms.	PATRA (Lu et al., 2026), Augur (Cui et al., 2025), SGCMA (Sun et al., 2025), LeRet (Huang et al., 2024).	Estimated structures may reflect modeling assumptions rather than stable properties of the environment.
Multimodal evidence	Cross-modal observations involving time series, charts, tables, text, logs, weather, remote sensing data, medical records, or EHRs.	Aligns temporal signals with external modalities and exposes contextual evidence beyond the numerical sequence alone.	TimeXL (Jiang et al., 2026), MAS4TS (Ruan and Liang, 2026), AGCD (Wu et al., 2026a), TSAD-Agents (Xu et al., 2026f), Time-VLM (Zhong et al., 2025), MILM (Chung et al., 2026).	Modality alignment errors or conflicting context can distort the observed temporal state.

3.5 Multimodal Perception

Multimodal perception extends the observable environment beyond the numerical sequence. In realistic temporal settings, the meaning of a time series often depends on external context: a price movement may depend on news or filings; a traffic spike may depend on weather or incidents; a sensor anomaly may depend on maintenance logs; a physiological signal may depend on medications, notes, and electronic health records; a climate or agricultural series may depend on satellite imagery and remote sensing data (Xie et al., 2024; Chen et al., 2026). Multimodal perception therefore aligns time series observations with charts, tables, text, logs, weather reports, EHRs, images, maps, or other domain-specific artifacts (Jiang et al., 2026).

This interface can be instantiated in several ways. Visual perception converts time series into plots, dashboards, spectrograms, heatmaps, or chart images that can be inspected by vision-language models. Table-based perception organizes temporal records, metadata, or irregular measurements into structured rows and columns. Text-context perception aligns time series windows with news, reports, logs, clinical notes, or domain documents. Remote-sensing and spatial perception align temporal trajectories with images, maps, or geographic context. Medical perception may combine continuous signals with EHR events, laboratory measurements, and clinical

narratives. Systems such as TimeXL, MAS4TS, AGCD, TSAD-Agents, Time-VLM, and MILM reflect this broader movement toward multimodal temporal evidence, where the observable state is no longer contained in the sequence alone (Jiang et al., 2026; Ruan and Liang, 2026; Wu et al., 2026a; Xu et al., 2026f; Zhong et al., 2025; Chung et al., 2026).

The perceptual benefit is contextualization. Multimodal evidence can explain why a signal changes, whether a deviation is expected, and which external factors should be considered part of the temporal environment (Chen et al., 2026). For example, a load curve may be abnormal relative to previous weekdays but normal under extreme weather; a financial drop may appear as a numerical anomaly but align with a known macroeconomic announcement; a sensor spike may be meaningless without maintenance logs; a clinical measurement may require medication timing to be interpreted. By aligning external artifacts with time series windows, multimodal perception expands what the agent can observe.

The main limitation is alignment. Modalities differ in temporal resolution, uncertainty, noise level, and semantic granularity. News may affect a market series within minutes or days; clinical notes may describe events that occurred before they were written; weather reports may apply to a region rather than a sensor; tables may contain irregularly sampled records; images may represent spatial context rather than direct temporal values. Conflicts across modalities are also common: a textual report may suggest one explanation while the numerical signal suggests another. Noisy or irrelevant context can contaminate the evidence state, and vision-based perception may be sensitive to chart scaling, resolution, or visual design (Chen et al., 2026). Thus, multimodal perception should not be treated as adding context unconditionally. It must decide which external signals are temporally aligned, semantically relevant, and reliable enough to enter the current evidence state.

3.6 Evidence Map and Design Implications

The preceding subsections describe five perceptual interfaces through which temporal environments become observable. Together, they form an evidence map of the perception layer: they specify what is exposed from the temporal environment, how that evidence is shaped, and what distortions or omissions may enter before the evidence state is passed downstream (Jin et al., 2024; Xie et al., 2024). Tab. 3 summarizes this map.

Several design implications follow from this evidence map. First, perception should be typed. Raw values, diagnostic outputs, symbolic descriptions, structural objects, and multimodal context have different reliability profiles and should not be collapsed into a single undifferentiated embedding (Ansari et al., 2024). A normalized window, a decomposed seasonal component, a textual trend label, a causal candidate, and a chart-derived observation each preserve and lose different information. An agentic time series system should therefore record not only the perceived evidence, but also the interface through which that evidence was produced.

Second, perception should preserve links back to the observations that support it. Symbolic and structural interfaces are useful because they make temporal evidence compact and inspectable, but they are also lossy (Sun et al., 2025; Huang et al., 2024). A trend label should remain linked to the window and statistic from which it was derived; an anomaly interval should remain linked to the raw values and detector configuration; a multimodal interpretation should remain linked to the timestamped external artifact that supplied the context. Without such grounding links, perception can silently transform numerical uncertainty into fluent but unverifiable evidence (Liu et al., 2025d; Wu et al., 2026b).

Third, perception should be quality-aware. Temporal observations are often missing, misaligned, nonstationary, noisy, or affected by preprocessing choices. This is especially important in diagnostic and multimodal perception, where tool assumptions, resampling policies, chart design, document timestamps, and cross-modal alignment can change what the agent appears to observe (Jiang et al., 2026). A reliable evidence state should therefore include observation quality signals such as missingness, sampling frequency, alignment confidence, tool parameters, and uncertainty estimates.

Finally, perception should remain separable from interpretation. The goal of this layer is to construct an observable evidence state, not to settle what the evidence means. Raw numerical perception may expose a sharp change; diagnostic perception may localize a changepoint; symbolic perception may label the interval as an increase; structural perception may connect it to a regime boundary; and multimodal perception may align it with an external event (Cui et al., 2025). Whether these observations justify a temporal claim is the

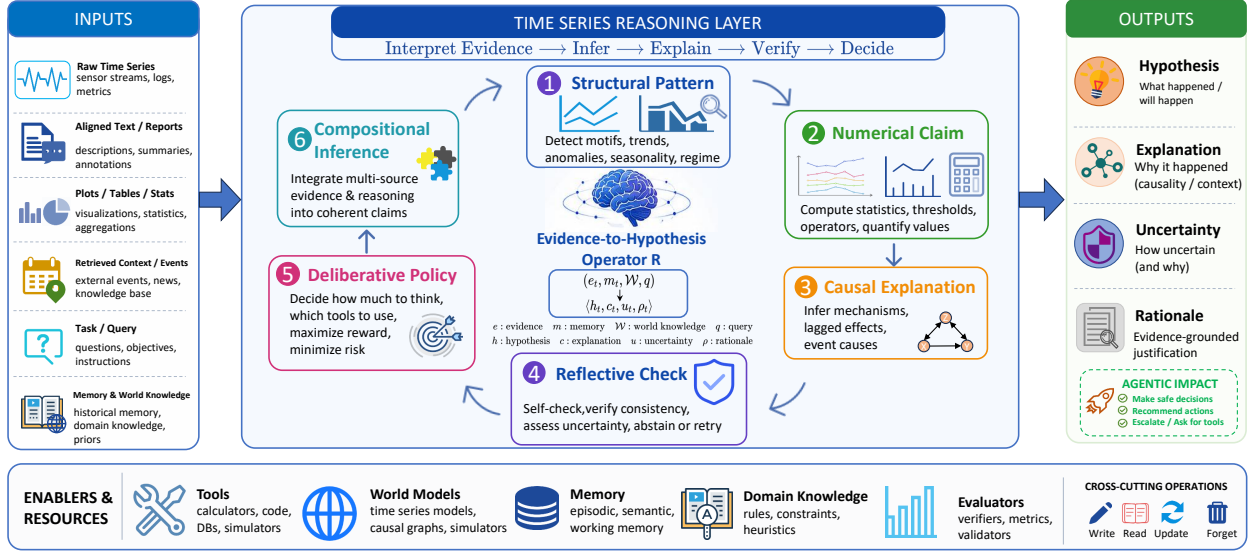


Figure 4 Overview of the time series reasoning layer, which transforms heterogeneous temporal evidence into hypotheses, explanations, uncertainty estimates, and evidence-grounded rationales for agentic decision-making.

responsibility of the reasoning layer. This separation is important because it prevents perceptual artifacts from being mistaken for conclusions and makes the transition from evidence to inference auditable (Liu et al., 2025d; Wu et al., 2026b).

Tab. 3 therefore serves as the evidence map for perception. It highlights a central lesson of this layer: agentic time series systems do not simply receive time series; they construct a view of temporal environments through a combination of numerical, diagnostic, symbolic, structural, and multimodal interfaces. The next section turns from this constructed view to the inferential problem it enables: how an agent transforms perceived evidence into temporal claims, explanations, uncertainty statements, and verifiable hypotheses.

4 Time Series Reasoning: Inferring Temporal Dynamics and Causality

Time series reasoning is the layer that turns temporal evidence into claims. Perception layers assemble evidence from raw series, aligned text, plots, tables, and retrieved context; planning and action layers decide which tools, models, or interventions to execute. Reasoning sits between them: it interprets evidence to infer what happened, why it may have happened, how reliable the inference is, and what it implies for the next decision. This distinction matters because many recent systems no longer treat forecasting or question answering as direct sequence-to-label prediction. They instead expose intermediate hypotheses, tool observations, causal narratives, and self-checks that can be audited before an agent acts. Fig. 4 summarizes this role by positioning the time series reasoning layer as an evidence-to-hypothesis interface between heterogeneous temporal inputs, supporting resources, and downstream agentic outputs.

A compact way to view this layer is as an evidence-to-hypothesis operator,

$$\mathcal{R} : (e_t, m_t, \mathcal{W}, q) \rightarrow \langle \eta_t, c_t, \nu_t, \rho_t \rangle. \quad (13)$$

Here, e_t denotes evidence available at time t , m_t denotes task or episodic memory, $\mathcal{W}$ denotes world-model or domain knowledge, and q is the query. The output contains a temporal hypothesis η_t , a causal or contextual explanation c_t , an uncertainty statement ν_t , and an evidence-grounded rationale ρ_t . The formalization is intentionally modest: the central design question is not whether a model produces a long chain of thought, but whether each inferential step is anchored to an observed window, computed statistic, retrieved event, executable tool output, or validated domain constraint.

Tab. 4 gives the section-level map used below. The rows are ordered from evidence recognition to policy-level deliberation: each later form of reasoning depends on earlier evidence being represented with enough fidelity

Table 4 Reasoning targets in the time series reasoning layer, organized by the evidence each target should consume and the agentic risk if the inference is left unverified.

Reasoning target	Evidence that should ground it	Agentic implication	Representative systems
Structural pattern	Windows, plots, decomposition, motif or anomaly labels.	Determines what the agent can safely summarize or escalate.	TimeSeriesExamAgent (Gwiazda et al., 2026), PATRA (Lu et al., 2026), MTBench (Chen et al., 2025a), MAS4TS (Ruan and Liang, 2026), Segment Selection (Messica et al., 2026).
Numerical claim	Statistics, thresholds, operators, executable tool observations.	Turns fluent explanations into auditable evidence traces.	TS-Agent (Liu et al., 2025d), TimeART (Wu et al., 2026b), TS-Reasoner (Ye et al., 2026), Argos (Gu et al., 2025), VeriTime (Zhou et al., 2026).
Causal explanation	Temporal order, lag, event context, graph or test-based constraints.	Controls whether the agent may recommend intervention or only report association.	Augur (Cui et al., 2025), MTBench (Chen et al., 2025a), TimeOmni-1 (Guan et al., 2025), TS-Reasoner (Ye et al., 2026).
Reflective check	Intermediate traces, raw-series evidence, verifier comments, output contracts.	Decides whether to answer, retry, ask for tools, or abstain.	T3LLM (Su et al., 2025), TS-Agent (Liu et al., 2025d), CastFlow (Pan et al., 2026), AlphaCast (Zhang et al., 2026a), TS-Debate (Trirat et al., 2026), VeriTime (Zhou et al., 2026).
Deliberative policy	Process rewards, task taxonomies, forecast error, trend and seasonal scores.	Allocates slow thinking to hard temporal questions rather than every query.	Time-R1 (Luo et al., 2025b), Cast-R1 (Tao et al., 2026b), TimeOmni-1 (Guan et al., 2025), VeriTime (Zhou et al., 2026), PATRA (Lu et al., 2026), CastFlow (Pan et al., 2026).

to support verification. The table is a target taxonomy; the mechanism map at the end of the section instead asks how an agent should invoke, check, and hand off each reasoning mode.

This notation also provides the organizing thread for the remainder of the section. Pattern and structural reasoning primarily constrains (h_t) by selecting the relevant temporal segment and assigning a structural description. Numerical and statistic-grounded reasoning strengthens (ρ_t), and sometimes revises (h_t), by linking temporal claims to executable operators, input windows, statistics, thresholds, and tool observations. Causal and compositional reasoning shapes (c_t), while also bounding whether the explanation is a validated mechanism, an event-conditioned hypothesis, a lagged association, or only a correlation. Reflective reasoning checks the consistency of the whole tuple ($\langle h_t, c_t, \nu_t, \rho_t \rangle$), especially whether (ρ_t) supports the claim and whether (ν_t) reflects evidence gaps, tool limits, and task risk. Slow-thinking and reinforcement-enhanced methods operate at the policy level of ($\mathcal{R}$): they determine when the agent should deliberate, which intermediate operations it should perform, and how process rewards or tool feedback should shape the resulting tuple.

4.1 Pattern and Structural Reasoning

Pattern reasoning is the base layer of time series reasoning. Before an agent can explain a forecast, diagnose a fault, or compare possible causes, it must identify the structural evidence in the series: trend, seasonality, local shape, similarity, volatility regime, anomaly, changepoint, and recurring motif. Benchmarks such as TimeSeriesExamAgent make this requirement explicit by generating and filtering questions over pattern recognition, noise understanding, similarity analysis, anomaly detection, and causal reasoning across synthetic and real domains (Gwiazda et al., 2026). In this setting, reasoning is not merely naming a visual pattern. It requires mapping a temporal segment to the property being queried, deciding which distractors are plausible, and preserving enough local evidence to justify the answer. Segment-selection methods sharpen this requirement by treating the relevant window itself as a reasoning decision: the agent must first choose which

temporal evidence is admissible before it can justify a pattern label or answer (Messica et al., 2026).

Recent alignment methods similarly treat pattern structure as the interface between time series encoders and language reasoning. PATRA decomposes time series representations into trend, seasonal, and full latent components, aligns those components with textual tokens, and trains on heterogeneous time series question-answering tasks (Lu et al., 2026). Rationale-grounded in-context learning adds another form of alignment by storing observation-to-implication rationales and retrieving them as evidence-grounded precedents for new multimodal time series queries (Liu et al., 2026b). Evidence-pruning agents make the same granularity requirement more operational: ZARA first turns motion windows into sensor-specific statistical evidence, prunes candidate features by distributional support, and only then forms a decision insight for classification (Li et al., 2026e). The important survey-level lesson is that pattern evidence has multiple granularities. A long-run upward trend may coexist with a short local reversal; a strong seasonal component may obscure an anomaly; two series may share shape while differing in lag or amplitude. Agents that reason from a single coarse label therefore risk hiding the very evidence that makes the downstream conclusion true or false.

Multimodal benchmarks broaden this problem by asking models to reason across plotted curves, tables, and textual context. MTBench pairs time series signals with text in financial and weather domains, testing forecasting, technical-indicator calculation, news-driven question answering, and cross-modal temporal reasoning (Chen et al., 2025a). MAS4TS takes an agentic route: analyzer, reasoner, and executor roles use visual anchors from plotted series, latent trajectory reconstruction, shared memory, and tool routing to solve forecasting, classification, imputation, and anomaly tasks (Ruan and Liang, 2026). These systems illustrate why visual or language-aligned reasoning is valuable for structural interpretation, but they also expose a bounded claim. Visual descriptions can be too coarse for exact thresholds, text-aligned pattern summaries can miss small local changes, and benchmark quality can depend on prompt coverage, data coverage, and human validation. Pattern reasoning should therefore hand off uncertain or numerical claims to statistic-grounded verification rather than treating a plausible visual account as sufficient.

4.2 Numerical and Statistic-Grounded Reasoning

Many time series claims are numerically checkable. Statements about a threshold crossing, rate of change, volatility increase, correlation, anomaly score, lag, or forecast horizon should be grounded in computed quantities rather than fluent verbal impressions. TS-Agent makes this principle concrete by iteratively gathering insights from raw sequences with ReAct-style thought, action, and observation steps, using analytical actions such as summarization, extraction, query, detection, prediction, and relation analysis, and maintaining an evidence log with critic and quality-gate checks (Liu et al., 2025d). Its core contribution for reasoning is the separation between a candidate answer and the observations that support it: the agent can defer the final response until enough raw-series evidence has been collected.

Tool-augmented systems generalize this pattern. TimeART formulates time series reasoning as a Query–Thought–Action–Observation–Final Answer process over a library of analytical tools and trains tool-use behavior with TimeToolBench trajectories (Wu et al., 2026b). TS-Reasoner decomposes domain questions into executable operator programs that combine time series model operators, numerical calculations, retrieval, and generated custom operations, then uses execution feedback to repair invalid pipelines (Ye et al., 2026). In both cases, reasoning becomes partly symbolic and partly procedural. A claim such as “the second variable leads the first” is stronger when it follows from an operator trace over lagged association or causal tests than when it appears only in a free-form explanation.

Anomaly-oriented agents show the same contract in a narrower but useful setting. Argos turns anomaly explanations into reproducible detection rules that are proposed, repaired, reviewed, and aggregated with model predictions (Gu et al., 2025). AnomaMind similarly builds evidence in coarse-to-fine stages, combining localization, adaptive feature construction, reasoning-based anomaly decisions, and self-reflection over intermediate choices (Tao et al., 2026c). Specialist-analyzer systems push this contract further by decomposing anomaly evidence into point, structural, seasonal, and pattern-oriented analyses, then aggregating conflicts with rubric-guided confidence rather than a single undifferentiated explanation (Kang et al., 2026). Domain-specific question-answering systems make tool choice itself auditable: TSAG evaluates whether financial claims about volatility, correlation, or seasonality are backed by the intended quantitative operator instead of a free-form response (Kolonin et al., 2026). These systems are not only anomaly detectors or financial

assistants; they make visible the distinction between a verbal diagnosis and an auditable rule, interval, feature, operator, or decision trace.

The same pressure appears in process-verifiable training. VeriTime synthesizes time series reasoning trajectories with process-verifiable annotations, organizes them by task difficulty and taxonomy, and applies two-stage reinforcement fine-tuning with rewards for format, length, final-answer correctness, and intermediate step validity (Zhou et al., 2026). This supports a bounded claim: process supervision can reduce coherent but numerically wrong reasoning when the verifier has access to the relevant evidence, task-specific labels, and a sufficiently broad task taxonomy.

Numerical grounding does not make reasoning automatically reliable. Tool-backed agents can still select the wrong operator, pass the wrong window, or misinterpret an observation. Pure language chains of thought can sound precise while fabricating calculations. Process labels and validators are expensive, and their coverage is limited by the task taxonomy used to construct them. For this reason, statistic-grounded reasoning should be treated as an auditable contract: the rationale should name which computation supports which claim, and the uncertainty statement should reflect the coverage and failure modes of the available tools.

4.3 Causal and Compositional Reasoning

Reasoning becomes more demanding when the question shifts from *what pattern exists?* to *which variables, events, or mechanisms produced it?* Causal and compositional reasoning must respect temporal order, lag, confounding, exogenous events, and domain constraints. It also has to compose heterogeneous evidence: a co-movement between two channels, a retrieved news event, a physical prior, and a forecast model’s residual pattern may each support a different explanation. A time series agent should therefore distinguish causal hypotheses from correlational summaries, and should expose the evidence type behind each claim.

Augur illustrates one route toward this goal. It uses a teacher–student pipeline for covariate causal association modeling: heuristic pruning narrows candidate relationships, pairwise LLM judgments compare directed hypotheses, confounding, or spurious association, graph refinement removes inconsistent edges, and causal textual prompts are used to guide downstream forecasting (Cui et al., 2025). This is not proof of intervention-level causality, but it is a useful architectural pattern because causal reasoning is represented as an object that can be pruned, checked, and reused rather than as a single narrative answer. Event-detection agents make a complementary object visible: Grammar of the Wave compiles natural-language event definitions into event-logic trees, instantiates them on multivariate signals, and verifies logical consistency before reporting an event explanation (Wan et al., 2026). Such neuro-symbolic intermediates are valuable because they constrain what can be composed from local signal evidence, domain knowledge, and visual inspection.

Benchmarks and generalized reasoning models also expose causal composition as a stress test. MTBench includes finance and weather tasks in which news, reports, and temporal signals must be interpreted jointly, including news-driven question answering and correlation-oriented financial questions (Chen et al., 2025a). TimeOmni-1 formalizes reasoning-critical tasks around scenario understanding, causality discovery, event-aware forecasting, and decision-making, using supervised and reinforcement learning to improve task-grounded reasoning over TSR-Suite (Guan et al., 2025). TS-Reasoner includes multi-step inference settings where retrieval, numerical operators, and forecasting tools can be combined under domain constraints (Ye et al., 2026). Across these examples, the agent must decide not only whether a relationship is present, but also whether the relationship is temporally plausible and relevant to the query.

The main risk is causal overreach. Time series systems often observe correlation, synchronized shocks, or shared seasonality; none of these by itself establishes causation. LLM-generated causal rationales are especially vulnerable because they can fill gaps with plausible domain stories. Causal claims should therefore be bounded unless backed by graph structure, lag-sensitive statistics, explicit interventions, or external event evidence. In a survey taxonomy, this makes causal reasoning a higher-risk reasoning type than pattern recognition or statistic calculation: it carries more decision value, but also demands stronger validation.

4.4 Reflective and Metacognitive Reasoning

Reflective reasoning lets an agent inspect its own intermediate conclusions before committing to an output. In time series settings, this reflection is useful because failures often arise early: the model chooses the wrong

segment, mistakes seasonal variation for anomaly, copies a trend label without checking scale, or treats a coincident event as causal. A reflective module can ask whether the current trace has enough evidence, whether the final answer follows from the observations, and whether the output satisfies a task-specific format.

T3LLM provides a clean example of reflective correction at the reasoning-trace level. Its reviewer checks the worker’s chain of thought against the original series and answer, finds the earliest erroneous step, and produces comments that guide continuation (Su et al., 2025). TS-Agent uses a critic and final quality gate to assess evidence sufficiency before returning an answer (Liu et al., 2025d). CastFlow embeds reflection in a planning–action–forecasting–reflection workflow: after diagnostic tools and forecasting, the reflection module validates format and evidence alignment and can trigger bounded retries (Pan et al., 2026). AlphaCast similarly separates generator and reflector roles, using reflective evaluation to revise forecast reasoning (Zhang et al., 2026a). Debate-based designs extend the same idea from reviewer correction to multi-agent critique: TS-Debate uses collaborative argumentation over multimodal time series evidence, making agreement and disagreement among agents part of the verification signal (Tirrat et al., 2026).

Process-verifiable pipelines make this separation trainable rather than only procedural. VeriTime labels intermediate reasoning steps and rewards process validity in addition to final-answer correctness, giving the reviewer signal a task-specific target instead of relying only on another free-form model judgment (Zhou et al., 2026). Practical harnesses generalize the same idea from model training to task execution: AION uses temporal hypothesis expansion, rules-aware reasoning, a temporal governor, and a final reviewer to make critique rounds and task artifacts part of the evaluation protocol (Zhan et al., 2026). Streaming anomaly systems add a different form of reflection: CALM delays judgment until enough context is available, then asks an LLM judge whether an anomaly is transient noise or persistent drift before curating adaptation data (Devireddy and Huang, 2025). The common design pattern is metacognitive separation: the component that proposes a temporal hypothesis is not the only component that judges whether the hypothesis is warranted.

The limitation is that reflection can become another source of plausible text. Reviewers may inherit the generator’s blind spots, especially when both rely on the same model family or the same lossy visual representation. Reflection also adds cost and latency, and more reflection is not automatically better: AlphaCast reports that a more complex iterative reflection variant can increase forecasting error by disrupting temporal continuity and amplifying small deviations (Zhang et al., 2026a). The strongest reflective agents therefore ground review in raw windows, tool observations, explicit answer formats, or process labels. Reflection is most valuable when it can fail a claim for a concrete reason, not when it merely rewrites the rationale in a more polished form.

4.5 Slow-Thinking and Reinforcement-Enhanced Reasoning

Prompted reasoning can expose intermediate steps, but recent systems increasingly try to train temporal deliberation itself. The motivation is that time series questions often require a sequence of operations: identify the relevant window, decompose trend and seasonality, compute or retrieve supporting quantities, compare candidate explanations, and only then forecast or answer. Slow-thinking and reinforcement-enhanced methods aim to make this multi-step behavior more reliable than ad hoc prompting.

Time-R1 casts forecasting as slow-thinking reasoning. It uses supervised fine-tuning as a warm-up stage for structured temporal analysis, followed by reinforcement learning with GRIP and a multi-objective reward design that includes output format, prediction error, seasonal–trend decomposition, and extrema or change-point matching (Luo et al., 2025b). Cast-R1 complements this view by treating tool-augmented forecasting as a sequential decision policy, where reinforced reasoning chooses and orders analytical actions before producing the forecast (Tao et al., 2026b). VeriTime uses a related process-verifiable route, combining thinking-data synthesis, scheduling, and two-stage reinforcement fine-tuning to reward intermediate validity as well as final answers (Zhou et al., 2026). TimeOmni-1 trains across scenario understanding, causality discovery, event-aware forecasting, and decision-making so that complex reasoning is tied to task-grounded rewards rather than a single benchmark format (Guan et al., 2025). Reasoning-aware distillation offers another route: STRIDE projects LLM reasoning traces into the continuous embedding space of time series foundation models, making semantic reasoning a prior for numerical forecasting rather than a separate textual afterthought (Ahamed et al., 2026a).

Table 5 Mechanism map for using reasoning inside an agentic time series system. Unlike [Tab. 4](#), which classifies inference targets, this table summarizes when each mechanism should be invoked and what evidence contract it should leave for planning.

Mechanism	When to invoke	Evidence contract	Planning handoff
Pattern abstraction	The agent must name structure or choose a relevant segment.	Segment or plot id, pattern label, local counter-evidence, and ambiguity note.	Summarize if qualitative; call tools for thresholds, lags, or amplitudes.
Executable verification	The claim concerns a value, statistic, lag, volatility, anomaly score, or forecast number.	Operator name, parameters, input window, observation, and supported sentence.	Accept only within tool coverage; escalate if the statistic is used causally.
Causal composition	The agent explains direction, event attribution, confounding, or mechanism-level relationships.	Temporal order, lag evidence, graph or test constraints, event context, and association-versus-causation wording.	Recommend intervention only when the causal standard is met; otherwise report association.
Reflective critique	A generated trace may contain unsupported steps, format errors, evidence gaps, or inconsistent forecasts.	Reviewer decision tied to raw series, tool observations, output schema, or process labels, plus retry reason.	Retry, ask for more tools, abstain, or pass only the verified prefix to planning.
Deliberative training	The task requires multi-step temporal deliberation or repeated tool-policy decisions.	Rewards for task correctness, process validity, format, and calibrated forecast error, with over-thinking checks.	Route easy tasks directly; reserve slow policies for causal, long-horizon, or event-conditioned decisions.

Balanced reward design is especially important for heterogeneous time series tasks. PATRA combines supervised alignment with reinforcement learning and task-aware balanced rewards across time series question-answering categories ([Lu et al., 2026](#)). CastFlow extends the slow-thinking theme toward workflow-oriented forecasting: SFT aligns reasoning signals and tool-derived evidence with numerical forecasting, while RL with verifiable rewards refines the evidence-guided correction process ([Pan et al., 2026](#)). More general CoT-elicitation results similarly suggest that reinforcement can change whether models expose usable intermediate temporal reasoning rather than only final predictions ([Parker et al., 2025](#)). Taken together, these systems suggest that long-form temporal reasoning is not just a prompting style; it can be a trainable behavior shaped by format constraints, task correctness, process rewards, and tool-use feedback.

The limitation is reward dependence. Reinforcement learning can overfit output templates, exploit validator weaknesses, or learn unnecessarily long traces for simple tasks. TimeOmni-1 explicitly notes that overly simple QA can induce over-thinking and that underspecified context can force guessing rather than reasoning ([Guan et al., 2025](#)). Inference-time studies of slow-thinking LLMs make the caution empirical: one-shot, decoupled, and rollout reasoning can help capture high-level trends and context shifts, but the gains remain uneven and sensitive to how temporal context is represented ([Cheng et al., 2026b](#)). Process rewards are only as reliable as the validators, decompositions, and task taxonomies that define them. Slow thinking should therefore be adaptive: simple pattern or arithmetic questions should not require expensive deliberation, while causal, long-horizon, and event-conditioned tasks should expose enough intermediate evidence for external evaluation.

[Tab. 5](#) summarizes the main reasoning mechanisms as design choices rather than mutually exclusive model families. A mature time series agent may use several rows at once: it can identify a local motif, compute a lagged statistic, compare a causal hypothesis with retrieved context, ask a reflector to check the trace, and invoke a slow-thinking policy only when the task warrants extra deliberation.

The first design implication is that reasoning should be evidence-grounded. Every rationale should be tied to raw series windows, statistics, retrieved events, tool observations, or explicit domain constraints. This is especially important when the agent’s explanation will guide a downstream planning module. A forecast rationale that says “the pattern is rising” is weak; a rationale that identifies the relevant window, reports the slope or changepoint evidence, and explains why the claim affects the next action is stronger.

The second implication is that reasoning should preserve uncertainty. Pattern recognition, long-horizon forecasting, and causal attribution have different evidentiary standards. Causal and event-conditioned claims should use bounded language unless supported by lag-aware tests, graph constraints, intervention evidence, or externally verified events. Uncertainty is not a cosmetic confidence score; it determines whether the planning layer should act, request more evidence, choose a conservative model, or ask for human review.

The third implication is that reasoning should be process-evaluable. Final answer accuracy cannot tell whether an agent used the right temporal segment, performed the right calculation, or reached the answer through a spurious shortcut. Benchmarks and training pipelines increasingly evaluate intermediate steps, tool traces, answer formats, and verifier feedback (Zhou et al., 2026; Wu et al., 2026b; Su et al., 2025). This shift is central for agentic systems because a correct but unsupported rationale may still cause unsafe downstream actions.

Finally, reasoning is the handoff to planning. Once a temporal hypothesis is trusted, planning and action can decide which tool, model, intervention, or monitoring policy to execute. Conversely, if reasoning remains uncertain, the agent should not simply produce a fluent final answer. It should trigger additional evidence collection, tool execution, reflection, or abstention. In this sense, time series reasoning is the controllable bridge from observed dynamics to autonomous decision-making.

5 Planning and Action: From Forecasting to Autonomous Decision-Making

Planning and action form the operational layer of an agentic time series system: **they determine how temporal evidence is acquired, which computations are executed, how intermediate results are revised, and when a prediction is converted into a consequential decision.** This layer is therefore the point at which time series intelligence stops being a single mapping from history to future and becomes a controlled process over data, tools, models, memories, and environments (Cheng et al., 2026a; Zhan et al., 2026; Poikarayil et al., 2026). The distinction matters because a strong reasoning model can describe a trend or justify a forecast in one pass, yet still remain passive. A time series agent, by contrast, must choose the next diagnostic, retrieve or construct the relevant context, invoke a forecasting or anomaly-detection module, inspect validation feedback, revise its plan, and, in decision-centric settings, issue an action whose consequences may only be observed later (Wu et al., 2026b; Zhao et al., 2025; Garza and Rosillo, 2025; Tao et al., 2026b). Thus, the central object of this layer is not a chain-of-thought transcript, but an executable and auditable control process.

We formalize the planning and action layer as

$$\mathcal{C} = \langle \mathcal{G}, \mathcal{S}, \mathcal{A}, \pi, \mathcal{O}, \mathcal{V} \rangle, \quad (14)$$

where $\mathcal{G}$ is the task goal, $\mathcal{S}$ is the evolving evidence state, $\mathcal{A}$ is the typed action space, π is the planning or action policy, $\mathcal{O}$ is the set of observations returned by tools or environments, and $\mathcal{V}$ denotes verification signals. At step t , the agent selects an action by

$$a_t \sim \pi_\theta(a_t \mid g, s_t, \eta_{<t}), \quad o_t = \text{Env}(a_t), \quad s_{t+1} = \text{Update}(s_t, a_t, o_t), \quad (15)$$

where $\eta_{<t}$ denotes the memory and the prior interaction trace. The action a_t may be a statistical diagnostic, a retrieval operation, a model call, a data-selection decision, a code execution step, an inter-agent delegation, a report-generation step, or an external intervention such as a trade or traffic-control action. This view makes planning and action a closed-loop interface: every action changes either the agent’s evidence state, the computational pipeline, or the external temporal environment.

5.1 Workflow Planning: Making Forecasting Pipelines Explicit

The most immediate form of planning is **workflow decomposition**. Real time series analysis rarely consists of a single forecast call. Even a conventional forecasting task may require frequency inference, missing-value treatment, seasonal or regime diagnosis (Zhao et al., 2025; Li et al., 2026b; Yang et al., 2025a), feature construction (Li et al., 2026d; Yeh et al., 2025; Lee et al., 2025; Das et al., 2026), model-space pruning (Yang et al., 2025e; Tao et al., 2026c), cross-validation, ensembling, error attribution (Cai et al., 2025b; Kang et al., 2026; Gu et al., 2025), and a final report (Zhan et al., 2026; Zhao et al., 2025). Agentic workflow planning makes these latent decisions explicit: a planner converts an underspecified temporal goal into a sequence of executable stages, each with its own inputs, tools, and verification criteria.

Several systems instantiate this idea by *turning forecasting into a white-box pipeline*. TimeSeriesScientist separates the process into Curator, Planner, Forecaster, and Reporter agents, so prior mentioned stages (e.g., data analysis, model selection, validation, reporting, etc.) become inspectable ones rather than hidden implementation details (Zhao et al., 2025). TimeCopilot pursues a controller-oriented version of the same

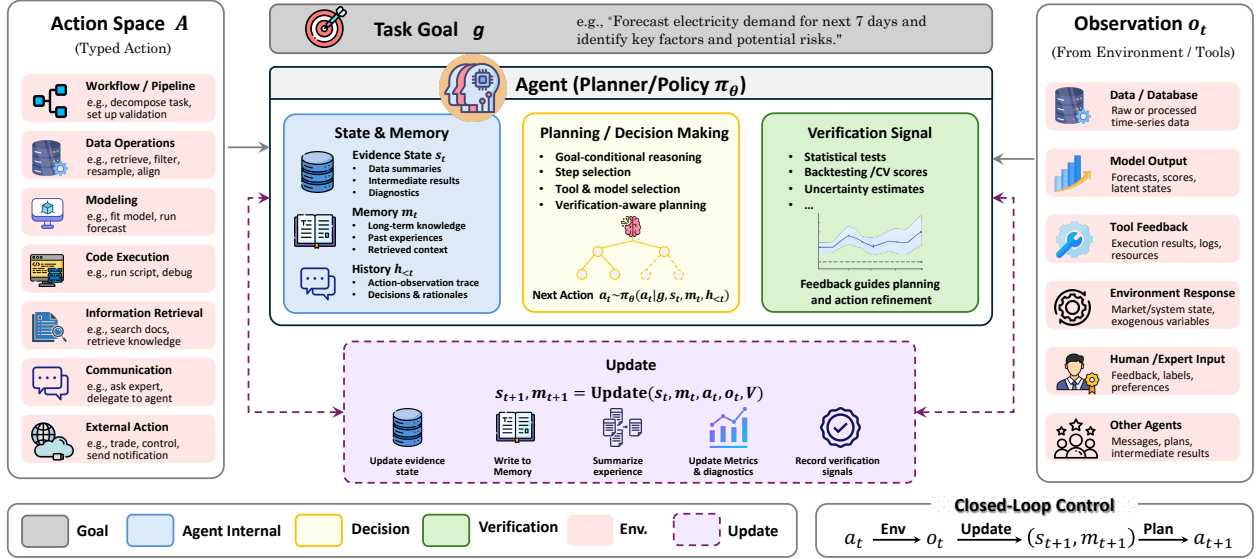


Figure 5 Closed-loop planning and action in agentic time series systems. The agent conditions on the task goal g , evidence state s_t , memory m_t , and history $h_{<t}$ to select typed actions from $\mathcal{A}$ under policy π_θ . Actions invoke workflows, data operations, models, code, retrieval, communication, or external interventions. Observations o_t and verification signals V update the agent state and memory, enabling iterative, auditable decision-making over time.

principle: the LLM coordinates feature analysis, model selection, cross-validation, and forecast generation through a unified forecasting API (Garza and Rosillo, 2025). AutoLFM further brings this pattern into building-load forecasting, using Retriever, Reasoner, Coder, and Validator agents to combine domain-knowledge retrieval, data-adaptive search-space construction, code generation, and executable checking (Li et al., 2026b). The common contribution is not simply automation. These systems expose the assumptions that normally sit between raw data and a reported forecast, making the pipeline easier to inspect, rerun, and revise.

A second line treats workflow planning as *data or context construction*. DCATS uses an LLM agent to select auxiliary traffic series from metadata, making data-set construction itself the action under optimization (Yeh et al., 2025). TimeCAP first contextualizes a series into event-centered textual descriptions and then uses that augmented context for downstream event prediction (Lee et al., 2025). Nexus decomposes forecasting evidence into macro-level fluctuations, micro-level fluctuations, and textual events before synthesizing the final prediction (Das et al., 2026). These systems show that a planner may improve forecasting by deciding what evidence should exist before a model is fitted. For time series, this decision is often as important as model choice: auxiliary variables, event descriptions, and domain context can help under distribution shift, but they can also introduce leakage or spurious correlations if they are selected without temporal constraints.

A third line focuses on *iterative workflow refinement*. CastFlow learns role-specialized forecasting workflows in which planning, forecasting, reflection, and memory update form a repeated loop (Pan et al., 2026). FLAIRR-TS combines retrieval with iterative refinement, using retrieved examples and feedback to revise forecasting-agent outputs (Jalori et al., 2025). AlphaCast frames forecasting as interaction-driven co-reasoning, where external evidence and prior experience condition the next reasoning step (Zhang et al., 2026a). SEA-TS moves revision into code: candidate forecasting algorithms are generated, evaluated, reviewed, and improved through a self-evolving loop (Xu et al., 2026e). The shift from static workflow to revisable workflow is important for agentic systems because the first plan is often wrong: sampling frequency may be misread, seasonality may be local, validation windows may be unrepresentative, and the best next action depends on the observations produced by earlier actions.

Workflow planning also appears outside forecasting. AD-Agent converts a natural-language anomaly-detection request into an end-to-end executable pipeline, linking intent parsing, data preparation, library selection, model selection, and debugging (Yang et al., 2025d). CityGPT decomposes urban IoT analysis into requirement understanding, temporal analysis, spatial analysis, spatiotemporal fusion, and visualization (Feng et al., 2025).

AION abstracts these patterns into a general task harness with task files, workspaces, validation interfaces, agents, skills, rules, memory, evaluation, and protocols (Zhan et al., 2026). In this sense, workflow planning grows from a forecasting convenience into a general interface for realistic time series engineering tasks.

5.2 Tool Routing and Evidence Acquisition

The second form of action is **tool routing**. Tool use is especially important for time series because many relevant facts are numerical, statistical, and local in time. Pure language reasoning can hallucinate averages, trends, anomalies, correlations, or future values. Tool-augmented agents reduce this risk by turning intermediate reasoning steps into executable observations, so that a claim about trend, seasonality, anomaly, correlation, or forecast quality can be checked against a diagnostic rather than only asserted in language (Wu et al., 2026b; Liu et al., 2025d; Ye et al., 2026; Tao et al., 2026c; Tan et al., 2026). The central question is therefore not simply whether an agent can call a tool, but how the tool call changes the agent’s evidence state and what verification signal is returned.

The clearest tool-use pattern begins with *ReAct-style evidence gathering*. TimeART defines a Query–Thought–Action–Observation–Final Answer loop and trains on TimeToolBench, where actions call time series tools such as decomposition, autocorrelation, change-point, anomaly, and lightweight forecasting modules (Wu et al., 2026b). TS-Agent keeps raw series outside the LLM and lets operators extract statistics, trends, anomalies, and relations, with an evidence log and quality gate ensuring that final answers are grounded in executed observations (Liu et al., 2025d). TS-Reasoner makes the same idea more programmatic by translating natural-language questions into operator programs over model, numerical, retrieval, and custom-generation tools (Ye et al., 2026). These systems expose a design spectrum rather than a single historical trajectory: TimeART emphasizes learnable tool-use traces, TS-Agent emphasizes accumulated evidence and answer gating, and TS-Reasoner emphasizes executable operator composition. The common principle is that temporal reasoning becomes more reliable when each step leaves a typed observation that later steps must consume.

Tool routing also supports *retrieval and knowledge grounding*. Here retrieval should not be read as passive background context; it is an action that changes which examples, domain facts, or prior patterns are available to downstream tools. Agentic RAG for Time Series Analysis uses a hierarchical controller to route tasks such as forecasting, anomaly detection, classification, and imputation to specialized sub-agents with task-specific prompt memories (Ravuru et al., 2024). TSAG (Kolonin et al., 2026) narrows this idea to financial QA, where the agent must select quantitative tools and align time series outputs with domain documents rather than answer from parametric memory alone. Hubble goes further by constraining financial alpha discovery to operator trees checked by deterministic evaluators, so generation happens inside a DSL-like action space instead of unconstrained natural language (Shi et al., 2026a). Across these cases, retrieval and constrained generation serve the same reliability function: they restrict what can count as admissible evidence.

Anomaly and event-detection agents use tools to *localize and verify abnormal patterns*. The difficulty is that anomaly evidence is heterogeneous: a spike, a seasonal break, a distributional shift, and a domain event require different views of the same sequence. AnomaMind addresses this with a coarse-to-fine workflow in which broad evidence acquisition precedes more specific diagnostic tool selection (Tao et al., 2026c). SAGE makes the heterogeneity explicit by assigning point, structural, seasonal, and pattern anomalies to specialized analyzers whose tool-backed evidence is later aggregated (Kang et al., 2026). Grammar of the Wave uses neuro-symbolic VLM agents to compile multivariate event definitions into explainable symbolic evidence, while TS-Debate uses modality-specific agents and debate to cross-check visual, numerical, and textual interpretations (Wan et al., 2026; Trirat et al., 2026). Thus, anomaly-oriented tool routing is best understood as evidence arbitration. The planner must decide which diagnostic family is relevant, how conflicting signals are reconciled, and when the accumulated evidence is sufficient.

Domain agents extend tool routing into *executable environments*. Open-TI connects traffic analysis to map acquisition, simulation, and signal-control tools, while CityGPT exposes temporal, spatial, and visualization tools for urban IoT analysis (Da et al., 2024; Feng et al., 2025). AgriWorld generalizes this idea into a World-Tools-Protocol, where geospatial alignment, remote-sensing time series analysis, crop simulation, code execution, and validity checks are all callable actions (Zhang et al., 2026b). In RFID, retrieval and optimization tools are used to construct expert actions that later guide MARL-style execution in a retail supply setting (Yang et al., 2025g). This domain shift matters because the observation returned by a tool may

be a simulated queue length, a crop-model validity check, or an energy-trading constraint violation rather than a textual answer. Once tool outputs become environmental feedback, planning and action move from evidence acquisition toward closed-loop control.

5.3 Model, Data, and Code Orchestration

The third action family is **orchestration over computational resources**. As statistical baselines, neural models, time series foundation models, external data sources, and domain-specific toolkits proliferate, the bottleneck shifts from having a predictor to deciding which resource should be used, when it should be trusted, and how its artifact should be audited. This is broader than ordinary model selection. An orchestrating agent must define the object being optimized, choose the search or routing space, expose a feedback signal, and preserve enough traces for rollback and comparison.

Model orchestration starts from *choosing among forecasting engines*. TimeCopilot treats the LLM as a controller over statistical models, machine-learning models, neural models, time series foundation models, and ensembles, using feature analysis and cross-validation to decide when to escalate model complexity (Garza and Rosillo, 2025). TSOchestra (Cao et al., 2025) similarly casts the LLM as a metacognitive controller that evaluates optimization history and decides whether to continue, reweight, or accept the current forecast ensemble. The agentic action is not the final forecast value. It is the allocation of computation across candidate engines under validation feedback. This separates agentic forecasting from a static ensemble: the controller can revise its resource allocation as diagnostics, horizon-specific behavior, and cross-validation traces accumulate.

Data orchestration *changes the intervention target*. DCATS improves traffic forecasting by selecting auxiliary series from metadata and validation feedback (Yeh et al., 2025). R&D-Agent-Quant extends the idea to financial research, jointly exploring data-centric factor construction and modeling choices (Li et al., 2026d). Human-in-the-Loop Adaptive Optimization adds user feedback to post-training forecast correction, showing that action spaces may include transformations suggested by humans as well as by agents (Tiomoko et al., 2025). These methods remind us that planning can act on the data set, factor space, or correction rule before it acts on model parameters. For temporal data, this is a high-risk action: an auxiliary series may help because it captures a real leading signal, or hurt because it leaks future information, reflects a transient regime, or encodes a spurious calendar artifact.

Code orchestration *turns the agent into a time series ML engineer*. SEA-TS generates forecasting algorithms, evaluates them, performs automated code review, and uses best/worst memories to refine later generations (Xu et al., 2026e). AD-Agent performs the analogous operation for anomaly detection by selecting data-processing steps, libraries, models, and debugging actions from a natural-language task (Yang et al., 2025d). Time-SeriesGym evaluates this broader capability directly: engineering agents must edit code, tune pipelines, manage artifacts, and satisfy task-specific validation in realistic time series codebases (Cai et al., 2025b). The evaluation target therefore expands from prediction error to engineering competence: whether the agent can maintain executable pipelines, interpret logs, recover from failed runs, and avoid overfitting to a weak validation protocol.

Constrained orchestration provides a safer alternative to arbitrary code generation. Hubble restricts alpha discovery to operator trees evaluated by deterministic pipelines, linking agent generation to reproducible financial factors (Shi et al., 2026a). MONAQ queries neural architectures for resource-constrained time series analysis, while AegisTS uses hierarchical reinforcement learning to select multivariate cleaning and imputation actions (Tiriat and Lee, 2025; Shi et al., 2026b). CALM adapts anomaly-detection pipelines over streams, showing that orchestration may need to remain active after deployment rather than stop at training time (Devireddy and Huang, 2025). The trade-off is structural. Narrower action spaces improve reproducibility and safety, but they make the exposed operators, reward signals, and validators the bottleneck. Open-ended code gives greater flexibility, but demands stronger sandboxing, testing, and audit trails.

5.4 Multi-Agent Coordination: Separating Incompatible Responsibilities

Multi-agent coordination is useful when a time series task contains responsibilities that should not be collapsed into a single prompt. Generation, execution, critique, domain translation, risk assessment, and reporting

often require different tools, incentives, and stopping criteria. The value of multiple agents is therefore not the number of LLM calls; it is the separation of incompatible roles under an explicit communication protocol.

General-purpose systems often *divide perception, reasoning, and execution*. MAS4TS uses Analyzer, Reasoner, and Executor agents, linking VLM-based plot perception to latent trajectory reasoning and task-specific tool routing (Ruan and Liang, 2026). TS-Debate replaces a single executor with collaborative debate, allowing modality-specific agents to challenge and aggregate zero-shot time series interpretations (Trirat et al., 2026). MERIT applies coordination to representation learning, where agents collaborate to construct transferable unsupervised time series representations (Zhou et al., 2025b). MAFS specializes agents by forecasting subtasks, using cooperation to divide horizon-, component-, or pattern-specific responsibilities (Huang et al., 2025).

Anomaly systems specialize by *failure mode and validation role*. Argos separates Detection, Repair, and Review so that rule generation, syntax/runtime repair, and validation feedback are handled by different agents (Gu et al., 2025). SAGE decomposes diagnosis into point, structural, seasonal, and pattern analyzers before aggregating evidence into anomaly records (Kang et al., 2026). AnomaMind combines tool-augmented evidence construction with RL-optimized anomaly decisions, while ChatAD uses multi-turn instruction evolution to improve anomaly reasoning over iterations (Tao et al., 2026c; Sun et al., 2026). The design pattern is consistent: one role proposes, another executes or repairs, and a third validates or adjudicates. This separation is valuable precisely because anomaly decisions are easy to overfit to one visual or statistical cue.

Semantic, multimodal, and domain-facing systems specialize by *interpretation role*. TESSA separates general cross-domain annotation from domain-specific annotation and review, connecting feature extraction to terminology adaptation (Lin et al., 2026). TimeCAP uses one agent to contextualize events and another to predict, while ZARA and FETA use evidence-grounded or in-context agent pipelines for classification (Lee et al., 2025; Li et al., 2026e; Sui et al., 2025). Hierarchical AI-Meteorologist organizes weather reporting across temporal scales, and multi-agent weather captioning improves consensus through feedback over generated text (Sukhorukov et al., 2026; Liu, 2026). AGCD extends role specialization to weather forecasting through agent-guided cross-modal decoding (Wu et al., 2026a).

Decision systems add analysts, reviewers, and risk managers because the output may change the environment. TradingAgents simulates a trading firm with market, news, sentiment, technical, bull/bear, trader, and risk-management roles (Xiao et al., 2024). R&D-Agent-Quant, ATLAS, FinCon, FinMem, FinArena, and FinHEAR preserve the financial multi-agent pattern but vary the adaptation mechanism: prompt optimization, conceptual verbal reinforcement, layered memory, human-agent collaboration, or risk-aware expertise (Li et al., 2026d; Papadakis et al., 2025; Yu et al., 2024, 2025b; Xu et al., 2025c; Chen et al., 2025b). CityGPT and ColaCare show the same architectural principle outside finance, aligning agents with urban-IoT or EHR data modalities and decision roles (Feng et al., 2025; Wang et al., 2025b). The main failure mode is also clear: communication overhead and inconsistent intermediate states can accumulate, so multi-agent designs require stronger state management than single-agent pipelines.

5.5 Sequential Decision-Making and External Action

The most agentic form of planning is **sequential decision-making**. Here the output is not merely a forecast or explanation; it is an action in a process with state, delayed feedback, and potential external consequences.

Cast-R1 is the direct bridge from forecasting to sequential decision-making: the agent maintains a decision state, chooses actions such as analysis, feature extraction, model invocation, forecast update, and revision, and learns long-horizon policies through supervised and reinforcement training (Tao et al., 2026b). ARTIST applies a related policy view to time series QA, where a controller sequentially selects informative time segments before the reasoner answers (Messica et al., 2026). VeriTime (Zhou et al., 2026), COUNTS (Parker et al., 2025), TimeOmni-1 (Guan et al., 2025), and Time-R1 (Luo et al., 2025b) do not always execute external tools, but they show that RL and process-verifiable rewards can train longer temporal reasoning trajectories. This line links slow-thinking reasoning to action policies: the model learns not only what answer to give, but what intermediate reasoning or evidence-selection path to follow.

Financial systems make the transition from prediction to external action concrete. TradingAgents maps analyst reports and debates into buy/sell/hold decisions, while ATLAS uses an order-aware action space and adapts prompts from delayed market feedback (Xiao et al., 2024; Papadakis et al., 2025). FinMem

emphasizes layered memory for trading decisions, whereas FinCon abstracts previous episodes into reusable investment concepts and combines them with risk control (Yu et al., 2025b, 2024). CryptoTrade (Li et al., 2024), Hi-DARTS (Sagong et al., 2025), Trading-R1 (Xiao et al., 2025b), and ElliottAgents (Chudziak and Wawer, 2025) extend this action view through reflection, hierarchical reinforcement trading, RL-trained investment reasoning, and natural-language stock-market agents. Agentic Trading surveys this design space and stresses that execution, risk, adaptation, and reproducibility must be evaluated together (Xia et al., 2026).

Sequential action is broader than trading. Open-TI connects natural-language traffic requirements to map retrieval, simulation, and signal-control actions (Da et al., 2024). AgriWorld grounds agricultural reasoning in executable tools and crop-simulation interventions, so actions can be checked by spatiotemporal, unit, and counterfactual constraints (Zhang et al., 2026b). StockAgent, and RFID model stock trading, retail demand control, or ensemble weighting as MDP/Dec-POMDP-style decision processes (Zhang et al., 2025b; Yang et al., 2025g). SocioDojo extends the interaction setting to lifelong socio-economic analysis agents that combine text, time series, tools, and memory across tasks (Cheng and Chin, 2024). Across these domains, delayed feedback and irreversible decisions make action quality, calibration, and safety more important than forecasting error alone.

5.6 Evidence Map and Design Implications

Table 6 summarizes representative planning and action mechanisms. The table is organized by the type of action space rather than by application domain, because the same design question recurs across forecasting, anomaly detection, trading, traffic, agriculture, and urban IoT: **what can the agent do, what feedback does it observe, and how is the action verified?**

Several design implications follow from this evidence map. First, the action space of a time series agent must be **typed**. Actions such as “compute autocorrelation”, “fit a baseline”, “retrieve weather covariates”, “generate code”, and “place an order” have different risk profiles and should not be represented as undifferentiated text. Second, planning should be **diagnosis-aware**. A planner that ignores sampling frequency, missingness, seasonality, multivariate dependence, horizon length, and operational constraints is unlikely to produce reliable workflows. Third, every action should leave an **evidence trace**. Time series agents are often deployed in domains where users need to know not only the final forecast, but also which diagnostics, data, tools, and validation checks supported it. Fourth, **verification** must be part of the action loop rather than a post-hoc evaluation step. Systems such as Argos, TS-Agent, TS-Reasoner, AutoLFM, Hubble, AgriWorld, and AION all point toward the same principle: generation must be paired with *executable checking*.

Finally, planning and action reveal the *boundary* between time series reasoning models and time series agents. If a system only emits a forecast or explanation, it remains a reasoning or prediction model. If it can decompose a temporal goal, choose and execute tools, update its state from observations, revise intermediate decisions, coordinate specialized agents, and act under validation and risk constraints, it becomes an agentic time series system. This distinction is central to the rest of the taxonomy: memory determines what experience can be reused, world models determine what environments can be simulated, and verification determines whether actions are trustworthy enough for deployment.

6 Memory and Knowledge: Accumulating Temporal Experience

Memory is the layer that determines whether a time series agent accumulates experience or merely reacts to the present. In a conventional forecasting pipeline, the model parameters encode all past experience; at inference time, no additional memory is consulted, updated, or selectively forgotten. An agentic system, by contrast, must decide *what* to remember from each interaction, *when* to retrieve it, *how* to update it as distributions shift, and *when* to discard it because it no longer applies. Recent position papers argue that agentic time series forecasting requires structured memory operating alongside perception, reasoning, and planning (Cheng et al., 2026a; Xu et al., 2026a), and surveys of LLM-driven time series analysis identify memory as one of the core architectural modules that distinguishes agentic systems from static predictors (Zhang et al., 2025c). The central claim of this section is that *stuffing a longer context window is not memory*: without selection, indexing, updating, forgetting, and reuse, a log of past observations is merely a log.

Table 6 Representative planning and action mechanisms in agentic time series systems.

Category	Action space	Representative systems	Feedback and limitation
Workflow planning	Preprocessing, feature engineering, model selection, validation, reporting	TimeSeriesScientist (Zhao et al., 2025), AutoLFM (Li et al., 2026b), AION (Zhan et al., 2026), Cast-Flow (Pan et al., 2026)	Validation metrics and report traces improve auditability, but planning quality depends strongly on the exposed tool and model space.
Tool routing	Statistical diagnostics, decomposition, anomaly tools, forecasting tools, external-data operators	TimeART (Wu et al., 2026b), TS-Agent (Liu et al., 2025d), TS-Reasoner (Ye et al., 2026), AnomaMind (Tao et al., 2026c), Agri-World (Zhang et al., 2026b)	Tool observations ground reasoning and reduce hallucination, but tool selection errors and tool-output misuse remain central failure modes.
Model and data orchestration	TSFM routing, statistical baselines, ensembles, auxiliary-series selection, context integration	TimeCopilot (Garza and Rosillo, 2025), DCATS (Yeh et al., 2025), MoiraiAgent (Liu et al., 2025a), Nexus (Das et al., 2026), TimeSeries-GYM (Cai et al., 2025b)	Cross-validation and benchmark scores guide selection, but cost-performance trade-offs and regime shift are still difficult.
Code and formula generation	Pipeline code, forecasting architecture code, anomaly-detection scripts, alpha-factor formulas	SEA-TS (Xu et al., 2026e), AD-Agent (Yang et al., 2025d), Hubble (Shi et al., 2026a), TS-Agent (financial) (Ang et al., 2025)	Sandboxes and deterministic evaluators improve reliability, but generated artifacts can overfit validation environments or exploit weak metrics.
Multi-agent coordination	Delegation among analyzers, reasoners, executors, reviewers, risk managers, and reporters	MAS4TS (Ruan and Liang, 2026), Argos (Gu et al., 2025), TimeCAP (Lee et al., 2025), R&D-Agent (Yang et al., 2025e), CityGPT (Feng et al., 2025)	Role separation improves interpretability and control, but communication overhead and inconsistent intermediate states can accumulate.
External environment action	Trading orders, traffic control, energy trading, crop-simulation intervention, inventory or demand-control actions	Cast-R1 (Tao et al., 2026b), TradingAgents (Xiao et al., 2024), ATLAS (Papadakis et al., 2025), Open-TI (Da et al., 2024), Agri-World (Zhang et al., 2026b), FinCon (Yu et al., 2024), FinMem (Yu et al., 2025b)	Delayed feedback and irreversible actions make verification and risk constraints essential; offline forecasting metrics are insufficient.

6.1 Beyond Conversation History: Why Time Series Agents Need Structured Memory

NLP agent memory has been studied extensively in the context of conversational assistants and tool-using chatbots, where memory is primarily textual, relatively stable, and evaluated by whether the agent remembers a user preference or a previously mentioned fact. Time series agent memory differs along five dimensions that make direct transfer from NLP memory designs insufficient.

Tab. 7 summarizes these differences. The first dimension, *non-stationarity*, means that a memory entry that was correct yesterday may be wrong today because the underlying data-generating process has shifted. This is not the same as a fact becoming outdated in a knowledge base; it is a change in the statistical regime that requires the agent to detect the shift and adjust which memories are relevant. The second dimension, *multi-scale dependency*, means that a single flat memory store cannot serve all temporal queries. Seasonal patterns, hourly fluctuations, and rare multi-year events must be indexed at different resolutions and retrieved according to the current time scale of interest. The third dimension, *delayed feedback*, means that the agent cannot immediately verify whether a stored strategy was correct. A trading decision or a maintenance schedule may only reveal its consequences after days or weeks, so the memory system must support delayed credit assignment. The fourth dimension, *statistical structure*, means that memory entries are not merely text. They carry trend, seasonality, and changepoint information that must be preserved during storage and matched during retrieval. The fifth dimension, *numerical grounding*, means that retrieved patterns must be verifiable by diagnostic tools rather than accepted on the basis of language plausibility alone.

Table 7 Why time series agents require specialized memory designs: five fundamental dimensions that distinguish TS-agent memory from NLP-agent memory.

Dimension	NLP Agents	Time Series Agents
Non-stationarity	Relatively stable knowledge base; facts update infrequently	Continuous distribution drift; patterns valid under one regime may be harmful under another
Multi-scale dependency	Single granularity; conversation turns are uniform	Second-level, hourly, seasonal patterns coexist at different resolutions
Delayed feedback	Immediate evaluation; tool output confirms/refutes action	Consequences appear hours/days later; credit assignment spans time steps
Statistical structure	Text fragments; retrieval by semantic similarity	Carries trend, seasonality, changepoints; retrieval must respect numerical structure
Numerical grounding	Facts accepted/rejected by language reasoning	Patterns must be verifiable by statistical tools, not textual plausibility

To formalize these requirements, we define the memory layer of a time series agent as a six-tuple

$$\mathcal{M} = \langle \mathcal{M}_{\text{ctx}}, \mathcal{M}_{\text{epi}}, \mathcal{M}_{\text{temp}}, \mathcal{M}_{\text{know}}, \mathcal{M}_{\text{proc}}, \mathcal{M}_{\text{conf}} \rangle, \quad (16)$$

where each component corresponds to a memory subtype and each subtype supports four operations: write($\cdot$), read($\cdot$), update($\cdot$), and forget($\cdot$). Context memory $\mathcal{M}_{\text{ctx}}$ stores short-term task-level evidence. Episodic memory $\mathcal{M}_{\text{epi}}$ stores complete case trajectories for analogical retrieval. Temporal memory $\mathcal{M}_{\text{temp}}$ stores regime-specific, season-aware, and rare-event patterns indexed by time scale. Knowledge memory $\mathcal{M}_{\text{know}}$ stores domain rules, tool manuals, and causal structures that are not instance-specific. Procedural memory $\mathcal{M}_{\text{proc}}$ stores strategies, workflows, and reflective conclusions about what works. Confidence memory $\mathcal{M}_{\text{conf}}$ stores failure patterns, uncertainty estimates, and calibration signals. This taxonomy extends the memory discussion in recent surveys (Cheng et al., 2026a; Zhang et al., 2025c; Xia et al., 2026) by distinguishing temporal memory and confidence memory as first-class subtypes, motivated by the non-stationarity and delayed-feedback dimensions that are unique to time series domains.

Memory does not operate in isolation. It receives structured evidence from the perception layer (statistical features, anomaly labels, pattern descriptions), which is written into context and episodic memory; it stores reasoning traces and reflective conclusions from the reasoning layer, which are written into procedural and confidence memory; it records workflow patterns and tool-use trajectories from the planning layer, which are written into procedural memory; and it accumulates domain rules and causal structures from the world-model layer, which are written into knowledge and temporal memory. Conversely, memory feeds back into every other layer: retrieved historical patterns guide perception focus, stored reasoning strategies shape reasoning paths, past cases inform task decomposition and tool selection, and confidence profiles constrain verification thresholds and fallback strategies. Verification closes the loop by writing success and failure signals back into confidence memory, creating a continuous cycle of evidence accumulation, strategy refinement, and reliability improvement. Fig. 6 illustrates this architecture. The upper row shows the five surrounding architectural layers and their bidirectional data flows with the memory layer: perception writes structured evidence into context and episodic memory; reasoning writes traces and conclusions into procedural and confidence memory; planning writes workflow patterns into procedural memory; the world model writes causal rules and regime information into knowledge and temporal memory; and verification writes success and failure signals into confidence memory. In the reverse direction, memory feeds retrieved patterns, strategies, cases, domain knowledge, and calibration profiles back into each layer. The middle row organizes the six memory subtypes by their function, each supporting the four cross-cutting operations (write, read, update, forget). The lower row shows the three phases of memory dynamics, formation, retrieval, and evolution, which operate across all subtypes and form a closed loop. Memories are formed through interaction, training, and reflection; retrieved by similarity, condition, or gradient; and evolved through forgetting, confidence updating, and strategy refinement.

The following subsections examine each subtype in turn (Sections 6.2–6.7), followed by a synthesis of cross-cutting memory dynamics (Section 6.8) and an evidence map with open challenges (Section 6.9).

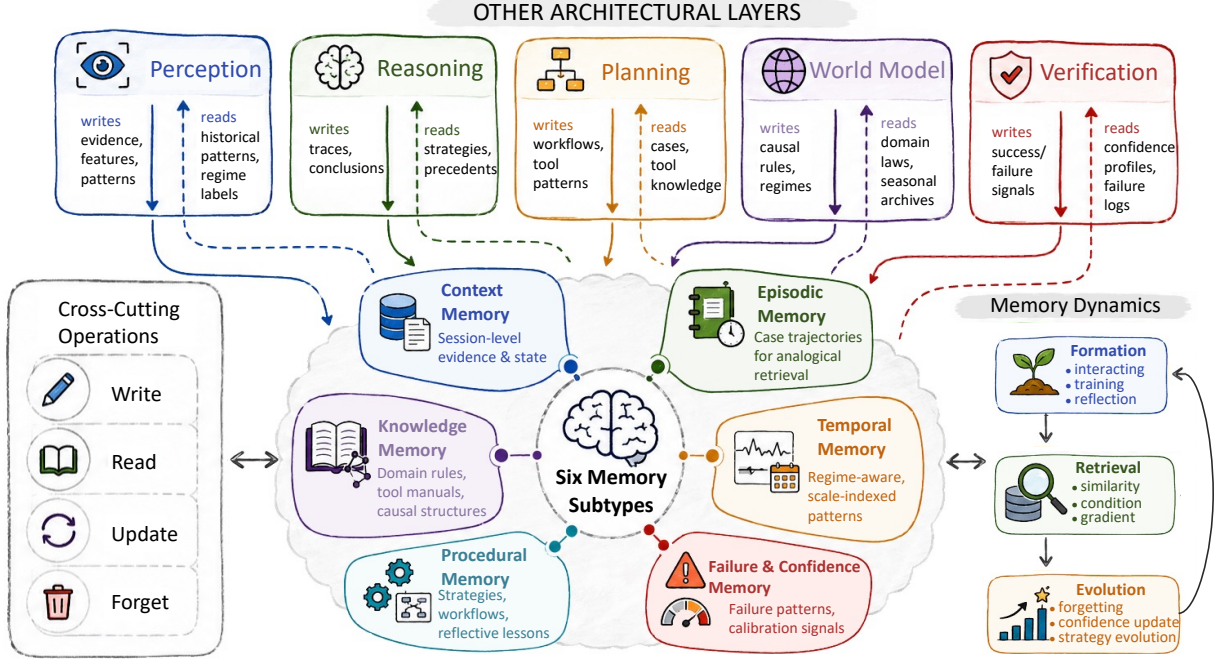


Figure 6 Architecture of the memory layer in agentic time series systems. The six memory subtypes, context, episodic, temporal, knowledge, procedural, and failure/confidence, each support four cross-cutting operations (write, read, update, forget). The upper row shows the five surrounding architectural layers (perception, reasoning, planning, world model, verification) and their bidirectional data flows with the memory subtypes: perception writes structured evidence into context and episodic memory; reasoning writes traces and conclusions into procedural and confidence memory; planning writes workflow patterns into procedural memory; the world model writes causal rules and regime information into knowledge and temporal memory; verification writes success and failure signals into confidence memory. The lower row shows the three phases of memory dynamics, formation, retrieval, and evolution, which operate across all subtypes in a closed loop.

6.2 Context and Working Memory

Context memory is the shortest-lived and most widely implemented subtype. It maintains the intermediate evidence, tool outputs, reasoning steps, and decision states that are generated during a single task or session. Its defining limitation is that it does not persist across tasks: when the session ends, the evidence is discarded unless it is explicitly promoted to a longer-lived store.

The simplest form of context memory is an *evidence log*. TS-Agent (Liu et al., 2025d) accumulates observations from statistical operators in a dynamically growing log, ensuring that every subsequent reasoning step has access to all previously gathered evidence rather than only the most recent observation. TS-Reasoner (Ye et al., 2026) maintains a buffer memory that records explored solution paths to avoid redundant re-evaluation within a single inference session. These logs are read-only within a session: they accumulate evidence but do not revise or compress it.

A more structured form is *state management*, where the agent maintains a decision-relevant state across interaction steps. Cast-R1 (Tao et al., 2026b) uses a memory state $M_t = (I_v, S_v)$ that records the validation result and code configuration at each round, enabling the agent to compare current and previous attempts and reject changes that increase loss. AIVV (Kwon et al., 2026) maintains a fixed-size sliding window of structured JSON messages passed between its three LLM agents, providing context stability while bounding memory length.

In multi-agent settings, context memory often takes the form of *shared working memory*. AD-AGENT (Yang et al., 2025d) distinguishes between short-term shared memory that stores the current session context and long-term cache that persists model information across sessions, avoiding redundant web searches. VisualTimeAnomaly (Xu et al., 2026f) uses a shared dynamic memory that stores key context information

such as detected anomaly types, accessible to all four agents in the pipeline.

Session-level persistence extends context memory beyond a single inference step. A PatchTST-based agent (Buratto et al., 2026) stores the full message history in SQLite and optionally indexes it with FAISS for semantic retrieval, supporting multi-turn analysis sessions where earlier diagnostic results inform later decisions. Hierarchical AI-Meteorologist (Sukhorukov et al., 2026) provides climatology context from Meteostat monthly aggregates and ERA5 climate normals, so that the agent can compare current weather patterns against long-term baselines within a single reporting session. AnomaMind (Tao et al., 2026c) uses a knowledge memory tool that provides semantic and contextual support during detection, including anomaly type descriptions, domain knowledge, and tool semantics. Although this tool provides domain knowledge rather than session-specific evidence, its content is read within the current detection session and does not persist, placing it at the boundary between context memory and knowledge memory.

The central limitation of context memory is that it cannot address regime shifts, rare events, or cross-session experience. When the context window overflows, information is lost unless a selection or compression mechanism is applied. A log without selection is not a memory system; it is a buffer. The following subsections describe the memory subtypes that address this limitation, beginning with episodic memory, the first subtype that enables cross-session experience reuse.

6.3 Episodic and Case-Based Memory

Episodic memory stores complete case trajectories, including the evidence gathered, the reasoning applied, and the outcome achieved, so that the agent can retrieve similar past cases and apply analogical reasoning rather than starting from scratch at every encounter. The core question is what counts as “similar”: for time series cases, similarity involves statistical properties, temporal patterns, and contextual features that are more complex than textual semantic overlap. MemCast (Tao et al., 2026a) constructs a three-layer episodic store. Its Historical Patterns layer retrieves similar past series from training data by pattern matching and uses the retrieved patterns to condition the reasoning trajectory. The retrieval is based on time series similarity rather than text embedding, which is essential because two series with similar textual descriptions may exhibit different statistical behaviors. TS-Agent (Ang et al., 2025) for financial time series uses a Case Bank that stores successful modeling methodologies from past tasks, enabling case-based reasoning for model selection in new financial forecasting tasks. Argos (Gu et al., 2025) employs a Data Selector that retrieves contrastive examples from the training set, selecting samples with similar distributions to the current input so that anomaly detection rules are calibrated against relevant baselines.

Synthetic and augmented episode construction extends the episodic store beyond raw historical data. SAGE (Kang et al., 2026) builds a synthetic in-context-learning database by extracting prototypes from normal reference segments via k -medoids clustering and injecting nine types of anomaly perturbations, producing contrastive references that support confidence scoring during detection. MERIT (Zhou et al., 2025b) uses three LLM agents to collaboratively generate positive views for contrastive representation learning: a Retrieval Agent selects semantically similar sequences, an Augmentation Agent chooses augmentation strategies, and a Review Agent filters out views that distort the original semantics, storing approved views in a memory bank. MarsTSC (Li et al., 2026a) maintains a self-evolving knowledge base with structured sections covering intra-class features, inter-class distinctions, error-avoidance strategies, and domain background, organized into three confidence tiers at test time: prototype, reference, and candidate.

Rationale libraries store reasoning traces rather than raw data. RationaleTS (Liu et al., 2026b) generates label-conditioned abductive rationales (Observation $\rightarrow$ Implication paths) on the training set and stores them in a rationale library, which is then queried during inference using a hybrid retrieval mechanism that combines data-centric similarity with semantic similarity. FinSrag (Xiao et al., 2025a) trains a domain-specific retriever, FinSeer, that selects high-value historical time series segments from a candidate pool of 34 financial indicators, augmenting the LLM’s stock movement predictions with relevant historical patterns.

Expert precedents bridge episodic memory and domain knowledge. FinHEAR (Chen et al., 2025b) uses expert-informed precedents that combine historical trend analysis with outcome-based refinement, so that past trading decisions and their consequences inform future risk-aware reasoning. Decoding Time Series with LLMs (Lin et al., 2026) transfers cross-source domain knowledge to the target domain through a

domain-decontextualization step that generalizes annotations before re-specializing them, enabling few-shot annotation in new domains without target-domain labels.

A critical challenge for episodic memory in time series settings is the definition of similarity. Text-based retrieval via embedding similarity may miss statistical correspondences, while pure numerical methods such as dynamic time warping may miss semantic relevance. The systems reviewed here adopt different trade-offs: MemCast and SAGE use time series similarity, RationaleTS uses hybrid retrieval, and FinSrag trains a task-specific retriever. No single approach dominates, and the choice depends on whether the downstream task rewards statistical fidelity or semantic relevance more heavily. Episodic memory answers “what happened last time”; the next subtype, temporal memory, addresses the distinct question of “what patterns hold at which time scale and under which regime.”

6.4 Long-Horizon Temporal and Regime Memory

Temporal memory stores regime-specific, season-aware, and rare-event patterns indexed by time scale. It is the memory subtype that most clearly distinguishes time series agents from NLP agents: in a text dialogue, there is no equivalent of a regime shift, a seasonal cycle, or a once-per-decade shock. Temporal memory must support three capabilities that NLP memory does not require: regime detection, so that the agent knows which stored patterns are currently relevant; temporal decay, so that stale patterns lose influence; and rare-event recall, so that the agent can draw on experience from infrequent but high-impact events. FinMem (Yu et al., 2025b) provides the clearest example of scale-indexed temporal memory. Its three-layer architecture mimics human trader cognition: Shallow Memory stores daily price and volume data with a high forgetting rate, Intermediate Memory stores news with a moderate forgetting rate, and Deep Memory stores quarterly earnings and SEC filings with a low forgetting rate. The forgetting rates are not arbitrary; they reflect the empirical persistence of information at each time scale. A daily price pattern may be irrelevant after a week, but an earnings report affects valuation for months. MemCast (Tao et al., 2026a) organizes temporal memory at three levels of abstraction: Historical Patterns store concrete past series, Reasoning Wisdom distills successful and failed reasoning strategies as reusable templates, and General Laws capture domain-invariant regularities such as trend continuation and mean reversion. This hierarchy allows the agent to retrieve a specific past series when the current regime matches, fall back to a reasoning template when no specific match exists, and apply a general law when even templates are unavailable. SocioDojo (Cheng and Chin, 2024) implements lifelong memory that combines a short-term sliding window with a long-term procedural store, enabling the agent to accumulate reusable analysis strategies across multiple socioeconomic analysis sessions. CALM (Devireddy and Huang, 2025) addresses distribution shift directly: its fine-tuning loop accumulates pattern-drift data that has been verified by an LLM-as-judge, so that the detection model adapts to persistent regime changes while filtering out transient noise. AgriWorld (Zhang et al., 2026b) records complete interaction trajectories $\tau = \{(code, observation, product)\}$ that capture not only what the agent observed but what it tried and what it produced, providing a trace that can be revisited when similar agricultural conditions recur.

Hierarchical priors can also serve as temporal memory. TimeOmni-1 (Guan et al., 2025) constructs layered chain-of-thought priors through a two-stage training process: supervised fine-tuning injects temporal priors, and reinforcement learning refines them, producing reasoning structures that encode temporal regularities at multiple scales. CryptoTrade (Li et al., 2024) maintains a reflection memory where the Reflection Agent analyzes recent trading performance, identifies effective and ineffective strategies, and stores strategy-improvement suggestions that inform future decisions under similar market conditions. TradingAgents (Xiao et al., 2024) uses historical debate records from Bull and Bear researcher agents to provide context for current market assessments, so that past disagreements and their resolutions can inform present deliberation.

The fundamental difference between temporal memory in time series agents and memory in NLP agents is that temporal memory must be *regime-aware*. In NLP, a stored fact either remains correct or becomes outdated; in time series analysis, a stored pattern may be correct under one regime and actively harmful under another. This requires memory systems that tag entries with regime labels, monitor distribution shifts, and adjust retrieval weights dynamically. Among the systems reviewed, FinMem and CALM are the only ones that implement explicit forgetting or adaptation mechanisms tied to distribution change; most others rely on the LLM’s parametric knowledge to handle regime shifts implicitly, which is insufficient when the shift is large or abrupt.

6.5 Knowledge Memory

Knowledge memory stores domain rules, tool manuals, causal structures, and other information that is not tied to a specific episode but applies across tasks. Unlike episodic memory, which answers “what happened last time,” knowledge memory answers “what is generally true.” The primary delivery mechanisms are retrieval-augmented generation (RAG), knowledge graphs, and structured knowledge bases.

Structured knowledge bases provide the most direct form of knowledge memory. AutoLFM (Li et al., 2026b) uses a structured JSON knowledge base that covers data preparation, feature engineering, modeling tips, and code pitfalls, with a Retriever agent that dynamically selects relevant entries based on the current agent role and pipeline stage. R&D-Agent-Quant (Li et al., 2026d) maintains a knowledge base K of (task, code, feedback) triples accumulated from historical experiments, a SOTA set of best-performing solutions, and an idea forest that tracks hypothesis evolution through hierarchical clustering.

Knowledge graphs encode relational structure that flat text cannot represent. ChatKG (Christino and Paulovich, 2024) builds a knowledge graph that links temporal sequences, detected patterns, temporal overlap relations, IA-generated knowledge, and suggested related datasets, enabling visual exploration of the “pattern $\rightarrow$ knowledge $\rightarrow$ dataset” chain. ElliottAgents (Chudziak and Wawer, 2025) stores Elliott Wave Principle theory in a RAG knowledge graph, enabling the LLM to retrieve wave-counting rules and Fibonacci-ratio targets during analysis. Its extension adds a Neo4j graph database for backtest results and a dynamic context manager that supports continuous learning (Wawer and Chudziak, 2025).

Anomaly detection and diagnostics require specialized knowledge about failure modes and their signatures. AnomaMind (Tao et al., 2026c) provides a knowledge memory tool that supplies anomaly type descriptions, domain knowledge, and tool semantics, giving the detection agent high-level contextual features that complement low-level statistical evidence. ZARA (Li et al., 2026e) constructs a pairwise feature importance knowledge base offline by computing permutation importance for each activity pair using AutoGluon, then uses this statistical prior to guide feature selection and evidence pruning during zero-shot classification.

RAG-based knowledge memory retrieves relevant documents or patterns at inference time. An agentic RAG for time series analysis (Ravuru et al., 2024) uses a differentiable prompt pool where each sub-agent maintains key-value pairs: keys encode global features such as seasonality and periodicity, and values contain corresponding pattern knowledge, with retrieval performed by cosine similarity over the key space. TS-RAG (Ning et al., 2025b) builds a multi-domain retrieval knowledge base that stores context-window and prediction-window pairs with their embeddings, retrieving the top- k most similar historical segments and fusing them with the current query representation through an adaptive retrieval mixer. TimeRAG (Park, 2026) maps time series segments to single-token representations via patch encoding and gated cross-attention, storing 3249 historical event documents in a vector database for similarity retrieval during cloud failure root cause analysis.

Parameterized knowledge memory internalizes domain knowledge into model parameters rather than storing it externally. TS-Memory (Lyu et al., 2026) constructs a pattern memory bank from training data and retrieves similar historical patterns via patch-level similarity, injecting them as additional context into time series foundation models without retraining. MEMTS (Yu et al., 2026b) goes further by replacing external retrieval with a Knowledge Persistence Module that internalizes domain-specific temporal dynamics as learnable latent prototypes, eliminating the $O(N)$ retrieval bottleneck at inference time.

The key trade-off in knowledge memory is between coverage and precision. External knowledge bases (RAG, KG) can be updated without retraining but incur retrieval latency and may retrieve irrelevant entries. Parameterized memory (TS-Memory, MEMTS) offers zero-latency access but requires retraining or fine-tuning to update. Among the systems reviewed, most adopt RAG for its flexibility; only MEMTS and TS-Memory pursue parameterization as an alternative. Knowledge memory answers “what is generally true”; the next subtype, procedural memory, addresses the distinct question of “what to do next time.”

6.6 Procedural and Reflective Memory

Procedural memory stores “how to do” rather than “what happened.” It captures strategies, workflows, code modifications, and reflective conclusions about why a particular approach succeeded or failed. Reflective memory, a closely related subtype, stores the lessons extracted from experience: which strategies are effective

under which conditions, which errors recur, and which heuristics generalize. The distinction from episodic memory is that episodic memory is retrospective (“what happened last time”) while procedural memory is prospective (“what to do next time”). MemCast (Tao et al., 2026a) provides the most complete example of procedural and reflective memory in the time series literature. Its Reasoning Wisdom layer distills successful and failed reasoning experiences into reusable strategy templates, while its General Laws layer captures domain-invariant regularities that apply regardless of the specific series. The Dynamic Confidence Adaptation mechanism updates the confidence of each memory entry based on its recent utility, so that stale or counterproductive strategies are gradually downweighted rather than explicitly removed.

Workflow memory stores distilled planning trajectories. CastFlow (Pan et al., 2026) records planning trajectories and tool-use patterns in a vector store, retrieving similar past workflows by embedding similarity to guide new forecasting plans. SEA-TS (Xu et al., 2026e) encodes code-review findings into a running prompt that prevents the same class of error from recurring in subsequent iterations, combining procedural memory (the corrected code pattern) with reflective memory (the diagnosis of why the error occurred). TS-Agent (Ang et al., 2025) maintains a Refinement Knowledge Bank that stores expert heuristics for preprocessing, training optimization, and hyperparameter tuning, providing procedural guidance that complements the episodic Case Bank.

Cross-episode reflective memory generalizes experience across sessions. FinCon (Yu et al., 2024) introduces conceptual verbal reinforcement: after each trading episode, the Manager agent abstracts successful and failed experiences into conceptual strategy templates that are stored and reused in subsequent episodes. MarsTSC (Li et al., 2026a) maintains a self-evolving knowledge base whose Generator, Reflector, and Modifier agents iteratively add, revise, and delete atomic knowledge points, with a discard mechanism that removes entries exceeding a staleness threshold. ATLAS (Papadakis et al., 2025) stores an optimization history H of (prompt, score) pairs that guides subsequent prompt evolution through its Adaptive-OPRO mechanism, with template separation ensuring that the runtime interface remains stable while the prompt content improves.

Reflection and debate provide another route to procedural memory. CryptoTrade (Li et al., 2024) dedicates a Reflection Agent that analyzes recent trading performance, identifies effective and ineffective strategies, and evaluates whether the agent is too aggressive or too conservative, storing improvement suggestions for future decisions. TradingAgents (Xiao et al., 2024) uses historical debate records from Bull and Bear researcher agents as procedural context for current market deliberation. ChatAD (Sun et al., 2026) evolves multi-turn instruction data through its TSEvol algorithm, producing task memory that improves cross-task generalization for anomaly detection, classification, and imputation. StockAgent (Zhang et al., 2025b) uses a summarizing manager agent that is responsible for aggregating analysis results and generating predictions, while a reflective management agent optimizes and adaptively adjusts the weights of the expert agents based on experiential feedback.

Internalization transfers procedural memory from external storage into model parameters. T3LLM (Su et al., 2025) uses a worker-reviewer-student framework where the Worker generates chain-of-thought reasoning, the Reviewer identifies and corrects the earliest error, and the corrected high-quality CoT is fine-tuned into a Student model, effectively converting external procedural traces into parametric knowledge. SocioDojo (Cheng and Chin, 2024) accumulates lifelong procedural memory that stores reusable analysis strategies, enabling the agent to improve across sessions without retaining raw interaction data.

Audit trails and session memory provide a lighter form of procedural memory. TSOchestra (Cao et al., 2025) records a complete optimization history as an audit trail, enabling the LLM controller to review past weight configurations and their performance when deciding whether to continue optimizing. A recent autonomous LLM-Agent framework (Buratto et al., 2026) stores full message histories in SQLite with optional FAISS indexing, supporting multi-turn sessions where earlier diagnostic results inform later decisions.

Recent position papers (Cheng et al., 2026a) formalize the structure of procedural memory. A recent position paper proposes a three-level hierarchy of instance-level, task-level, and domain-level memory, with retrieval mechanisms that selectively recall memories based on the current context. An agentic trading survey (Xia et al., 2026) views working memory, episodic memory, and semantic memory as the three layers of trading-agent memory, identifying procedural memory as the layer most responsible for cross-session strategy improvement.

6.7 Failure and Confidence Memory

Failure memory stores information about what went wrong: which strategies failed, which predictions were overconfident, and which patterns led to incorrect actions. Confidence memory stores the calibration structure that tells the agent how much to trust each stored pattern or strategy. These are the most scarce but most consequential memory subtypes: without them, an agent will repeat the same mistakes and cannot distinguish between high-confidence and low-confidence predictions. Hubble (Shi et al., 2026a) implements the most explicit failure memory in the time series literature. Its dual-channel RAG maintains both a success factor library and a failure factor library, so that the alpha factor generation process is guided by what has worked and explicitly guarded against what has not. The failure channel is essential: without it, the generator would repeatedly explore the same unproductive regions of the factor space. MemCast (Tao et al., 2026a) addresses confidence memory through its Dynamic Confidence Adaptation mechanism, which updates the confidence score of each memory entry based on whether the retrieved pattern led to a correct prediction. Entries that consistently lead to poor predictions are downweighted rather than deleted, preserving the possibility that they may become useful again under a different regime. SEA-TS (Xu et al., 2026e) encodes code-review findings into its running prompt, preventing the same class of error from recurring. This is a form of failure memory because the review explicitly identifies what went wrong (syntax errors, logical errors, performance regressions) and encodes this diagnosis for future avoidance. Nexus (Das et al., 2026) accumulates domain-specific calibration rules G through multi-fold backtesting, with a validation gate that prevents overfitting to the backtest data. The calibration rules serve as confidence memory: they tell the forecast synthesizer when to trust the macro-reasoning agent versus the micro-reasoning agent, and when to defer to the calibration agent’s corrections.

FinHEAR (Chen et al., 2025b) uses outcome-based refinement to adjust expert-informed precedents based on realized trading results, so that strategies that performed poorly in hindsight are downweighted. CryptoTrade (Li et al., 2024) stores strategy-improvement suggestions from its Reflection Agent, explicitly distinguishing between effective and ineffective strategies and adjusting the agent’s aggressiveness profile. CALM (Devireddy and Huang, 2025) uses an LLM-as-judge mechanism to determine whether a detected anomaly represents transient noise (to be removed from the training set) or a persistent regime shift (to be retained), effectively curating a failure-aware training memory.

The scarcity of failure memory designs is striking. Among the systems reviewed in this section (excluding three survey and position papers), Hubble is the only system with an explicit dual-channel architecture that maintains separate success and failure libraries, and only MemCast and Nexus implement explicit confidence calibration mechanisms. Several other systems implement partial failure awareness: SEA-TS encodes code-review findings to prevent recurring errors, FinHEAR downweights precedents based on realized outcomes, CryptoTrade distinguishes effective from ineffective strategies, and CALM curates a failure-aware training memory through LLM-as-judge filtering. However, none of these provides the explicit success/failure dichotomy or confidence calibration that would constitute a full failure and confidence memory. Most systems record what worked but not what failed, which means they cannot learn from mistakes and are vulnerable to repeating them. This gap is especially consequential in high-stakes domains such as financial trading, industrial control, and medical diagnosis, where the cost of repeating a failure can be large.

6.8 Memory Dynamics: Formation, Retrieval, and Evolution

The preceding subsections have described what is stored in each memory subtype. This subsection addresses how memories are formed, retrieved, and updated, synthesizing mechanisms that cut across all subtypes (see the bottom row of Fig. 6 for an overview of the formation–retrieval–evolution cycle).

Memory Formation. Memories are formed from three sources: interaction, training, and reflection. Interaction-formed memories arise from tool outputs, reasoning traces, and verification feedback generated during task execution. The evidence log in TS-Agent (Liu et al., 2025d), the decision state in Cast-R1 (Tao et al., 2026b), and the interaction trajectories in AgriWorld (Zhang et al., 2026b) are all formed during interaction. Training-formed memories are constructed before deployment. TS-Memory builds a pattern memory bank from training data by computing patch-level embeddings and storing representative patterns (Lyu et al., 2026). TS-RAG constructs a multi-domain retrieval knowledge base from historical context-window

and prediction-window pairs (Ning et al., 2025b). MEMTS internalizes domain-specific temporal dynamics as latent prototypes through a Knowledge Persistence Module trained on domain data (Yu et al., 2026b). Reflection-formed memories are produced by post-hoc analysis of outcomes. FinCon’s conceptual verbal reinforcement (Yu et al., 2024) and the Reflection Agent in cryptocurrency trading (Li et al., 2024) both extract strategy-level lessons from episode-level results.

Memory Retrieval. Retrieval mechanisms vary along two axes: the similarity metric and the retrieval architecture. For similarity, vector-embedding cosine similarity is the most common. CastFlow (Pan et al., 2026) retrieves similar planning trajectories by embedding similarity, and an Autonomous LLM-Agent uses FAISS (Buratto et al., 2026) to retrieve semantically relevant past messages. Dynamic time warping (DTW) is used when the statistical shape of the series matters more than its semantic description: SAGE (Kang et al., 2026) retrieves normal prototypes by DTW similarity. TS-RAG (Ning et al., 2025b) uses Euclidean distance in the embedding space of a pre-trained time series encoder. Hybrid retrieval combines data-centric and semantic similarity: RationaleTS (Liu et al., 2026b) unifies TabPFN-encoded statistical features with text-embedding-based semantic features. Conditional retrieval filters by task type, domain, or regime label: MemCast (Tao et al., 2026a) conditions retrieval on the current series characteristics, and FinMem (Yu et al., 2025b) adjusts retrieval weights based on the trader’s risk profile. Differentiable retrieval is used by Agentic RAG (Ravuru et al., 2024), which optimizes prompt-pool retrieval weights through gradient descent rather than relying on fixed similarity metrics.

Memory Evolution. Memories must be updated as distributions shift, strategies become obsolete, and new patterns emerge. Forgetting mechanisms range from simple to structured. FinMem (Yu et al., 2025b) applies differential forgetting rates across its three layers: high for daily data, moderate for news, and low for earnings reports. Sliding windows are used by AIVV (Kwon et al., 2026) and SocioDojo (Cheng and Chin, 2024) to bound context length. Confidence-based updating adjusts the weight of each memory entry rather than deleting it: MemCast’s (Tao et al., 2026a) Dynamic Confidence Adaptation downweights entries that lead to poor predictions. Strategy evolution mechanisms modify the content of memory rather than just its weight: ATLAS’s (Papadakis et al., 2025) Adaptive-OPRO evolves prompts based on historical performance, and SEA-TS’s (Xu et al., 2026e) running prompt refinement encodes code-review findings to prevent recurring errors. Cross-session transfer is implemented by SocioDojo’s (Cheng and Chin, 2024) lifelong procedural memory and by TS-Agent’s (Ang et al., 2025) online learning from case-bank updates.

These dynamics reveal a gap. Most systems implement either formation or retrieval but not both with equal sophistication, and evolution mechanisms are rare. Among the systems reviewed, MemCast is the only one that implements formation, retrieval, and evolution across multiple memory subtypes (episodic, procedural, and confidence). The position paper on agentic time series forecasting (Cheng et al., 2026a) explicitly calls for layered memory with instance-level, task-level, and domain-level retrieval, and the survey on LLM-driven time series analysis (Zhang et al., 2025c) identifies memory as one of the least developed components in current systems.

6.9 Evidence Map and Open Challenges

Tab. 8 summarizes the six memory subtypes, their defining characteristics, and representative implementations across the systems reviewed in this section.

Several patterns emerge from this summary. Context memory is widely implemented but remains shallow: most systems merely maintain session histories or evidence logs without explicit selection, compression, or forgetting mechanisms. Episodic memory appears prominently in anomaly detection and financial forecasting where case-based reasoning aligns with diagnostic workflows, yet similarity metrics often ignore statistical properties, and retrieval may miss numerically similar but semantically different series. Temporal memory is the most distinctive to time series agents but shows the largest implementation gap: explicit regime detection and temporal decay mechanisms remain rare, with FinMem, MemCast, and CALM as notable exceptions. Knowledge memory relies heavily on RAG; structured knowledge bases and knowledge graphs remain less common, and parameterized approaches such as MEMTS and TS-Memory offer zero-latency alternatives at the cost of update flexibility. Procedural memory shows strong potential but lacks benchmark evaluation for cross-session improvement; few systems measure whether agents actually avoid repeating mistakes. Failure

Table 8 Summary of memory subtypes in agentic time series systems: characteristics and representative systems.

Memory subtype	Characteristics	Representative systems
Context	Short-term task evidence, tool outputs, reasoning traces within a session; typically implemented as session logs or evidence buffers without explicit selection, compression, or forgetting	TS-Agent (Liu et al., 2025d); Cast-R1 (Tao et al., 2026b); AD-AGENT (Yang et al., 2025d); AIVV (Kwon et al., 2026)
Episodic	Complete case trajectories for analogical retrieval; case-based reasoning and pattern matching; similarity metrics must respect statistical properties rather than relying solely on text embeddings	MemCast (Tao et al., 2026a); SAGE (Kang et al., 2026); Argos (Gu et al., 2025); FinSrag (Xiao et al., 2025a); RationaleTS (Liu et al., 2026b)
Temporal	Regime-aware, scale-indexed, and season-aware patterns indexed by time scale; requires regime detection, temporal decay, and rare-event recall, the subtype most distinctive to time series agents	FinMem (Yu et al., 2025b); MemCast (Tao et al., 2026a); CALM (Devireddy and Huang, 2025); SocioDojo (Cheng and Chin, 2024); AgriWorld (Zhang et al., 2026b)
Knowledge	Domain rules, tool manuals, and causal structures delivered via RAG, knowledge graphs, or structured knowledge bases; also includes parameterized memory that internalizes domain knowledge into model weights	AutoLFM (Li et al., 2026b); ChatKG (Christino and Paulovich, 2024); ElliottAgents (Chudziak and Wawer, 2025); TS-Memory (Lyu et al., 2026); MEMTS (Yu et al., 2026b)
Procedural	Strategies, workflows, code modifications, and reflective conclusions about what works; stores “how to do” rather than “what happened”; supports cross-session improvement through strategy evolution and prompt refinement	MemCast (Tao et al., 2026a); CastFlow (Pan et al., 2026); SEA-TS (Xu et al., 2026e); FinCon (Yu et al., 2024); ATLAS (Papadakis et al., 2025); T3LLM (Su et al., 2025)
Failure	Failure patterns, uncertainty estimates, and confidence calibration signals; the most scarce but most consequential subtype; without it, agents repeat mistakes and cannot distinguish high- from low-confidence predictions	Hubble (Shi et al., 2026a); MemCast (Tao et al., 2026a); Nexus (Das et al., 2026); SEA-TS (Xu et al., 2026e)

and confidence memory represents the most critical gap: most systems record successes but not failures, and Hubble remains the only system with an explicit dual-channel success/failure architecture.

These patterns point to eight open challenges for memory in time series agents:

- **Memory Evaluation.** To date, to our best knowledge, no benchmark specifically evaluates the quality of an agent’s memory system. Existing evaluations measure prediction accuracy or task completion but not whether the memory is well-formed, efficiently retrieved, or correctly updated. A memory evaluation protocol would need to test formation (does the agent store the right entries?), retrieval (does it retrieve the most relevant entries?), and evolution (does it update or forget entries when distributions shift?).
- **Regime-Aware Memory.** Current systems rarely tag memory entries with regime labels or monitor distribution shifts in real time. A regime-aware memory system would need to detect when a regime change has occurred, adjust the retrieval weights of entries associated with the old regime, and promote entries from historically similar regimes. FinMem’s layered forgetting rates and CALM’s drift curation are initial steps, but neither implements automatic regime detection.
- **Cross-Session Memory Reuse.** Most systems operate within a single session and discard memory when the session ends. SocioDojo’s lifelong memory and TS-Agent’s (Ang et al., 2025) online learning are exceptions, but they do not address the challenge of transferring memory across tasks or domains.
- **Memory-Reasoning Alignment.** Retrieved memory is useful only if it aligns with the current reasoning needs. A memory entry that is statistically similar to the current series but irrelevant to the current task can be worse than no memory at all. Hybrid retrieval mechanisms such as those in RationaleTS (Liu et al., 2026b) and conditional retrieval in MemCast (Tao et al., 2026a) are partial solutions, but the general problem of aligning memory retrieval with reasoning intent remains open.
- **Memory Cost-Efficiency.** Storing, indexing, and retrieving memory incurs computational cost. Parame-

terized memory (MEMTS, TS-Memory) offers low-latency access but requires retraining for updates. External memory (RAG, knowledge bases) is flexible but incurs retrieval latency and storage overhead. The trade-off between memory quality and computational cost has not been systematically studied in the time series agent literature.

- **Multi-Agent Shared Memory.** In multi-agent systems, memory consistency becomes a concern. AD-AGENT’s short-term shared memory (Yang et al., 2025d) and the shared dynamic memory in multimodal anomaly detection (Xu et al., 2026f) are initial designs, but they do not address the challenge of maintaining a consistent and up-to-date shared memory when multiple agents read and write concurrently.
- **Memory Conflict Resolution.** When multiple retrieved memories contradict each other, for example, a seasonal pattern suggesting upward trend while a recent regime-shift signal suggests decline, the agent must decide which memory to trust. Current systems do not provide explicit conflict-resolution mechanisms; most delegate this decision to the LLM’s parametric reasoning, which may not be reliable when the contradiction involves numerical or statistical evidence rather than textual plausibility.
- **Cross-Domain Memory Transfer.** Whether strategies, patterns, or failure cases learned in one domain (e.g., financial trading) can transfer to another (e.g., energy forecasting) remains an open question. Domain-specific knowledge memory is often not portable, but procedural memory, general strategies for diagnosis, planning, and error recovery, may be. No existing system explicitly evaluates cross-domain transfer of any memory subtype.

Taken together, these challenges reinforce the central claim of this section: without structured selection, indexing, updating, forgetting, and reuse, a log of past observations does not constitute a memory system. This analysis reveals that most current implementations stop at context memory or simple episodic retrieval, leaving temporal, procedural, and failure memory largely unexplored, a gap that directly impacts agent reliability in high-stakes domains such as financial trading, industrial control, and medical diagnosis. The memory layer is also the primary channel through which verification feedback returns to the system: when a prediction fails or a tool output is rejected, the resulting signal must be written back into confidence and failure memory so that the agent can avoid repeating the same error. Building memory systems that close this loop, from evidence accumulation through strategy refinement to failure-aware adaptation, is a prerequisite for the reliable, long-lived time series agents that the following sections on world models and reliability will require.

7 Time Series World Models: Simulating Dynamic Temporal Environments

Time series world modeling concerns how temporal systems move from representing observed sequences toward modeling the dynamic environments that generate them. Conventional time series models are usually optimized for task-specific outputs, such as predictions, classifications, risk scores, or decisions. World modeling asks a different question: what is the temporal environment being modeled, how may it evolve, and how would its evolution change under different conditions? This distinction matters for agentic time series systems because an agent often needs to reason about consequences, interventions, and alternative futures rather than merely produce an output for the current task.

Let $x_{1:t}$ and $c_{1:t}$ denote, respectively, the observed temporal evidence and the relevant context available up to time t (as in Sec. 2.3). A world model maps this information into a state representation

$$z_t = \text{Enc}_\psi(x_{1:t}, c_{1:t}), \quad (17)$$

and characterizes how this state evolves under condition-changing factors:

$$z_{t+1} \sim p_\theta(\cdot \mid z_t, \xi_t). \quad (18)$$

Here z_t denotes the system’s representation of the temporal environment at time t , which may be latent, structured, symbolic, observation-based, or simulator-defined. The variable ξ_t denotes a factor that affects the environment’s evolution, such as an agent action a_t , an external event, or a contextual condition, depending on the setting. The purpose of z_t is not necessarily to reconstruct raw observations, but to provide a state

Table 9 Current routes toward time series world models. Each route exposes a different interface for representing, simulating, comparing, or acting on temporal environments.

Route	World-model interface	Representative systems	Core capability and limitation
Environment understanding	Structured state over temporal covariates, dependencies, and domain context.	Augur (Cui et al., 2025).	Organizes the environment for relational reasoning, but inferred dependencies do not establish controllable dynamics or intervention-level causality.
Temporal simulation	Probabilistic rollout from history or generation conditioned on textual descriptions and semantic prototypes.	Chronicle (Quinlan et al., 2026); BRIDGE (Li et al., 2025).	Produces plausible or controllable futures, but the mechanism connecting conditions to temporal transitions may remain implicit.
Counterfactual simulation	Baseline and alternative trajectories under factual or hypothetical future conditions.	Counterfactual Time Series Forecasting (Gu et al., 2026).	Supports what-if comparison, but alternative futures are unobserved and textual conditions do not identify the causal pathway.
Executable grounding	Tool- or simulator-defined environment with callable actions, returned artifacts, and verification signals.	AgriWorld (Zhang et al., 2026b).	Enables intervention and execution-based verification, but requires costly domain engineering and remains difficult to scale.

representation from which the system can reason about how the temporal environment may evolve under different conditions.

Recent visual and embodied world models provide a useful reference point for this formulation. Genie-style generative environments and newer world foundation models show how learned representations can support action-controllable or physically grounded simulation in visual domains (Bruce et al., 2024; NVIDIA et al., 2025; Team et al., 2025). Predictive-representation models such as V-JEPA 2 further suggest that world modeling need not be tied to reconstructing observations directly; useful dynamics can also be learned in abstract representation spaces (Assran et al., 2025). Time series world modeling shares this general logic of state representation, transition, and imagined rollout, but the modeled environment is different. Rather than visual scenes, robotic settings, or interactive 3D worlds, the target is often a partially observed temporal system shaped by multivariate dependencies, exogenous conditions, domain constraints, and possible interventions.

This difference motivates a distinct set of time series world-modeling directions. Current studies approach the objective through environment understanding (Cui et al., 2025), temporal simulation via multimodal forecasting or controlled generation (Quinlan et al., 2026; Li et al., 2025), counterfactual reasoning under changed conditions (Gu et al., 2026), and executable grounding in constructed domain environments (Zhang et al., 2026b). The following discussion synthesizes these directions and clarifies how existing work contributes to time series world modeling.

Table 9 summarizes these complementary routes. Environment understanding structures the state, temporal simulation rolls it forward, counterfactual simulation compares changed conditions, and executable grounding exposes environment transitions to agent actions and verification.

7.1 Environment Understanding: Structuring Temporal Dependencies

Environment understanding concerns what must be represented for a temporal environment to be modeled. A time series is not only an ordered sequence of values; it may also reflect interactions among variables, delayed effects, external events, and domain constraints. For a temporal world model, such a structure is important because future evolution is shaped not only by temporal continuation but also by how components of the environment influence one another.

Augur (Cui et al., 2025) represents this direction. It uses large language models to help model directed associations among temporal covariates. Rather than treating covariates as flat auxiliary inputs, Augur

organizes them into relational structures that make the environment state more explicit. Its value for world modeling lies in giving this state an internal organization: possible futures are shaped not only by the continuation of past values, but also by how components of the environment are related. At the same time, these inferred relations should be treated as modeled dependencies or causal hypotheses rather than fully validated mechanisms. Thus, Augur illustrates how relational environment understanding can enrich temporal world states and provide constraints for later reasoning about temporal evolution.

7.2 Temporal Simulation: From Forecasting to Controlled Generation

Temporal simulation concerns how a modeled temporal environment can be rolled forward from available evidence. Here, simulation does not require a complete physical simulator; it refers more broadly to producing future trajectories that are consistent with observed history, contextual information, or specified semantic conditions. Forecasting becomes relevant to world modeling when it is interpreted as such a rollout rather than only as a task-specific output. Existing methods differ in how explicitly they expose this process: forecasting-oriented models extend observed histories under uncertainty, whereas controlled generation synthesizes trajectories according to semantic constraints.

Chronicle (Quinlan et al., 2026) represents a forecasting-oriented route. It learns a shared modeling space for language and time series and uses this shared backbone for autoregressive time series forecasting. Given historical temporal evidence, Chronicle can predict future time series patches and produce probabilistic forecasts through quantile outputs. This combination of multimodal context and probabilistic rollout allows the model to use numerical history and available textual context to estimate how a temporal process may continue under uncertainty.

However, this rollout remains different from explicit intervention-based world modeling. Chronicle can incorporate textual information into temporal modeling, but such text mainly functions as contextual evidence rather than as an explicitly modeled event, action, or intervention. The model therefore approximates future trajectories from multimodal evidence, rather than learning how a specified condition-changing factor ξ_t transforms a world state from z_t to z_{t+1} .

BRIDGE (Li et al., 2025) represents a more explicitly generative route. It uses textual descriptions together with semantic prototypes to control time series generation, connecting natural-language conditions and pattern-level temporal representations with generated trajectories. Its contribution is not only that text can guide generation, but also that semantic prototypes provide reusable pattern-level structures for producing time series under different descriptions.

Together, Chronicle and BRIDGE show how temporal simulation can be approximated without a complete physical simulator. Chronicle rolls forward from observed histories under uncertainty, while BRIDGE generates trajectories under semantic conditions. The common difficulty is that plausibility and controllability do not automatically identify the mechanism of evolution. A generated trajectory may be consistent with the observed history or satisfy a textual condition, but the link between that condition and the underlying temporal dynamics can remain implicit. This leaves an important gap for world modeling: the model may approximate how a process could continue, while still lacking a grounded account of why the environment would evolve in that particular way.

7.3 Counterfactual Simulation: Alternative Futures Under Changed Conditions

This direction extends temporal simulation from producing possible futures to comparing how futures diverge when conditions change. Forecasting-oriented rollout estimates how a process may continue from observed history, and controlled generation can synthesize trajectories under specified descriptions. The additional step here is contrastive: the model must relate a baseline evolution to an alternative one and estimate what difference the changed condition makes. This contrastive structure makes counterfactual simulation closer to causal reasoning, because the question is no longer only whether a future is plausible, but how a change in condition is associated with a change in future evolution.

Counterfactual time series forecasting (Gu et al., 2026) provides a direct formulation of this problem by using natural language to specify factual or hypothetical future conditions. The model takes historical observations

together with historical and future textual conditions, and generates a future trajectory that both preserves the intrinsic characteristics of the past sequence and reflects the external influence described by the future text. Compared with binary or categorical interventions, textual conditions can express more complex real-world events and scenarios; compared with ordinary controlled generation, the goal is not only to satisfy a condition, but to estimate how the future would differ when that condition changes. This makes the task a useful approximation to causal what-if reasoning in temporal environments.

The main limitation is that this remains an approximation rather than a fully identified causal model. The text may describe a what-if event, but it does not necessarily specify the mechanism or pathway through which the event affects the temporal process. Because the alternative future is usually unobserved, evaluation must rely on indirect signals such as semantic consistency, attribution quality, or domain constraints. Thus, textual counterfactual forecasting moves time series modeling toward causal world-modeling questions, while leaving grounding and validation as open challenges.

7.4 Executable Grounding: Tool-Exposed World-Model Interfaces

Executable grounding refers to executable domain environments that can be used by agents as temporal world models. The preceding subsections focus on learned approximations of temporal dependencies, future rollout, and counterfactual evolution. Here the emphasis is different: the temporal environment is organized outside the language model as a computational substrate, and the agent interacts with it through executable interfaces. In this setting, the world model is valuable not only because it encodes domain structure, but also because an agent can use it: domain data, executable procedures, returned artifacts, and verification results become part of the reasoning loop.

AgriWorld (Zhang et al., 2026b) represents this route. It provides a Python-based agricultural world environment that integrates heterogeneous agricultural data, remote-sensing time series analysis, crop simulation, task-specific prediction, and verification within a unified world-tools-protocol framework. In this formulation, AgriWorld is the constructed agricultural environment, while its tools serve as the executable access layer through which the agent can operate on that environment. The Agro-Reflective agent uses this layer to issue code-based operations, observe returned artifacts, and refine its analysis through execution feedback. This separation is central to executable grounding: the agent can reason with a transparent and monitorable world substrate, while intermediate claims remain traceable to domain procedures and verification signals.

At the same time, this route is difficult to reproduce and scale. Constructing such a world requires substantial domain engineering, because the relevant data sources, domain procedures, simulators, predictors, and verification rules must be explicitly defined and integrated. When an important process cannot be captured by well-defined tools or a reliable execution environment, the constructed world model has limited ability to extend to that process. Thus, AgriWorld demonstrates an alternative route to time series world modeling: instead of learning the whole temporal environment as a unified transition model, it constructs an executable domain world that an agent can use for grounded reasoning. Its strength comes from the transparency and operability of this constructed world, while its main bottleneck lies in the cost of building, maintaining, and scaling such an environment.

7.5 Synthesis and Outlook

Taken together, these studies suggest that time series world modeling is not a fixed model family, but an emerging capability layer that extends temporal modeling toward environment-level reasoning. Current approaches develop this capability from different directions, including structured environment representation, temporal simulation, counterfactual comparison, and executable grounding. However, each direction still faces its own constraints: relational methods can support forecasting but do not by themselves provide fully controllable rollout, simulation methods may leave dynamics implicit, counterfactual methods are difficult to validate, and executable environments are costly to scale. A natural next step is to combine these capabilities, rather than treating them as separate routes, so that temporal world models can better represent, simulate, compare, and verify possible evolutions of dynamic systems.

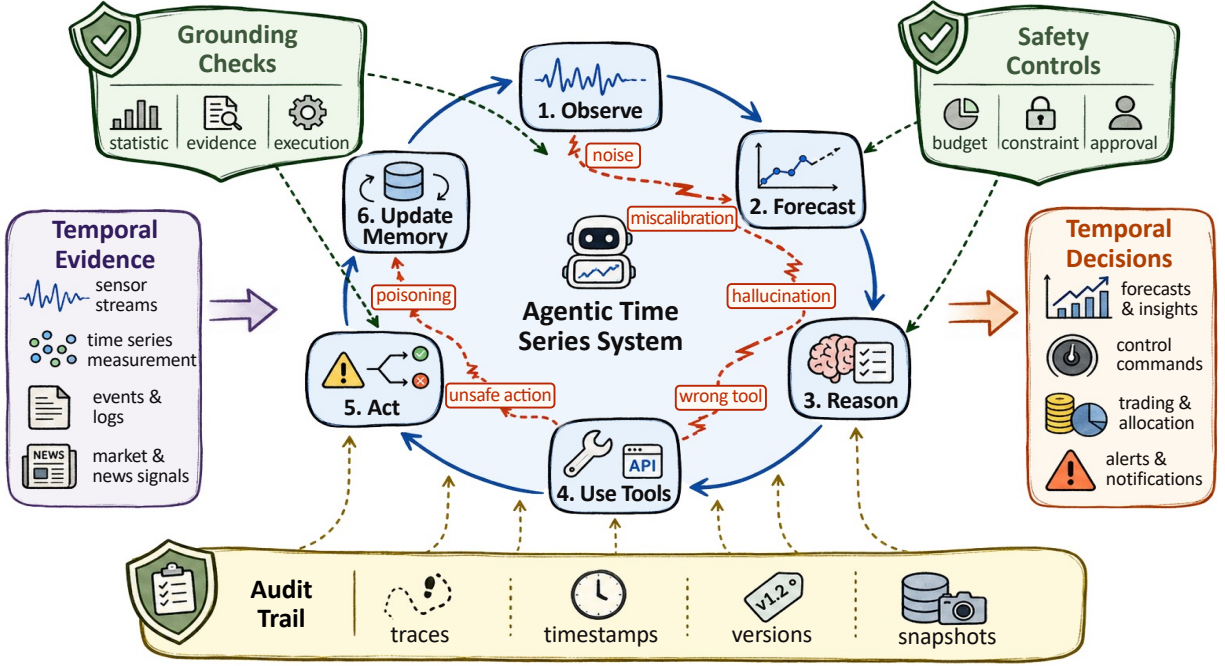


Figure 7 Failure propagation and reliability controls in agentic time series systems. Errors can enter the system through temporal evidence, propagate through forecasting, reasoning, tool use, actions, and memory updates. They can then become reinforced by future feedback. Mechanisms for interrupting this propagation include grounding checks, safety controls, and audit trails.

8 Reliability and Trustworthiness in Agentic Time Series Systems

Reliability is not a downstream evaluation add-on for agentic time series systems. It is a design constraint that governs how an agent observes temporal evidence, reasons about uncertain dynamics, invokes tools, updates memory, and acts under delayed feedback (Cheng et al., 2026a; Ning et al., 2025a). This setting differs from conventional forecasting. A model-centric forecaster is often evaluated by point or probabilistic error. An agent can fail by selecting the wrong tool, fabricating an intermediate statistic, relying on stale memory, overreacting to a transient shock, leaking private context through an external API, or taking an action whose consequence is observed only after a delay (Liu et al., 2025d; Debenedetti et al., 2024; Shi et al., 2026a; Xia et al., 2026). We use *failure propagation* to describe this risk. An error in temporal interpretation can trigger a wrong tool call. A wrong tool result can become an unsupported rationale. That rationale can then become an unsafe action (Wu et al., 2026b; Tao et al., 2026b; Zhan et al., 2026).

We organize reliability as a stack. Forecasting quality is the base condition. Reasoning correctness, tool-use reliability, and grounding determine whether intermediate claims are supported. Robustness and security determine whether the system handles shift and adversarial inputs. Decision safety determines whether predictions can be converted into actions. Human alignment and auditability determine whether the system can be inspected, corrected, and governed. The stack is a set of contracts between layers rather than a metric list. Each layer constrains what evidence can be passed upward, what uncertainty must be preserved, and which actions require escalation. Position and vision papers argue that time series forecasting is moving from single-pass prediction toward perception, planning, action, reflection, and memory. They also argue that trustworthy temporal intelligence requires faithful evaluation and system-level reasoning rather than language-only explanations (Cheng et al., 2026a; Ning et al., 2025a; Dai et al., 2026a). Tab. 10 summarizes this view as a diagnostic checklist.

Table 10 A reliability stack for agentic time series systems. The table is organized as a diagnostic checklist that identifies what to inspect, how failure typically appears, and which time series or agentic time series works provide representative anchors.

Reliability layer	What must be checked	Characteristic failure	Representative anchors
Forecasting quality	Accuracy, calibration, horizon stability, leakage control	Strong average error but weak tails, weak long horizons, or contaminated splits	GIFT-Eval (Aksu et al., 2024), TFRBench (Ahamed et al., 2026b), FLAIRR-TS (Jalori et al., 2025), TimeSeriesGym (Cai et al., 2025b)
Robustness and stress	Shift tolerance, extreme-event behavior, sensor corruption, recovery after breaks	Regime break, black-swan shock, corrupted stream, or aging dynamics causes confident failure	FOIL (Liu et al., 2024a), MAA-TSF (Qiao et al., 2025), SocNet (Dai et al., 2025a), SocGate (Dai et al., 2025b)
Reasoning correctness	Arithmetic validity, temporal ordering, causal discipline, health-domain consistency	Correct-looking answer with invalid steps, reversed event order, or correlation-as-causation	TS-Agent (Liu et al., 2025d), VeriTime (Zhou et al., 2026), TSAIA (Ye et al., 2025), TimeSeriesExamAgent (Gwiazda et al., 2026)
Tool-use reliability	Tool routing, argument validity, execution recovery, agreement with tool outputs	Wrong tool, malformed call, failed execution, or fabricated result after a failed call	TimeART (Wu et al., 2026b), Cast-R1 (Tao et al., 2026b), TSAG (Kolonin et al., 2026), TimeSeriesGym (Cai et al., 2025b)
Hallucination and grounding	Support from raw series, event context, statistics, retrieved evidence, executed tools	Invented trend, event, seasonality, causal story, or anomaly explanation	CaTS-Bench (Zhou et al., 2025a), AXIS (Lan et al., 2025), Argos (Gu et al., 2025), AnomaMind (Tao et al., 2026c)
Security and provenance	Instruction-data isolation, event provenance, permissioned tool access, poisoned-memory resistance	Untrusted news, logs, tool outputs, or market narratives steer forecasts or actions	ToolHijacker (Shi et al., 2025c), AgentDojo (Debenedetti et al., 2024), ASB (Zhang et al., 2025a), Hubble (Shi et al., 2026a)
Decision safety	Constraint satisfaction, risk budgets, abstention quality, escalation policy	Useful forecast becomes unsafe action through leverage, over-intervention, or ignored uncertainty	FinHEAR (Chen et al., 2025b), Trading-R1 (Xiao et al., 2025b), TradingAgents (Xiao et al., 2024), Hubble (Shi et al., 2026a)
Auditability	Reproducible traces, seeds, data timestamps, prompts, tool versions, generated artifacts	Fluent summary without replayable evidence or comparable experimental protocol	TimeSeriesGym (Cai et al., 2025b), Structured financial workflows (Ang et al., 2025), TimeSeek (Mostafa et al., 2026), AlphaCast (Zhang et al., 2026a), Agentic Trading (Xia et al., 2026)

8.1 From Predictive Accuracy to a Reliability Stack

The first layer is *forecasting quality*. Agentic systems should not use autonomy to hide weak predictive performance. They still need calibrated point and distributional forecasts across domains, frequencies, horizons, and covariate settings. GIFT-Eval provides zero-shot evaluation over datasets from multiple domains and reports metrics such as MASE and CRPS (Aksu et al., 2024; Dai et al., 2026b). Forecasting quality is necessary but not sufficient. TemporalBench shows that numerical forecasting performance does not imply contextual or event-informed temporal reasoning when external conditions must be interpreted and used to revise predictions (Weng et al., 2026). TFRBench evaluates forecasting rationales over trends, cross-channel dependencies, and external events rather than treating forecasting systems as black boxes (Ahamed et al., 2026b). FLAIRR-TS and TimeSeriesGym broaden evaluation toward retrieval-refinement workflows and ML engineering artifacts such as code, models, and submissions, since deployed agents execute workflows rather

than only output arrays (Jalori et al., 2025; Cai et al., 2025b). Predictive metrics define the lowest layer contract. They certify numerical output under a specified data protocol, but not the validity of the reasoning or action policy built on top of it.

The second layer is *reasoning correctness*. Time series agents often produce natural-language rationales, causal claims, diagnostic summaries, and action recommendations. These statements must be checked against raw sequences, statistical tests, retrieved events, and tool outputs. TS-Agent keeps the LLM in an evidence gathering and synthesis role. It delegates statistical extraction to atomic time series operators, maintains an evidence log, and applies self-critic and quality-gate checks to reduce hallucination and knowledge leakage (Liu et al., 2025d). VeriTime synthesizes time series chain-of-thought data with verifiable process annotations. MMTS-Bench and HEARTS expose weaknesses in structural awareness, cross-modal alignment, and health-signal reasoning that generic language benchmarks may not show (Zhou et al., 2026; Yin et al., 2026; Li et al., 2026c). TSAIA tests multi-step time series assistants on constraint-aware forecasting, anomaly detection, causal discovery, and decision tasks. TimeSeriesExamAgent uses agentic benchmark generation to probe pattern recognition, noise understanding, similarity, anomaly detection, and causality across synthetic and real-world settings (Ye et al., 2025; Gwiazda et al., 2026). TimeART, TS-Debate, and Cast-R1 move reliability into the agent policy. TimeART trains tool-augmented reasoning trajectories, TS-Debate uses reviewer agents and programmatic verification to reduce numeric hallucinations, and Cast-R1 frames forecasting as a sequential decision policy that decides when and how to call tools (Wu et al., 2026b; Trirat et al., 2026; Tao et al., 2026b). These works indicate that temporal reasoning should be evaluated at the process level, not only by the final answer.

The third layer is *tool-use reliability*. Time series agents call tools for decomposition, anomaly detection, correlation, causal discovery, forecasting, retrieval, and simulation. A tool-using agent can fail by choosing the wrong tool, passing invalid arguments, ignoring an execution error, or summarizing a tool result incorrectly. TimeSeriesScientist, TimeCopilot, and TSAG provide three responses. They decompose forecasting into specialized agents and reports, route across time series foundation models and validation routines, and measure financial tool selection, faithfulness, and hallucination rates with verifiable quantitative tools (Zhao et al., 2025; Garza and Rosillo, 2025; Kolonin et al., 2026). AD-AGENT, AutoLFM, and AION extend this view from isolated tool calls to executable workspaces, domain tools, shared memory, validation interfaces, and layered review (Yang et al., 2025d; Li et al., 2026b; Zhan et al., 2026). Multi-agent workflows also introduce coordination overhead and brittle phase ordering. Adjacent work on skill distillation shows that compressing multi-agent behavior into single-agent skills can reduce cost and latency, but the benefit is metric-dependent rather than automatic (Xu et al., 2026b). In such settings, the reliable artifact is not only a prediction but a trace. The trace should record which evidence was collected, which tools were invoked, which assumptions were made, which errors were observed, and which checks passed. A tool result without an execution trace is closer to an unverifiable claim than to a scientific artifact.

8.2 Robustness Under Temporal Shift and Extreme Events

Robustness is difficult for time series because the environment is non-stationary. Distribution shift may appear as slow drift, abrupt regime change, seasonal phase shift, missingness, sensor replacement, policy intervention, or an exogenous shock (Liu et al., 2024a; Fan et al., 2024; Weng et al., 2026). In finance, this includes market crashes, liquidity droughts, volatility spikes, and other black-swan-like events where the agent must distinguish a regime break from noise (Kolonin et al., 2026; Ang et al., 2025). In healthcare or industrial monitoring, failures may arise when patient state, machine behavior, or operating context changes outside the training distribution (Li et al., 2026c; Dai et al., 2025a,b). For agentic systems, robustness is not only model robustness but also *policy robustness*. The agent must decide whether to trust the forecast, invoke another tool, widen uncertainty, abstain, or ask for human review (Tao et al., 2026a; Qiao et al., 2025). A reliable agent needs more than a low average error. It needs stress tests, worst-case slices, tail-risk metrics, and uncertainty-aware escalation (Mostafa et al., 2026; Nechepurenko and Shuvalov, 2026).

Agentic architectures can support robustness, but they also add failure modes. An agent can detect shift, retrieve related historical episodes, switch forecasting engines, widen prediction intervals, and ask for human review. MemCast reuses historical patterns and reasoning experience. Multi-agent adversarial forecasting studies competition between heterogeneous generators and discriminators under perturbed dynamics (Tao

et al., 2026a; Qiao et al., 2025). Domain constraints can serve as robustness anchors. In battery state-of-charge estimation, physics-guided and physics-gated architectures improve stability under temperature variation, sensor noise, incorrect initial values, aging, and long multicycle operation (Dai et al., 2025a,b). The agent may also overfit recent memory, repeatedly call a tool whose assumptions are violated, or treat rare events as outliers to be smoothed away. This concern is consistent with critiques that retrieval-heavy contextual memory behaves more like lookup than true memory, creating generalization ceilings and poisoning risks when stored content is adversarial or stale (Xu et al., 2026a). Robustness evaluation should report aggregate error, regime-conditional performance, calibration under shift, degradation curves as anomaly severity increases, and recovery time after a distribution break.

8.3 Hallucination, Grounding, and Faithful Explanations

Hallucination in time series agents has a domain-specific form. The agent may invent a trend that is not present, claim seasonality without spectral support, attribute a spike to an event that occurred later, state a causal relation from correlation, or describe a tool result that was never computed. This is risky because the output may appear quantitative and may trigger downstream actions. Tool grounding is the main mitigation pattern in current work. TS-Agent preserves the native numerical sequence and requires answers to be supported by gathered statistical evidence (Liu et al., 2025d). TSAG uses external quantitative tools for financial question answering and evaluates tool selection accuracy, faithfulness, and hallucination (Kolonin et al., 2026). CaTS-Bench and AXIS evaluate numeric fidelity, contextual grounding, and anomaly explanations. They show that fluent time series descriptions can still be misaligned with the underlying signal (Zhou et al., 2025a; Lan et al., 2025). AgriWorld exposes agronomic tools through a world-tools protocol and uses an execute-observe-refine loop so that the agent can correct itself against executable results (Zhang et al., 2026b).

Faithful explanations should be separated from plausible explanations (Zhou et al., 2025a; Lan et al., 2025). A useful explanation should satisfy three checks. *Evidence grounding* requires each factual claim to link to a time window, statistic, retrieved document, or executed tool output (Liu et al., 2025d; Zhang et al., 2026b). *Temporal consistency* requires causes to precede effects and horizons to remain distinct (Weng et al., 2026; Ye et al., 2025). *Decision relevance* requires the explanation to support the recommended action or uncertainty statement (Chen et al., 2025b; Xiao et al., 2025b). For high-stakes domains, explanation quality should be evaluated with structured rubrics and, when possible, deterministic validators rather than only LLM-as-judge scores (Li et al., 2026c; Ahamed et al., 2026b). This matters for anomaly explanation, clinical temporal reasoning, and financial forecasting, where polished text can conceal unsupported claims (Lan et al., 2025; Li et al., 2026c; Kolonin et al., 2026). Argos and AnomaMind treat anomaly detection as an iterative diagnostic process with generated rules, tool-augmented evidence, review, and self-reflection rather than a single discriminative label (Gu et al., 2025; Tao et al., 2026c).

8.4 Safety, Security, and Adversarial Threats

Agentic time series systems inherit the security risks of LLM agents and add temporal-domain attack surfaces. Many systems read untrusted text streams such as news, earnings reports, clinical notes, maintenance logs, web pages, or user-provided incident descriptions. If these inputs are placed in the same context as system instructions, an attacker can attempt prompt injection to alter tool choices, leak memory, or trigger unsafe actions (Debenedetti et al., 2024; Zhang et al., 2025a). Tool-using agents also face tool-description attacks and protocol-level exploits. Malicious tool metadata can bias tool selection, and weak host-to-tool or agent-to-agent interfaces can create unauthorized data flows (Ferrag et al., 2025; Shi et al., 2025c). Adjacent adversarial-learning evidence shows that transferability can allow a single surrogate to induce targeted failures across models without target-specific queries. This suggests that black-box temporal agents should not rely on model obscurity as a defense (Xu et al., 2025a). These risks increase when the tool is a trading interface, control actuator, database connector, or code executor rather than a search API.

Time-series-specific adversarial threats include corrupted sensor streams, adversarial spikes, manipulated event logs, poisoned retrieval memories, and crafted market narratives designed to move an agent’s forecast or position. Backdoor research provides an adjacent warning. Poisoned data can remain stealthy even when clean accuracy is barely affected, which is relevant to agents that ingest external logs, user annotations, or retrieved examples (Xu et al., 2026d). A security design should impose least-privilege tool access, typed tool schemas,

sandboxed code execution, provenance tracking for retrieved context, separation between instructions and data, and approval gates for irreversible actions. Hubble provides a financial example of deterministic guardrails. Factor expressions generated by an LLM are constrained by a domain-specific language, checked through an AST sandbox, and stored with success and failure feedback to improve reproducibility and safety (Shi et al., 2026a). Defense work in backdoor security suggests two transferable design patterns for agentic time series systems. One is separating poisoned data from clean data before model reuse. The other is decoupling safety from the possibly compromised task model through an external semantic auditor (Xu et al., 2025b, 2026c). Safety should be treated as an architecture property rather than a post-hoc moderation layer. Security and grounding are coupled because a claim is not grounded unless the system can state where the supporting evidence came from, whether it was executable or retrieved, and which trust policy governed its use.

8.5 Decision Safety and Risk Control

A reliability problem appears when a time series agent acts. In decision-centric settings, the target is not only an accurate forecast but a policy that respects risk constraints under delayed feedback. Trading agents must manage drawdown, turnover, exposure, leverage, transaction cost, and market impact. Energy and traffic agents must avoid constraint violations. Clinical agents must remain conservative when uncertainty is high. AIVV demonstrates a verification-and-validation pattern for autonomous systems. Mathematically flagged anomalies are escalated to a role-specialized LLM council that validates faults against natural-language requirements and produces auditable V&V artifacts (Kwon et al., 2026). Financial reasoning agents such as FinHEAR, TradingAgents, and Trading-R1 move toward role-specialized analysis, risk-aware debate, confidence-aware sizing, or reinforcement-trained trading decisions rather than unconstrained prediction (Chen et al., 2025b; Xiao et al., 2024, 2025b).

For agentic time series systems, risk control should be explicit in the interface between prediction and action. The agent should output uncertainty, confidence, evidence, candidate actions, and constraint checks, not only a single recommendation (Chen et al., 2025b; Xiao et al., 2025b). One design is to place a risk manager between the reasoning agent and the external environment. This module can reject actions that violate hard constraints, shrink position sizes when calibration is poor, require human approval when the scenario resembles known failure cases, and log decisions for later audit (Shi et al., 2026a; Xiao et al., 2024). The risk manager should evaluate *decision regret*, not only forecast error. An inaccurate forecast may be harmless if the agent abstains. A moderately accurate forecast may be dangerous if it is translated into excessive leverage or an irreversible intervention (Nechepurenko and Shuvalov, 2026; Xia et al., 2026). Reliability is a closed-loop property in this setting. The system must monitor outcomes, update memory with failures as well as successes, and revise future behavior when a risk assumption is violated (Tao et al., 2026a; Zhang et al., 2026a).

8.6 Human Alignment and Auditability

Trustworthy time series agents should be legible to domain experts (Ning et al., 2025a; Zhang et al., 2026a). Producing a fluent explanation is not enough. The agent should expose the evidence it used, the tools it called, the assumptions it made, the uncertainty it estimated, and the alternatives it rejected (Liu et al., 2025d; Cai et al., 2025b; Zhan et al., 2026). Human alignment is not only a preference-alignment problem. It is also an interface problem (Cheng et al., 2026a; Ning et al., 2025a). In finance, medicine, energy, and industrial monitoring, users need to edit horizons, constraints, risk budgets, covariates, and escalation rules (Chen et al., 2025b; Li et al., 2026c,b). AlphaCast points in this direction by combining human wisdom with LLM-based co-reasoning for forecasting (Zhang et al., 2026a). Audit-oriented trading surveys show that reported agent performance is difficult to compare or trust without time-consistent splits, execution semantics, transaction-cost models, and artifact logs (Xia et al., 2026).

Auditability connects deployment to improvement. A reliable agent should preserve prompts, retrieved context, tool versions, code artifacts, random seeds, input timestamps, memory state, and final decisions so that failures can be replayed rather than only described (Cai et al., 2025b; Zhan et al., 2026; Ang et al., 2025). This requirement matters for self-reflective or memory-augmented agents, where future behavior depends on previous traces (Tao et al., 2026a; Xu et al., 2026a). If the system stores only successful examples, it may amplify survivorship bias. If it stores unverified rationales, it may institutionalize hallucinations (Liu

et al., 2025d; Zhou et al., 2025a). A trustworthy memory update should distinguish raw observations, verified evidence, user corrections, failed tool calls, and reusable procedural lessons (Cheng et al., 2026a; Ning et al., 2025a). For publication and deployment, the audit unit should be the full episode rather than the final answer. Observation, reasoning, tool calls, intermediate artifacts, action, feedback, and memory update should be recoverable as a single chain (Cai et al., 2025b; Nechepurenko and Shuvalov, 2026; Ang et al., 2025).

8.7 Evaluation Protocols for Trustworthy Deployment

Evaluation should follow the layered structure of reliability. At the model layer, benchmarks should report point error, probabilistic calibration, coverage, and performance under missingness, shift, and long horizons (Aksu et al., 2024; Ahamed et al., 2026b; Fan et al., 2024). At the reasoning layer, they should evaluate numerical correctness, temporal ordering, causal validity, evidence citation, and contradiction rates (Liu et al., 2025d; Zhou et al., 2026; Ye et al., 2025). At the tool layer, they should measure routing accuracy, execution success, argument validity, recovery from failed calls, and agreement between generated summaries and tool outputs (Wu et al., 2026b; Tao et al., 2026b; Kolonin et al., 2026). At the decision layer, they should report utility together with risk, including drawdown, constraint violation rate, worst-case loss, abstention quality, and human override frequency (Chen et al., 2025b; Xiao et al., 2025b; Nechepurenko and Shuvalov, 2026). A transaction-level reporting standard should also include the data timestamp, split policy, leakage controls, retrieval policy, tool versions, prompt templates, random seeds, execution logs, and whether the agent had access to future or contemporaneous external information (Cai et al., 2025b; Mostafa et al., 2026; Ang et al., 2025).

Recent benchmarks point in this direction. GIFT-Eval standardizes forecasting evaluation (Aksu et al., 2024). TimeSeriesGym evaluates ML engineering agents over multiple artifact types (Cai et al., 2025b). TemporalBench diagnoses contextual and event-informed reasoning (Weng et al., 2026). Foresight Arena evaluates forecasting agents in prediction markets using proper scoring rules such as Brier Score and Alpha Score, which separates predictive edge from market following (Nechepurenko and Shuvalov, 2026). TimeSeek evaluates agentic forecasters at multiple checkpoints in a prediction market lifecycle and shows that retrieval and model accuracy vary with market maturity and consensus level (Mostafa et al., 2026). ForecastBench is not a time series agent benchmark in the narrow sense, but it is useful as an adjacent evaluation design. Its dynamic, contamination-resistant future-event questions illustrate why static held-out datasets are insufficient for forecasting agents that can retrieve current information and act over time (Karger et al., 2025). Domain-specific audits support the same point. Structured financial workflows and agentic trading surveys find that execution semantics, transaction-cost reporting, and closed-loop reproducibility remain difficult to compare across studies (Ang et al., 2025; Xia et al., 2026). These works indicate that evaluation should combine static datasets, interactive workflows, event-conditioned tasks, continuously refreshed questions, and incentive-aware environments. The remaining gap is integration. Current benchmarks usually stress one layer of the stack, whereas deployed agents fail through interactions among layers. A mature protocol should report not only whether the final answer was correct, but also where a failure entered, whether it propagated, whether the agent detected it, and whether the system recovered or escalated. Without such auditability, an agentic system may appear reliable in a benchmark table while remaining hard to debug, reproduce, or certify in deployment.

9 Benchmarks and Evaluation Protocols

Benchmarks and evaluation protocols are the basis on which progress in the time series community is measured and compared, as they determine which capabilities are recognized, which failure modes are reported, and which are left implicit. Consequently, benchmarks are framed as an orthogonal dimension that spans foundation models, LLM4TS, and Time Series Agents in Fig. 1, highlighting their role as a universal evaluative pillar instead of a subordinate component. As the paradigm shifts, the targets and tasks that an adequate evaluation needs to cover are not fixed: as the field has progressed from foundation models to LLM4TS methods and time series agents, the object being evaluated has shifted from a numerical predictor over raw sequences, to a model that aligns temporal signals with language, and further to a system that plans, invokes tools, and acts under feedback. Each transition broadens the surface that evaluation must cover and renders protocols designed for the previous stage incomplete. Motivated by this, we propose a benchmark taxonomy structured

Table 11 Summary of Time Series Benchmark Taxonomy. As the paradigm evolves, the evaluation object shifts from numerical predictors to text-aligned reasoners, and ultimately to autonomous agents, fundamentally changing the input and output spaces as well as the core evaluation metrics.

Category	Core Evaluation Target	Input Space	Output Space	Key Metrics	Representative Works
Series-Only Foundation (Sec. 9.1)	Evaluate generalization capabilities of time series foundation models.	Univariate and multivariate time series	Numerical sequences or discrete labels	MSE, MAE, F1-score	GIFT-Eval (Aksu et al., 2024), fev-bench (Shchur et al., 2025), It’s TIME (Qiao et al., 2026)
Heterogeneous Augmented (Sec. 9.2)	Evaluate whether heterogeneous signals enhance standard time series tasks.	Time series paired with textual modalities (e.g., news, events) or/and images	Numerical sequences or discrete labels	MSE, MAE, F1-score	Time-MMD (Liu et al., 2024b), CiK (Williams et al., 2024), FNSPID (Dong et al., 2024), TTC (Kim et al., 2024)
Understanding & Reasoning (Sec. 9.3)	Assess the cognitive capacity of models to comprehend, explain, and perform reasoning over temporal dynamics via static question answering.	Time series aligned with textual prompts and queries	Natural language (e.g., choice, open-ended answers, chains of thought)	Answer Score, Text Generation Metrics (e.g., BLEU, ROUGE)	Time-MQA (Kong et al., 2025a), TSAQA (Jing et al., 2026), TSRBench (Yu et al., 2026a), MMTS-Bench (Yin et al., 2026), TimeSage-MT (Kong et al., 2026)
System-Level Agentic Tasks (Sec. 9.4)	Evaluate the autonomous end-to-end behavioral trajectories of agents in machine learning engineering, evidence discovery, open-world forecasting.	High-level user queries coupled with open environments and tool interfaces	Behavioral trajectories, executable code, and synthesized empirical evidence	Result Score, Completion Rate, Process Score, Forecasting Score	TimeSeriesGym (Cai et al., 2025b), Dr-CiK (Tang et al., 2026), AutoCast (Zou et al., 2022), FutureX (Zeng et al., 2025)

around different paradigms and tasks. Table 11 summarizes all categories, features and representative works, with detailed discussions elaborated in Sec. 9.1, Sec. 9.2, Sec. 9.3, and Sec. 9.4. Finally, in Sec. 9.5, we discuss the relationship between existing benchmarks and the constituent layers of Time Series Agents, highlighting the evaluation scopes, boundaries, and limitations of current benchmarks.

9.1 Series-Only Foundation Benchmarks

The evaluation of traditional time series tasks has long been dominated by static datasets within TSLib (Wang et al., 2026a), such as ETT, Electricity, and Weather for forecasting and imputation, SMD and MSL for anomaly detection, and UEA for classification. In this classical paradigm, small-parameter models are typically optimized on training and validation sets and evaluated on distinct test sets. With the emergence of Time Series Foundation Models (TSFMs) pre-trained on billions of time series tokens, more concerns including pre-training contamination, data freshness, data scale, and domain diversity have become more pressing issues. Consequently, the collective mission of benchmarks in this category is to recalibrate the baseline performance bound of series-only forecasting in the TSFM era, thereby providing a clean lower bound for subsequent works that integrate textual, visual, or agentic capabilities.

Specifically, GIFT-Eval (Aksu et al., 2024) establishes a standardized zero-shot evaluation protocol by unifying 23 datasets across 7 domains and 10 sampling frequencies, accompanied by a strictly contamination-free corpus to empirically verify generalization ability of TSFMs. Expanding upon this evaluative scale, fev-bench (Shchur et al., 2025) incorporates 100 forecasting tasks with 46 covariate variants and integrates bootstrap confidence intervals into its leaderboard, addressing the lack of statistical significance testing in the current literature. Recognizing that legacy benchmarks are prone to contamination in vast pre-training data, It’s TIME (Qiao

et al., 2026) introduces 50 dynamically updated datasets across 98 tasks and pioneers a pattern-level evaluation perspective to enable continuous benchmark evolution. Finally, shifting the focus from system-level rankings to internal capabilities, Investigating Compositional Reasoning in TSFMs (Potosnak et al., 2025) leverages spectral decomposition and the Top- k Basis Win metric to reverse-probe the models, revealing fundamental structural discrepancies in compositional generalization between patch-based Transformers and residual MLPs.

Collectively, these benchmarks delineate a definitive reality: series-only evaluation has been pushed to its conceptual and functional upper bound. Further scaling data volumes or refining contamination-proofing techniques can only quantify numerical fidelity; they fundamentally fail to reveal whether a model comprehends the real-world underlying events. This intrinsic limitation has directly catalyzed the next paradigm shift: introducing text as an input-side covariate to bridge numerical dynamics with semantic world knowledge.

9.2 Heterogeneous-Augmented End-to-End Benchmarks

Once series-only benchmarks reach their operational upper bounds, researchers begin to anchor heterogeneous external signals onto numerical sequences, such as news, events, reports, and variable descriptions. However, the output space of these benchmarks remains strictly numerical or confined to discrete labels. In this configuration, textual information serves purely as covariate or conditioning variable, keeping the underlying task essentially within the standard input-to-label realm (forecasting, anomaly detection, classification, and imputation). Benchmarks in this category fundamentally test whether models can achieve superior accuracy when assisted by heterogeneous information. Consequently, they constitute the entire literature for the first stage of multi-modality within the agentic time series forecasting landscape. We organize these works by progressing from **general-domain** scenarios to **domain-specific** ones, and finally to **boundary cases** situated between end-to-end tasks and question answering.

General-Domain In the field of general-domain multimodal time series analysis, benchmarks exhibit a clear trajectory toward finer alignment granularity, stricter contextual necessity, and higher data fidelity. Time-MMD (Liu et al., 2024b) establishes a fine-grained multimodal time series dataset featuring strict fact-prediction separation and deadline control to eliminate data contamination. CiK (Williams et al., 2024) elevates textual necessity to a hard constraint by manually designing tasks where the natural language context is absolutely indispensable for accurate forecasting, while NewsForecast (Wang et al., 2024b) introduces an LLM Agent iterative filtering mechanism to align news events with sequence fluctuations across multiple domains instead of relying on simple keyword matching. Addressing inherent modal irregularities, TimeMM (Chang et al., 2026) explicitly annotates components like triggers and constraints to treat alignment difficulty itself as a primary evaluation dimension. Fidel-TS (Xu et al., 2025e) establishes a high-fidelity benchmarking principle to make forecasting closer to real-world application scenarios. MoTime (Zhou et al., 2025c) extends the auxiliary modalities to text, metadata, and images while uniquely supporting zero-shot cold-start scenarios.

Domain-Specific Parallel to these general works, domain-specific benchmarks have flourished around specialized real-world data structures, particularly in finance, retail, and IT observability. In financial analysis, FinMultiTime (Xu et al., 2025d) offers a cross-market, quad-modal bilingual dataset, while FNSPID (Dong et al., 2024) provides massive-scale time-aligned news and price entries. These build upon classic baselines like StockNet (Xu and Cohen, 2018), which leverages deep generative models with temporal auxiliary objectives, and Hybrid Deep Sequential (Wu et al., 2018), which optimizes cross-modal trend classification. Recent works like DOW 30 (Chen et al., 2023) use LLMs to infer dynamic stock networks from headlines rather than relying on static knowledge graphs, and FinTexts (Lee et al., 2026) replaces keyword matching with semantic embedding retrieval and LLM-driven document parsing. Outside finance, VISUELLE (Skenderi et al., 2024) provides a rare multimodal benchmark for retail sales forecasting by integrating product images and Google Trends, whereas LEMMA-RCA (Zheng et al., 2024) addresses IT/IoT observability with a large-scale multimodal dataset for Root Cause Analysis that pairs metrics with system logs.

Boundary Cases Finally, a set of boundary benchmarks has emerged where the output space begins to carry explanatory elements. TIME-RA (Yang et al., 2025f) shifts anomaly detection from standard classification to a generative inference paradigm, providing fine-grained categorization and textual root-cause explanations.

TTC (Kim et al., 2024) expands the evaluation scope from numerical forecasting to time-aligned text and time series co-prediction, supporting the evaluation of textual generation alongside numerical output.

In summary, while benchmarks in this category significantly open up the input space of time series models, their output spaces remain confined to numerical sequence or discrete labels, meaning the core evaluation protocols are still fundamentally governed by metrics such as MAE, MSE, and F1 scores. Once a task demands that a model articulate why a certain analysis was made or decipher what underlying real-world events transpired within the sequence, the output modality must inevitably transition to natural language. This boundary naturally leads into the next paradigm: Time Series Understanding and Reasoning benchmarks powered by Language Model backbones.

9.3 From Label to Text: Time Series Understanding and Reasoning

When the output space shifts from numerical to natural language, including binary questions, multiple-choice questions, open-ended questions, captions, and chains-of-thought, the time series paradigm transitions from the object of prediction to the object of interrogation. Concurrently, the model backbone needs to be an LLM or MLLM, relegating the time series encoder to an auxiliary role. This category represents the most comprehensive tier within agentic time series evaluation. It tests whether the model can comprehend, explain, compare, and reason about time series data. To systematically chart this landscape, we organize existing works along two primary paradigm lines: **outcome-oriented** benchmarks that focus on flat task aggregation and unified question-answering results, and **capability-deconstruction** benchmarks that explicitly dissect models’ fine-grained cognitive layers, operational processes, and feature-level comprehension.

Outcome-Oriented Evaluation In the field of TSQA, early benchmarks primarily focus on outcome-driven evaluation by unifying fragmented time series tasks into a consolidated question-answering interface. In the general domain, Time-MQA (Kong et al., 2025a) pioneers this approach by integrating five classic time series tasks across 12 domains into a single QA format. MTBench (Chen et al., 2025a) pushes this boundary into cross-modal reasoning by fusing text and numerical dynamics, while TSAQA (Jing et al., 2026) marks the peak scale of this sub-category, unifying standard and advanced analytical tasks into standardized true/false, multiple-choice, and puzzle-solving formulations. In specialized fields, EngineMT-QA (Lei et al., 2025) maps multi-task aerospace sensor data directly to text-based answers using the ITFormer framework.

Capability-Deconstruction Evaluation Although pioneering benchmarks successfully consolidate disparate tasks under unified QA interfaces, they remain predominantly outcome-oriented. Recognizing that process evaluation is equally critical in agentic time series tasks, subsequent frameworks pivot toward decoupling and dissecting model capabilities into granular dimensions and reasoning processes. TSRBench (Yu et al., 2026a) and MMTS-Bench (Yin et al., 2026) isolate perception, reasoning, and alignment into hierarchical categories, while TemporalBench (Weng et al., 2026) uses progressive contextual layers. In specialized domains, ARFBench (Xie et al., 2026) stratifies software observability into a 3-tier, 8-class question hierarchy, and HEARTS (Li et al., 2026c) structures clinical reasoning from basic perception to complex medical deduction. Complementing these taxonomies, diagnostic frameworks probe behavioral and procedural limits. TimeSeriesExam (Cai et al., 2024) analyzes the sensitivities to properties like seasonality or structural breaks via template-driven data. Language Models Still Struggle to Zero-shot Reason about Time Series (Merrill et al., 2024) empirically questions whether context genuinely aids numerical understanding, while TFRBench (Ahamed et al., 2026b) monitors the reasoning process via a multi-agent verification loop. Finally, targeted anomaly studies (Zhou and Yu, 2025) discover that visual modalities often surpass text inputs, whereas CaTS-Bench (Zhou et al., 2025a) exposes systematic VLM deficiencies in numerical alignment and context-aware description.

In summary, Time Series Understanding and Reasoning benchmarks elevate models to a level of interactive comprehension; however, they fundamentally remain confined to a passive exam-taking paradigm. In this setting, questions are pre-formulated, contexts are static, and models are only required to output answers within a single turn. To transition into the agentic domain, models must progress from passive responders to proactive executors capable of autonomously searching open corpora for empirical evidence, writing code, and deploying analytical pipelines. This imperative directly steers us toward the final category of benchmarks.

9.4 Beyond Static Exam: Working on System-Level Agentic Tasks

Unlike the preceding categories that evaluate static outputs (e.g., numerical forecasts, class labels, or textual answers), this final tier focuses on the behavioral trajectories of the entire system. The autonomous agent is tasked with either generating code to execute a complete machine learning pipeline or searching and filtering open-ended corpora to synthesize empirical evidence. This paradigm represents the highest level of agentic autonomy in time series research, shifting evaluation toward mechanisms such as tool utilization, ReAct frameworks, AutoML, and multi-agent coordination. We organize this literature into three streams: **machine learning engineering agents** for time series, **autonomous evidence discovery**, and open-world **event forecasting**.

ML Engineering In this tier, evaluation centers on the system’s capacity to execute end-to-end data science workflows. TimeSeriesGym (Cai et al., 2025b) establishes an agent-agnostic platform that evaluates skills from data processing to modeling via a multi-artifact protocol that inspects code, models, and submissions. Its companion diagnostic study, LLM Agents Struggle at Engineering TS Solutions (Cai et al., 2025a), systematically benchmarks state-of-the-art systems like AIDE and OpenHands on this platform. The findings reveal that current agents consistently lack multi-step contextual reasoning, and highlight how framework designs drive long-trajectory failures, such as AIDE’s sequential chains diluting attention and OpenHands’ CodeAct employing short-sighted planning.

Evidence Discovery Moving beyond ML engineering pipelines, a parallel line of benchmarks evaluates an agent’s capability in autonomous evidence discovery and environment self-generation. Specifically, Dr-CiK (Tang et al., 2026) shifts multimodal forecasting from passive context reception to active acquisition by testing whether agents can autonomously retrieve predictive evidence from archival corpora while filtering out complex distractors. Conversely, TimeSeriesExamAgent (Gwiazda et al., 2026) automates the generation of evaluation itself, utilizing an LLM agent framework to synthesize reconfigurable time series QA benchmarks. Meanwhile, TSAIA (Ye et al., 2025) broadens the operational scope by evaluating eight LLMs under the CodeAct framework across 33 real-world time series task types, focusing strictly on multi-step reasoning and constraint satisfaction to expose critical architectural limitations.

Event Forecasting While the engineering and evidence-discovery frameworks significantly expand the operational space of agents, they still treat the time series itself as the primary query target. This boundary has catalyzed a recent frontier that upends this assumption by positioning open-world events as the core query, reducing time series data to auxiliary evidence. In this space, AutoCast (Zou et al., 2022), AutoCast++ (Yan et al., 2024) and ForecastQA (Jin et al., 2021) pioneer event prediction based on news by imposing strict temporal constraints to faithfully simulate future-looking environments. PROPHET (Yang et al., 2025c) evaluates the model’s ability to predict event outcomes by providing it with fixed information retrieved by LLM for each query. ForecastBench (Karger et al., 2025), FutureX (Zeng et al., 2025) and FutureX-Pro (Liu et al., 2026a) use agents to dynamically generate weekly queries on unresolved events, while gradually accumulating results from resolved historical queries for model evaluation. In specific fields, Foresight Arena (Nechepurenko and Shuvalov, 2026) establishes a blockchain-backed benchmark for forecasting agents.

In conclusion, these advanced benchmarks successfully shift the evaluation frontier from static prediction outputs to end-to-end behavioral trajectories and real-world event dynamics. By testing multi-step tool orchestration, autonomous evidence synthesis, and real-world event inference, they redefine the core standards of agentic time series evaluation. Ultimately, these frameworks provide insight into current systems, paving the way for the development of more strategically aligned agentic architectures.

9.5 Evaluation Boundaries: Mapping Benchmarks to the Layers

Coverage on Perception and Reasoning Layers. Existing benchmarks are primarily concentrated on evaluating the foundational layers of the agentic stack, specifically *Time Series Perception* (Layer A) and *Time Series Reasoning* (Layer B). Early series-only and heterogeneous-augmented datasets (Sec. 9.1 and Sec. 9.2) focus on testing raw numerical perception, feature alignment, and pattern recognition. More recent question-answering and capability-deconstruction benchmarks, such as TSRBench (Yu et al., 2026a), Time-MQA (Kong et al., 2025a), ODTQA-FoRe (Wang et al., 2026b), and TemporalBench (Weng et al., 2026) (Sec. 9.3), successfully

probe cognitive reasoning capabilities by assessing whether models can identify structural transitions, verify statistical claims, and formulate logical explanations. However, since these evaluations typically operate in a static, single-turn question-answering format, they treat perception and reasoning as passive, isolated outputs rather than state-dependent, adaptive operations within a closed decision-making loop.

The Frontier of Planning and Action Evaluation. The transition to evaluating the *Planning and Action* layer (Layer C) represents the current boundary of active benchmark development. Frameworks such as TimeSeriesGym (Cai et al., 2025b), Dr-CiK (Tang et al., 2026), and TSAIA (Ye et al., 2025) (Sec. 9.4) have initiated a shift from passive evaluation to assessing how agents autonomously decompose complex temporal goals, generate executable code, and route specialized tools under constraint satisfaction. While these benchmarks provide valuable insights into tactical execution within computational sandboxes, they are still bounded by simplified, immediate environment feedback. In realistic temporal environments, actions carry long-horizon consequences and delayed feedback. Current protocols primarily evaluate the correctness of an immediate tool call or a single coding submission, leaving the agent’s capacity for multi-step credit assignment and rolling plan revision under delayed outcomes largely untested.

Gaps in Memory and World Model Assessment. The most significant evaluation gaps lie in the *Memory and Knowledge* (Layer D) and *Temporal World Models* (Layer E) layers, both of which remain largely unaddressed by the current benchmarking landscape. Despite the architectural emphasis on structured memory, including efforts to manage forgetting rates under non-stationarity or to update temporal regimes, there are currently no standardized protocols for evaluating memory retrieval accuracy, context retention across sessions, or strategy adaptation over extended temporal horizons. Similarly, evaluating temporal world models is constrained by a lack of interactive simulation environments. Current benchmarks cannot systematically verify whether an agent’s internal simulation of a temporal process is causally valid, physically consistent, and capable of supporting counterfactual reasoning. Developing interactive, multi-session evaluation infrastructures that target memory dynamics and world-model simulation remains a critical open frontier for the community.

10 Applications of Agentic Time Series Systems Across Domains

Building on the preceding discussion of agentic architecture and reliability, this section examines how time series agents are instantiated across concrete application domains. The five-layer view developed earlier (i.e., perception, reasoning, planning and action, memory and knowledge, and temporal world models) describes the general capabilities of an agentic system, but applications determine which layers become operationally decisive and which failures are most consequential. From this perspective, applications are better understood not as an industry catalogue, but as deployment contexts that stress different parts of the agent stack. Some settings require open-ended workflow construction and transparent reporting; others foreground memory, risk control, and human oversight; still others require executable environments for testing interventions before real-world deployment. This application-centered view also connects to broader reliability concerns: forecasting quality, grounding, tool-use reliability, decision safety, auditability, and reproducibility carry different weights across domains and motivate different architectural responses.

In the following subsections, we review recent agentic time series systems across seven representative application areas. We begin with general-purpose time series analysis, which illustrates the low-prior setting where agents must construct reliable analytical workflows from heterogeneous data and underspecified user intent (Sec. 10.1). We then discuss finance and trading, where forecasts must be connected to delayed, costly, and often irreversible decisions (Sec. 10.2), followed by traffic, urban systems, and transportation, where time series prediction is integrated with spatial reasoning, simulation, and control (Sec. 10.3). The next three domains: weather, energy, and buildings; healthcare; and industrial IoT, observability, and root cause analysis - highlight multi-scale physical environments, conservative clinical decision support, and streaming incident diagnosis, respectively (Secs. 10.4 to 10.6). Finally, agriculture, autonomous systems, retail, and supply chains demonstrate how forecasts become inputs to simulation, verification, allocation, and operational control (Sec. 10.7). Together, these domains show that the practical significance of time series agents lies not only in improving predictive accuracy, but also in enabling trustworthy temporal workflows that can analyze, decide, adapt, and act under domain-specific constraints.

10.1 General-Purpose Time Series Analysis

General-purpose time series analysis (Yang et al., 2026a,b; Ding et al., 2023) represents the setting with the weakest prior assumptions. An agent may receive raw series and a natural-language request without knowing the task type, sampling frequency, data quality, forecast horizon, or relevant temporal scale in advance. This setting directly reflects the distinction between time series agents and ordinary NLP agents discussed earlier: the input is not a static document to be summarized, but a temporal environment whose structure must be diagnosed before it can be modeled. Two temporal properties are especially important. First, multi-scale dependency is unavoidable, since a single request may involve short-term fluctuations, seasonal structure, long-term trends, and rare events simultaneously. Second, uncertainty accumulates throughout the workflow, because early errors in diagnosing frequency, missingness, seasonality, or regime structure can propagate into model selection, validation, and reporting. The application-level challenge is therefore to make open-ended time series analysis reproducible and inspectable, rather than to optimize a single forecaster in isolation.

Existing general-purpose agents can be interpreted as attempts to turn time series analysis into an explicit service workflow. TimeSeriesScientist addresses the practical forecasting scenario in which users face many short, noisy, heterogeneous series and want an end-to-end forecast with minimal manual preprocessing. Its Curator, Planner, Forecaster, and Reporter roles are not merely architectural components; at the application level, they correspond to the work normally performed by a data analyst: diagnosing data quality, narrowing model choices, validating candidate forecasts, and producing a transparent report (Zhao et al., 2025). TimeCopilot serves a similar application need from the perspective of model-service orchestration. Rather than asking users to choose among statistical models, neural models, and time series foundation models, it coordinates forecasting engines and cross-validation to select a context-appropriate forecasting strategy (Garza and Rosillo, 2025). These systems are most relevant when the end user needs a dependable forecasting workflow, not simply a prediction vector.

A second group of systems focuses on open-ended time series question answering and analysis. TimeART targets scenarios in which a user asks analytical questions about time series and the agent must decide which computational tools to invoke. Its Query-Thought-Action-Observation loop makes the application interaction closer to an automated data scientist: the agent decomposes the question, calls time series operators, observes their outputs, and then synthesizes an answer grounded in executed analysis (Wu et al., 2026b). TS-Agent similarly keeps the language model in an evidence-gathering and synthesis role, while statistical operators extract trends, anomalies, correlations, and other structural information from raw numerical sequences (Liu et al., 2025d). TS-Reasoner extends this idea to domain-oriented inference by composing numerical, retrieval, model-based, and custom-generation tools for automated analysis (Ye et al., 2026). At the application level, these systems are useful not because they introduce new forecasting architectures, but because they lower the barrier for users to conduct reliable exploratory analysis over unfamiliar time series.

A third group expands the general-purpose setting from forecasting to workflow construction, code generation, and benchmarking. TimeSeriesGym evaluates whether agents can operate as time series machine-learning engineers: editing code, managing artifacts, tuning pipelines, and submitting executable solutions rather than only producing forecasts (Cai et al., 2025b). AION similarly frames time series tasks as practical agent harnesses involving workspaces, validation interfaces, skills, memory, and protocols (Zhan et al., 2026). These benchmarks reveal an important application-level shift. For general-purpose time series agents, the output is often a traceable workflow, including data diagnostics, code, model choices, validation results, and reports, rather than a single numerical answer.

General-purpose analysis most strongly stresses forecasting quality, reproducibility, and auditability. Because these agents claim broad applicability, autonomy must not hide weak predictive performance; benchmarks such as GIFT-Eval show that zero-shot forecasting accuracy and calibration still vary substantially across datasets, frequencies, and horizons (Aksu et al., 2024). At the same time, an agentic workflow is trustworthy only if its intermediate artifacts can be inspected and rerun. The central gap is therefore the lack of evaluation protocols that certify the full pipeline - from data curation and diagnostic reasoning to model execution and final reporting - as a single auditable object, rather than scoring only the final forecasting output.

10.2 Finance and Trading

Finance is one of the most developed application areas for agentic time series systems because it exposes many of the hardest properties of temporal agency simultaneously. A financial agent operates under delayed feedback, since the quality of a trading or investment decision becomes observable only after the market evolves. It also acts under irreversible and high-cost conditions, because trades incur losses, transaction costs, slippage, and market impact. Moreover, financial environments are highly non-stationary: regime changes, liquidity shocks, macroeconomic events, and black-swan disruptions can invalidate historical regularities with little warning. The application-level objective is therefore not merely to forecast returns or prices. A financial time series agent must decide whether evidence is credible, whether an action is justified, how much risk to take, and how to revise its strategy after gains, losses, or regime changes. Accordingly, financial applications provide a useful lens for observing how agentic time series systems evolve from quantitative research automation, to role-based decision simulation, and further to code-executing and reproducible trading workflows.

Existing financial agents reflect this decision-centric nature. **A first line** of work treats financial agents as assistants for quantitative research and alpha discovery, where the agent searches for informative factors, formulates hypotheses, and evaluates candidate signals under market feedback. R&D-Agent-Quant (Li et al., 2026d) automates quantitative strategy development through a research–development–feedback loop, combining factor mining, model innovation, code generation, and real-market backtesting. Hubble (Shi et al., 2026a) further illustrates a code- and execution-centric direction: instead of allowing unconstrained LLM-generated factors, it restricts generation to a domain-specific operator language, executes candidates in a sandboxed pipeline, and feeds structured diagnostics back into later search rounds. The application-level insight is that financial agents often begin not as autonomous traders, but as research agents for discovering, testing, and refining time series signals under strict reproducibility constraints. **A second line** of work moves from factor discovery to role-based simulation of financial decision processes. TradingAgents (Xiao et al., 2024) models stock trading as a collaborative decision process inspired by the organization of trading firms. Fundamental, sentiment, technical, bull/bear, trader, and risk-management roles are used to combine heterogeneous evidence and structured disagreement before a trading action is made. At the application level, the key contribution is not multi-agent communication itself, but the attempt to approximate the workflow of institutional decision-making, where different analysts examine different evidence sources and risk managers constrain final actions. FinCon (Yu et al., 2024) similarly targets financial decision-making through a manager-analyst hierarchy and conceptual verbal reinforcement, enabling lessons from previous investment episodes to influence later decisions. FinMem (Yu et al., 2025b) enhances trading-specific memory by separating short-term market signals, mid-term news, and long-term company knowledge, aligning memory granularity with financial evidence.

Beyond alpha discovery and role-based trading, other systems emphasize personalization, human oversight, reinforcement learning, and financial question answering. FinArena (Xu et al., 2025c) frames financial forecasting as human-agent collaboration: the system integrates stock prices, news, financial statements, and user risk preferences so that investment decisions can be adapted to different investor profiles. This makes it particularly relevant to decision-support settings where the agent should not replace the analyst, but should organize evidence and expose controllable risk preferences. FinHEAR (Chen et al., 2025b) incorporates human expertise, expert-guided retrieval, confidence-adjusted position sizing, and outcome-based refinement for adaptive risk-aware temporal reasoning. Trading-R1 (Xiao et al., 2025b) explores reinforcement-learning-based reasoning for financial trading, moving from passive prediction toward structured investment theses, volatility-adjusted decisions, and policy optimization under trading feedback. TSAG (Kolonin et al., 2026) evaluates whether financial agents can delegate quantitative questions to verifiable external tools, select tools accurately, and avoid hallucinated answers in financial time series analysis.

Finance most strongly activates decision safety, provenance, and reproducibility. A forecast that appears useful can become unsafe once leverage, excessive turnover, transaction costs, or ignored uncertainty are introduced. This concern is especially acute for code-executing agents and alpha-mining systems, where small changes in data splits, execution timing, transaction-cost assumptions, or factor evaluation protocols can materially change reported performance. Therefore, financial agents require confidence-aware sizing, explicit risk constraints, and comparable backtesting protocols rather than unconstrained action generation (Bao et al., 2026). Security and provenance are also central, because financial agents often ingest untrusted news, filings, social signals, and market narratives that may distort decisions. Audit-oriented analyses of

agentic trading (Xia et al., 2026) further caution that many reported gains are difficult to compare without standardized execution semantics, time-consistent splits, and transaction-cost assumptions.

The central open problem is reproducibility and credit assignment under delayed profit-and-loss, across factor discovery, role-based trading, and code-executing research workflows. Without explicit market simulators, transaction-cost models, temporal data splits, execution semantics, and reporting standards, it remains difficult to determine whether an agent’s success comes from genuine temporal reasoning, useful factor discovery, effective risk control, favorable evaluation design, leakage, or unmodeled trading assumptions.

10.3 Traffic, Urban Systems, and Transportation

Traffic, urban systems, and transportation provide a natural setting for agentic time series systems because the output is often not merely a numerical forecast, but a simulation result, planning recommendation, or control action. This application domain therefore brings the planning, action, and world-model layers to the foreground. A traffic-control policy can change the state of a real transportation network, and its effects may only become visible after congestion propagates through a spatial graph. Urban sensing tasks also require the agent to combine temporal signals with spatial structure, maps, mobility patterns, and operational constraints. The application-level challenge is therefore to bridge time series forecasting with spatial reasoning, simulator configuration, and closed-loop intervention.

Open-TI illustrates this bridge by using an augmented language model to interact with traffic-analysis tools and external traffic packages. Rather than treating traffic forecasting as an isolated prediction task, Open-TI supports traffic intelligence workflows in which the system can understand user requests, call transportation tools, and help configure analysis processes (Da et al., 2024). At the application level, this matters because transportation analysts often need to move from high-level scenario descriptions to concrete simulation, diagnosis, or signal-control experiments. CityGPT addresses a related but broader urban-intelligence problem by enhancing LLMs with urban spatial knowledge and evaluating their ability to solve urban tasks, making it relevant for applications where time series from urban IoT sensors must be interpreted together with spatial context and city-scale structure (Feng et al., 2025). Thus, CityGPT is less about improving a single time series model and more about enabling LLMs to reason over complex urban environments in which temporal signals are embedded within spatial, social, and infrastructural contexts.

DCATS approaches traffic forecasting from a data-centric perspective. Instead of changing the forecasting architecture, it uses an LLM agent to select and refine auxiliary traffic series based on metadata and validation feedback on the LargeST benchmark (Yeh et al., 2025). Its application significance is that, in large urban sensor networks, the choice of auxiliary series can be as important as the forecasting model itself: a poorly selected neighboring sensor may introduce noise, leakage, or regime mismatch, whereas a well-selected auxiliary set can improve forecasting across horizons. More generally, traffic and transportation agents reveal that data selection, spatial context construction, and simulator interaction are first-class actions in time series applications, rather than peripheral preprocessing steps outside the agentic workflow.

Traffic and urban systems most strongly stress tool-use reliability, decision safety, and closed-loop evaluation. Because the deliverable may be a simulator configuration or a control recommendation, the agent must route to the correct tool, pass valid arguments, interpret outputs faithfully, and avoid unsafe interventions. Evaluation therefore cannot stop at pointwise forecasting error. It must also consider control-oriented and system-level metrics such as average travel time, throughput, queue length, delay, and robustness under demand shifts. These metrics assess whether the agent’s action improves the transportation system, not merely whether its forecast is numerically accurate.

What remains missing is a decision-centric, closed-loop benchmark that connects forecasting, spatial reasoning, simulation, and control under realistic feedback delays, while evaluating whether time series agents can move beyond prediction toward safe and verifiable temporal decision-making.

10.4 Weather, Energy, and Buildings

Weather, energy, and building systems expose time series agents to multi-scale physical environments. Their defining temporal pressure is multi-scale dependency, since operational evidence may span hourly fluctuations,

six-hour synoptic changes, daily patterns, seasonal cycles, and rare extreme events (Sukhorukov et al., 2026). Non-stationarity is also central, because weather regimes, renewable generation, building occupancy, and electricity demand can shift under changing climate, market, or usage conditions. Uncertainty accumulates across the workflow: an early error in interpreting a weather front, solar ramp, occupancy change, or load spike can propagate into reporting, scheduling, control, or market decisions. Unlike general-purpose analysis, the output in this domain is often not only a forecast, but a report, warning, optimization plan, or control recommendation that must remain consistent with physical constraints.

Existing systems show two architectural responses. The first response turns numerical forecasts into grounded, decision-facing explanations. Hierarchical AI-Meteorologist uses an Assistant-Meteorologist-Writer structure to transform hourly, six-hour, and daily weather forecast tables into evidence blocks, keywords, warnings, and explanatory reports, addressing the need for multi-scale reasoning and expert-readable communication (Sukhorukov et al., 2026). AGCD uses agent-guided cross-modal decoding to improve weather prediction, showing how perception and generation can be coupled when temporal evidence must be translated across modalities (Wu et al., 2026a). Multi-agent weather captioning systems further use agent consensus to improve forecast narratives without full model retraining, which is useful when the main application need is faithful reporting rather than model replacement (Wu et al., 2026a). The second response connects forecasting to operation. AutoLFM uses multi-agent retrieval, tool calling, and AutoML to automate building load forecasting, so that the agent not only selects models but also generates and validates forecasting code under building-specific data constraints (Li et al., 2026b). SEA-TS extends this engineering pattern through self-evolving code generation for solar and load forecasting, while CastFlow and AlphaCast use agentic reasoning or learned workflows for forecasting benchmarks where procedural memory and interactive refinement matter (Xu et al., 2026e; Pan et al., 2026; Zhang et al., 2026a). Stock trading agents go one step further by asking LLMs to generate optimization models or expert actions for prosumers before MARL agents learn to operate under grid constraints, moving the architecture from forecasting and explanation toward forecasting, optimization, action, and physical feasibility (Zhang et al., 2025b).

The reliability signature of this domain is split between forecasting quality, grounding, tool-use reliability, and decision safety. A weather report must be semantically consistent with the forecast tables it summarizes, and an energy-control recommendation must respect operational constraints rather than only minimize a pointwise error. For building and energy agents, code generation reliability is part of the forecasting system itself, since an incorrect preprocessing step, invalid validation split, or unstable generated script can produce a plausible but unusable load forecast. The most important failure mode is therefore not a single inaccurate number, but an ungrounded report or unsafe operational action produced from a chain of otherwise fluent intermediate steps.

The open gap is a closed-loop evaluation protocol that jointly tests physical forecasting, evidence-grounded explanation, optimization, and control safety. Current systems demonstrate useful pieces of the stack, but they do not yet certify whether an agent can move from multi-scale temporal evidence to a physically valid and auditable operational decision.

10.5 Healthcare

Healthcare time series emphasize conservative, guideline-grounded decision support under high risk. The dominant temporal pressures are irreversible action, uncertainty accumulation, and irregular measurement. Clinical observations are sparse, noisy, and unevenly sampled; patient state changes over time; and the consequence of a wrong mortality, readmission, ECG, or deterioration assessment is not comparable to an ordinary forecasting error. This domain therefore should not be framed as a testbed for aggressive autonomy. Its application-level objective is to support clinicians with grounded evidence, calibrated uncertainty, and traceable reasoning while keeping the final decision aligned with medical expertise and institutional guidelines.

ColaCare is the clearest agentic example in this domain because it models EHR prediction as a multi-disciplinary consultation process. Multiple DoctorAgents reason from different clinical perspectives, expert models process structured EHR data, a MetaAgent aggregates the outputs, and a RAG module retrieves medical guideline knowledge to support ICU mortality and readmission prediction (Wang et al., 2025b). This design stresses the reasoning, memory-and-knowledge, and verification layers more than the action layer: the agent is valuable because it organizes evidence and exposes clinical rationales, not because it

independently intervenes on the patient. Translator and benchmark-style systems complement this direction without replacing it. GEM grounds ECG interpretation in both waveform time series and ECG images, enabling evidence-driven diagnosis and cardiologist-facing evaluation (Lan et al., 2026). OpenTSLM aligns multivariate medical time series and text with language models for QA and classification, while HEARTS evaluates health time series reasoning across diverse signal modalities and task types (Langer et al., 2025; Li et al., 2026c). Together these systems show that healthcare agents need medical grounding at three levels: physiological signals, clinical text, and external guideline knowledge.

The reliability signature of healthcare is dominated by grounding, human alignment, decision safety, and auditability. A clinically fluent explanation can still be unsafe if it cites the wrong signal segment, ignores uncertainty, or conflicts with guideline knowledge. Standard predictive metrics such as AUROC and AUPRC remain necessary for mortality or readmission prediction, but they are not sufficient for agentic evaluation because they do not measure whether the reasoning trace is faithful, whether retrieved knowledge is appropriate, or whether the system should abstain. Healthcare agents therefore require conservative verification, uncertainty-aware reporting, and clinician-reviewable evidence rather than unconstrained action generation.

The open gap is an evaluation setting that links clinical prediction, guideline grounding, uncertainty-aware abstention, and human review in the same loop. Existing systems point toward useful decision support, but they do not yet establish when a medical time series agent should answer, defer, or escalate.

10.6 Industrial IoT, Observability, and Root Cause Analysis

Industrial IoT, cloud observability, and root cause analysis turn anomaly detection (Yang et al., 2025b) into streaming incident diagnosis. These environments generate continuous metrics, logs, traces, alerts, and sensor streams whose distributions shift as workloads, devices, services, and operating conditions change. The main temporal pressures are non-stationarity, multi-scale dependency, and delayed feedback, because an incident may begin as a small leading indicator, propagate across components, and become interpretable only after downstream symptoms appear. For operators, the target output is rarely an anomaly score alone. They need to know whether an incident exists, when it started, how severe it is, which signals are related, and what root cause or remediation path is most plausible.

Agentic systems in this domain respond by making perception and reasoning more diagnostic. Argos uses LLM agents to autonomously generate rules for AIOps anomaly detection, turning expert rule construction into an auditable agent-assisted process (Gu et al., 2025). AnomaMind reformulates anomaly detection as a structured, tool-augmented workflow that localizes anomalous intervals in a coarse-to-fine manner, prepares features through multi-turn tool interaction, and refines decisions through self-reflection (Tao et al., 2026c). SAGE and related multi-agent anomaly-detection systems use tool-enhanced or collaborative reasoning to compare candidate explanations rather than relying on a single detector score (Shi et al., 2025b; Kang et al., 2026). MAS-LSTM addresses scalable anomaly detection for industrial IoT networks, showing that the perception layer must handle distributed and high-volume sensor streams before any higher-level diagnosis can be trusted (Qin et al., 2025). More conservative agent-assisted pipelines for IoT temperature series use rule-based agent selection to support statistical anomaly detection, which is useful when deployment demands interpretability and predictable behavior over open-ended autonomy. The benchmark side reinforces the same shift. ARFBench evaluates anomaly reasoning over real production incidents, including questions about existence, magnitude, start and end time, category, correlation, and leading-lagging indicators (Xie et al., 2026). LEMMA-RCA combines metrics and logs across IT and IoT domains for root cause analysis, making multimodal incident structure a first-class object of evaluation (Zheng et al., 2024).

The reliability signature of this domain is reasoning correctness, tool-use reliability, grounding, and auditability. A detector can be useful with imperfect recall, but an incident agent that invents a causal chain or misidentifies a leading indicator can waste operator time or trigger the wrong remediation. Tool calls must use valid windows, align logs and metrics correctly, and preserve temporal ordering, because root cause claims are often claims about timing. The central evaluation question is therefore whether the agent can produce a reproducible diagnostic trace from raw observability evidence, not whether it can merely label a subsequence as anomalous.

The open gap is a unified benchmark that connects streaming anomaly detection, multimodal evidence

alignment, root cause reasoning, and remediation-oriented reporting. Current datasets and systems make incident understanding measurable, but they do not yet evaluate the full closed loop from alert to verified diagnosis.

10.7 Agriculture, Autonomous Systems, Retail, and Supply Chain

Agriculture, autonomous systems, retail, and supply chains test whether forecasts can support executable decisions. These domains are heterogeneous, but they share a common application structure: prediction is only one component inside a larger decision loop. The dominant temporal pressures are multi-scale dependency, delayed feedback, and irreversible or costly action. A crop recommendation may unfold over a growing season, an autonomous-vehicle verification result may affect safety-critical deployment, and a retail demand decision may change stockouts, inventory turnover, and replenishment cost after the forecast horizon has passed. The agent therefore needs a temporal world model, not only a predictor.

The architectural response is clearest in world-model domains. AgriWorld defines a World-Tools-Protocol framework for agricultural reasoning, exposing tools for geospatial queries, remote-sensing time series analytics, crop growth simulation, and task-specific prediction, while an Agro-Reflective agent writes code, observes execution results, and refines its analysis through an execute-observe-refine loop (Zhang et al., 2026b). Its benchmark covers lookup, forecasting, anomaly detection, and counterfactual analysis, including out-of-distribution settings, making agriculture a useful testbed because the environment is complex but partly executable. Autonomous-system verification follows the same logic under higher safety pressure. AIVV integrates neuro-symbolic LLM agents into verification and validation for autonomous underwater vehicles, using agentic reasoning to support fault verification and classification rather than ordinary sequence prediction (Kwon et al., 2026). Operational domains show a second response. Multi-Agent Deep RL for demand forecasting coordinates store, distribution-center, and corporate agents so that demand prediction is evaluated together with stockout rate, inventory turnover, and inventory decisions (Yang et al., 2025g). VISUELLE adds product images, metadata, sales time series, and external trend signals for multimodal cold-start retail forecasting, showing how perception must integrate visual and temporal evidence when historical sales alone are insufficient (Skenderi et al., 2024). Across these examples, the value of agency is largest when the forecast becomes an input to simulation, verification, allocation, or control.

The reliability signature is decision safety, simulation validity, counterfactual consistency, and auditability. For AgriWorld-style systems, the key question is whether tool execution and simulator feedback actually ground the agent’s conclusions, especially for counterfactual claims. For autonomous systems, verification errors are safety errors, so the agent must provide traceable evidence rather than persuasive summaries. For retail and supply chain, forecast accuracy must be evaluated together with operational metrics, because a model that lowers error can still produce poor inventory decisions under biased demand, cold starts, or delayed replenishment feedback.

The open gap is an executable benchmark family that evaluates prediction, simulation, intervention, counterfactual consistency, and downstream operational outcomes together. These domains show why agentic time series systems matter, but they also expose how little current evaluation says about whether a forecast-driven action is safe, valid, and reproducible.

11 Positions and Frontiers

This survey argues that agentic time series systems are not merely a new interface to LLM4TS, but a shift in what the field treats as its central problem: **from learning a better mapping from history to forecast, to running a closed loop that perceives, reasons, acts, and adapts in a temporal environment**. The shift is forced by the problem rather than chosen for elegance. A single model is judged almost entirely by predictive accuracy, whereas a deployed system must cope with drifting distributions, delayed feedback, multimodal evidence, and actions that carry real operational risk; there, the hard part is no longer improving one number but sustaining a reliable analytic workflow (Cheng et al., 2026a; Dai et al., 2026a). Trustworthiness then has to be judged over the whole loop rather than the language it produces (Ning et al., 2025a), and once a system acts in a closed loop, the reproducibility and protocol failures that no static benchmark ever surfaces begin to

Table 12 The six frontiers of agentic time series systems. Each rides a shift the field is already undergoing, bookends one chapter of this survey, and leaves a concrete open question.

Frontier	Ch.	The shift it rides	The open question it leaves
Verifiable temporal reasoning	Sec. 4	From final-answer accuracy to step-level grounding.	Can every claim about trend, lag, or cause be bound to an executable check?
End-to-end agentic training	Sec. 5	From prompt-engineered pipelines to policies learned over trajectories.	Can perception, planning, memory, and action be optimized jointly?
Memory and lifelong adaptation	Sec. 6	From a fixed context window to layered, procedural, cross-session memory.	Which experience is worth keeping, and does it generalize or merely leak?
Temporal world models	Sec. 7	From point prediction to simulating and intervening on an environment.	What is the minimal world model that supports counterfactuals?
Decision-centric evaluation	Sec. 9	From forecast error to action quality under delayed feedback.	How to score prediction, uncertainty, and action jointly?
Standardized temporal tooling	Sec. 10	From ad hoc, per-paper tools to a typed, self-describing protocol.	What operations and metadata must a temporal tool expose?

dominate, so that a benchmark score in isolation no longer tells us whether the deployed system can actually be trusted ([Xia et al., 2026](#)).

This reframes the central question. The field should stop asking only whether an LLM can forecast a numerical sequence, and start asking whether an agent can perceive, reason, plan, remember, and act reliably under those same pressures. We pursue that question through six frontiers, each tied to one layer of the closed-loop object of [Sec. 2](#) and to the chapter that surveys it, following the order of the survey: verifiable temporal reasoning ([Sec. 4](#)), end-to-end agentic training ([Sec. 5](#)), memory and lifelong adaptation ([Sec. 6](#)), temporal world models ([Sec. 7](#)), decision-centric evaluation ([Sec. 9](#)), and a standardized temporal tooling protocol that connects them ([Sec. 10](#)). For each, we first look back at the shift that brought the field to its current position, then look forward to the properties the next generation of systems will need; [Tab. 12](#) maps the six, and we close by stating the survey’s position ([Sec. 11.7](#)).

11.1 Verifiable Temporal Reasoning

Look Back Early agentic systems inherited the assumption that a correct final answer signals correct reasoning, the same standard used to score a forecast. That assumption breaks on time series, where a fluent rationale can be numerically, causally, or temporally wrong while still landing on a plausible number. Two developments have exposed the gap. The first comes from evaluation: rather than scoring a single answer, recent benchmarks decompose temporal understanding into distinct abilities, including recognizing patterns, separating signal from noise, detecting anomalies, judging similarity, and reasoning about cause and lag, and they show that a system can pass on one while failing silently on another, so that accuracy on a held-out answer says little about which ability was actually exercised ([Cai et al., 2024](#); [Weng et al., 2026](#); [Ye et al., 2025](#); [Guan et al., 2026](#)). The second comes from the reasoning methods themselves, which have moved beyond a single chain of thought toward reviewing and correcting intermediate steps, explicitly selecting the segments and evidence a claim depends on, and gathering external observations before committing to an answer ([Su et al., 2025](#); [Messica et al., 2026](#); [Liu et al., 2025d](#)). Both lines reach the same conclusion that the accuracy of the final-answer is too coarse to certify a temporal argument because it only considers the destination and not the route taken to reach it.

Future The unit of trust has to move from the answer to the step, and a verifiable temporal reasoner should exhibit three properties. First, its reasoning should be **step-grounded**: every claim it makes about trend, seasonality, an anomaly, a lag, or a causal effect should be bound to something checkable, either an executable computation over the series or an explicit citation to an observation, so that a wrong intermediate claim is caught rather than carried into the conclusion. Second, it should be **calibrated about its own competence**: temporal evidence is often too short, too noisy, or too confounded to support a confident answer, and a system that can recognize when the data cannot settle a question is more trustworthy than one that always responds.

Third, this discipline should be **learned rather than prompted**, which means supervising the reasoning process and not only its output and training on traces whose every step has been verified (Zhou et al., 2026; Guan et al., 2025). The open problem is to make step-level verification the default contract of the reasoning layer (Sec. 4) rather than an optional add-on, so that a temporal argument arrives already carrying the evidence that makes it auditable rather than merely persuasive.

11.2 End-to-End Agentic Training

Look Back In today’s systems the loop is assembled, not learned: planning, tool routing, reflection, and memory access are each prompt-engineered or tuned in isolation and then wired together by hand. Two limits have pushed the field toward training the loop itself. The first is brittleness, since a pipeline tuned on one regime breaks when the data, the tool set, or the horizon changes, because nothing in it was optimized for the decision it ultimately makes. The second is the arrival of methods showing that the loop can be learned end to end, through behavior cloning and reinforcement learning over tool-use trajectories, tool-augmented sequential decision policies for forecasting, and reward-driven schemes that incentivize complex reasoning or structured financial decisions (Wu et al., 2026b; Tao et al., 2026b; Guan et al., 2025; Xiao et al., 2025b). A complementary line supplies the supervision such training needs, synthesizing process-verifiable traces whose every step, and not only the final answer, can be checked (Zhou et al., 2026). Together they suggest that an agent’s competence should come from optimization against the task rather than from the engineer’s intuition.

Future The open problem is to make the loop a single learnable object, and end-to-end training has to achieve three properties. First, the loop should be **jointly optimized**: perception, planning, tool selection, memory access, reflection, and action should be trained against one long-horizon objective rather than tuned as separate parts that merely share an interface. Second, it should be **credit-assignable under delay**, because a temporal decision is often validated only long after it is made, so training must propagate reward across long, sparse horizons and attribute outcomes to the steps that actually caused them. Third, the reward itself should be **multi-objective by construction**, combining accuracy, calibration, and operational safety into one signal rather than chasing predictive error alone, since an agent that acts can be confidently wrong in costly ways. Resolving these is what the planning and action layer (Sec. 5) must settle before end-to-end training pays off rather than merely relocating where the tuning happens.

11.3 Memory and Lifelong Adaptation

Look Back The first generation of time series agents treated the context window as memory: whatever mattered was pasted into the prompt. Distribution shift, rare events, and recurring regimes defeat that approach, because the evidence an agent needs is often older, sparser, or more structured than any window can hold. The response has split into two complementary lines. One gives declarative memory its own structure, separating historical patterns from distilled reasoning and general rules, layering short-term signals against long-term context, and storing failures alongside successes so that past mistakes inform later decisions (Tao et al., 2026a; Yu et al., 2025b; Shi et al., 2026a). The other stores procedure rather than fact, letting an agent that repeatedly builds pipelines reuse how it solved a problem and not merely what it concluded, so that competence accumulates across tasks instead of restarting each time (Pan et al., 2026; Xu et al., 2026e; Cheng and Chin, 2024). Both lines recast memory from a passive transcript into an actively curated asset, yet they also expose how little we can yet say about whether that asset genuinely helps.

Future The hard questions are which experience is worth keeping and whether keeping it helps, and a lifelong temporal agent should satisfy three properties. First, its memory should be **actively governed**: it needs principled policies for what to retrieve, when to update, and what to forget, rather than an ever-growing store that is never pruned. Second, every decision should be **memory-auditable**, so that one can trace which stored item shaped a given prediction or action, both to debug failures and to detect when stale memory is doing harm. Third, the value of memory must be **measured across sessions, not within one**, because the field still cannot tell whether memory improves an agent by capturing reusable temporal structure or by leaking examples from the evaluation distribution; settling that demands lifelong evaluation that separates in-session context from accumulated memory, tests under regime shift, and reports both the gains and the failures that memory amplifies (Jalori et al., 2025). These are the gaps the memory layer (Sec. 6) leaves open.

11.4 Temporal World Models

Look Back For most of its history the field has equated modeling a time series with predicting its continuation. An agent that only predicts, however, cannot answer the questions deployment actually poses, such as what would happen under a different intervention or why a regime changed, because it holds no model of the environment that produced the series. Two shifts are moving the field past pure prediction. The first reframes the target from a trajectory to the mechanism behind it, binding temporal reasoning to executable tools and checking predictions against counterfactual consistency rather than against a single held-out future (Zhang et al., 2026b). The second is the appearance of domain prototypes that begin to simulate and intervene rather than only extrapolate, such as RFID (Yang et al., 2025g) and Chronicle (Quinlan et al., 2026), even though none yet offers a full agentic simulator in which an agent can act, observe the consequences, and revise its beliefs. Together they mark a move from asking what comes next to asking what would happen if.

Future The practical question is what a usable temporal world model must minimally provide, and we identify three properties. First, it should be **generative over futures**, able to roll out plausible continuations conditioned on candidate actions rather than emit a single point or interval. Second, it should be **counterfactually queryable**, able to answer what-if questions about interventions and alternative histories, since an agent that plans must compare futures it never observed. Third, its mechanism should be **verifiable rather than opaque**, exposing the assumptions and tool calls behind a simulated future so that its claims can be checked against data. Because a universal simulator is out of reach, the realistic path is partially simulatable domains, such as agriculture, traffic, and energy, where physical and operational structure constrains the space of futures enough to make such a model both learnable and checkable, which is where the world-model layer (Sec. 7) argues that progress should begin.

11.5 Decision-Centric Evaluation

Look Back Time series research has long been scored by how close a prediction lands to the truth, and the first agentic systems were judged the same way. But the value of an agent usually lies in what it does with a forecast, not in the forecast itself, and a lower error does not guarantee a better decision. Evaluation has accordingly begun to follow the action along two lines. The first broadens what is measured beyond accuracy: engineering agents are scored by whether they can actually build and submit a working solution across datasets and code (Cai et al., 2025b), forecasting agents by proper scoring and verifiable settlement in prediction markets (Nechepurenko and Shuvalov, 2026), trading agents by realized return and drawdown rather than classification accuracy (Xiao et al., 2024; Xu et al., 2025c), and control agents by delay, throughput, and control reward (Da et al., 2024). The second changes when the measurement is taken, judging an agent over a rolling deployment rather than a frozen test split, so that adaptation and failure recovery enter the score. Both lines reveal that a single error number certifies neither the workflow that produced a decision nor the consequences of the decision itself.

Future These scattered measures should converge into one evaluation contract, which must have three properties. First, it should be **joint over prediction, uncertainty, and action**, scoring calibrated confidence and the quality of the resulting decision together rather than rewarding point accuracy in isolation. Second, it should be **robust to delayed feedback**, because the signal an agent acts on is often validated only much later, so the contract must reward sound process before outcomes resolve rather than merely tally outcomes after the fact. Third, it should be **deployment-faithful**, exercising agents under shift, partial observability, and operational cost so that a high score reflects behavior that would survive in production rather than on a benchmark. Constructing such a contract is the task left to the benchmark layer (Sec. 9) and to the application domains (Sec. 10) that supply the delayed, costly feedback it must be calibrated against.

11.6 Standardized Temporal Tooling

Look Back As agentic systems have multiplied, each has rebuilt the same substrate, including diagnosing data, routing to engines, selecting models, logging runs, and reporting results, and each has defined its own tools, observations, and return formats along the way (Zhao et al., 2025; Garza and Rosillo, 2025; Li et al.,

2026b; Ye et al., 2026). The cost shows up on two fronts. Scientifically, results are hard to compare, since an evidence trace produced by one system cannot be lined up against another’s when the operators, units, and logs differ, so progress is difficult to attribute. Practically, nothing is reusable, since a tool built for one agent rarely transfers to the next and each project re-implements decomposition, validation, and inference from scratch. Early domain protocols that package operations as named, callable tools hint at what a shared substrate could look like (Zhang et al., 2026b), but they remain tied to one system rather than offering a common interface the field can build on.

Future The field needs a temporal tooling protocol, and three properties should define it. First, operators should be **typed and self-describing**, exposing input frequency, value units, missingness policy, window semantics, covariate alignment, model version, and output confidence, so that a tool can be invoked correctly without reading its source. Second, every result should be **provenance-carrying**, recording the inputs, parameters, and randomness behind it so that an evidence trace is auditable and reproducible across systems. Third, the protocol should be **verifier-ready**, standardizing common operations such as decomposition, change-point detection, causal-lag discovery, rolling validation, and foundation-model inference so that the checks a reasoner relies on can run against any compliant tool. Such an interface would let the tool libraries, operator sets, and evidence-gathering routines now scattered across systems interoperate (Wu et al., 2026b; Ye et al., 2026; Liu et al., 2025d), turning the per-paper engineering documented across the application domains (Sec. 10) into shared infrastructure that later systems can build on instead of rebuilding.

11.7 Position: Time Series Agents as Temporal Decision Systems

This survey takes the position that agentic time series systems should be evaluated as temporal decision systems. A model-centric survey asks whether a model can reduce error or improve zero-shot forecasting; an agentic survey asks whether a system can perceive heterogeneous temporal evidence, reason with numerically grounded steps, choose tools and actions, remember useful experience, verify its conclusions, and adapt when the environment changes.

The frontier is therefore not a single larger time series foundation model but the closed-loop object $\mathcal{A}_{TS}$ of Sec. 2 made reliable: perception, reasoning, planning and action, memory, and world modeling composed into one loop, with verification cutting across every layer. The six frontiers above trace that loop and ask, in turn, whether each component can be trusted, can be learned, can accumulate, can simulate, and can be measured and connected. Their through-line is that progress now comes from making the components work together under decision-centric evaluation, not from improving any one of them alone. In this view, time series agents are not NLP agents applied to numbers; they are closed-loop systems for acting under temporal uncertainty, and the task ahead is to make that loop trustworthy, auditable, and capable of improving over time.

12 Conclusion

This survey has examined agentic time series systems as an emerging paradigm for transitioning from model-centric prediction to closed-loop temporal intelligence. By distinguishing between time series foundation models, LLM-based translators, temporal reasoners, and closed-loop agentic systems, we clarify the conceptual boundary between LLM-enhanced time series modeling and genuine time series agency. We further organize existing work through a five-layer architecture comprising perception, reasoning, planning and action, memory and knowledge, and world models. We position reliability and trustworthiness as cross-cutting evaluative dimensions governing every layer of the agentic pipeline. Rather than viewing recent progress as an unconnected assortment of tools, prompts, agents and benchmarks, this framework considers them as part of a coherent transition. Time series systems evolve from static predictors into adaptive systems that observe temporal environments, acquire evidence, reason about uncertain dynamics, act through tools and workflows, accumulate experience and verify their own behavior.

Looking forward, the key challenge lies in ensuring that time series agents are not only more autonomous, but also that their autonomy is reliable, auditable and useful in dynamic environments where decisions have consequences. Future systems need to integrate stronger temporal reasoning, memory-centric adaptation, decision-aware evaluation, standardized tool protocols, and temporal world models capable of simulation and counterfactual analysis. They must also be able to interact with data agents that oversee the entire lifecycle of

temporal data, from acquisition and cleaning to augmentation, governance, and deployment. Addressing these challenges could move agentic time series systems beyond forecasting assistants toward trustworthy temporal analysts, adaptive decision-support systems, and autonomous scientific instruments for understanding dynamic worlds. We hope that this survey will provide a structured foundation in this area and help to guide future research toward temporal agents that are not only capable, but also reliable, transparent, and aligned with human and domain-specific constraints.

References

- Md Atik Ahamed, Mihir Parmar, Palash Goyal, Chun-Liang Li, Qiang Cheng, Tomas Pfister, and Jinsung Yoon. Reasoning-aware training for time series forecasting. *arXiv preprint arXiv:2605.08625*, 2026a.
- Md Atik Ahamed, Mihir Parmar, Palash Goyal, Yiwen Song, Long T Le, Qiang Cheng, Chun-Liang Li, Hamid Palangi, Jinsung Yoon, and Tomas Pfister. Tfrbench: A reasoning benchmark for evaluating forecasting systems. *arXiv preprint arXiv:2604.05364*, 2026b.
- Taha Aksu, Gerald Woo, Juncheng Liu, Xu Liu, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. Gift-eval: A benchmark for general time series forecasting model evaluation. *arXiv preprint arXiv:2410.10393*, 2024.
- Samaneh Aminikhanghahi and Diane J Cook. A survey of methods for time series change point detection. *Knowledge and information systems*, 51(2):339–367, 2017.
- Yihao Ang, Yifan Bao, Lei Jiang, Jiajie Tao, Anthony KH Tung, Lukasz Szpurch, and Hao Ni. Structured agentic workflows for financial time-series modelling with llms and reflective feedback. In *Proceedings of the 6th ACM International Conference on AI in Finance*, pages 924–932, 2025.
- Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815*, 2024.
- Abdul Fatir Ansari, Oleksandr Shchur, Jaris Küken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, et al. Chronos-2: From univariate to universal forecasting. *arXiv preprint arXiv:2510.15821*, 2025.
- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba, Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. 2025. <https://arxiv.org/abs/2506.09985>.
- Yifan Bao, Xinyu Xi, Xinyu Liu, Wen Ge, Lei Jiang, Kevin Zhang, Raad Khraishi, Yihao Ang, Anthony KH Tung, Lukasz Szpruch, et al. Mosaic: Modular orchestration for structured agentic intelligence and composition. *arXiv preprint arXiv:2606.00708*, 2026.
- Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. Pangu-weather: A 3d high-resolution model for fast and accurate global weather forecast. *arXiv preprint arXiv:2211.02556*, 2022.
- George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. *Time series analysis: forecasting and control*. John Wiley & Sons, 2015.
- David R Brillinger. *Time series: data analysis and theory*. SIAM, 2001.
- Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments. 2024. <https://arxiv.org/abs/2402.15391>.
- William Gouvêa Buratto, Gabriel Villarrubia Gonzalez, Ademir Nied, Laio Oriel Seman, Anne Carolina Rodrigues Klaar, and Stefano Frizzo Stefenon. An autonomous large language model-agent framework for transparent and local time series forecasting. *Advanced Intelligent Discovery*, page e202500236, 2026.
- Yifu Cai, Arjun Choudhry, Mononito Goswami, and Artur Dubrawski. Timeseriesexam: A time series understanding exam. *arXiv preprint arXiv:2410.14752*, 2024.

- Yifu Cai, Xinyu Li, Mononito Goswami, Michał Wiliński, Gus Welter, and Artur Dubrawski. Llm agents struggle at time series machine learning engineering. In *1st ICML Workshop on foundation models for structured data*, 2025a.
- Yifu Cai, Xinyu Li, Mononito Goswami, Michał Wiliński, Gus Welter, and Artur Dubrawski. Timeseriesgym: A scalable benchmark for (time series) machine learning engineering agents. *arXiv preprint arXiv:2505.13291*, 2025b.
- Defu Cao, Furong Jia, Sercan O Arik, Tomas Pfister, Yixiang Zheng, Wen Ye, and Yan Liu. Tempo: Prompt-based generative pre-trained transformer for time series forecasting. *arXiv preprint arXiv:2310.04948*, 2023.
- Defu Cao, Michael Gee, Jinbo Liu, Hengxuan Wang, Wei Yang, Rui Wang, and Yan Liu. Conversational time series foundation models: Towards explainable and effective forecasting. *arXiv preprint arXiv:2512.16022*, 2025.
- Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM computing surveys (CSUR)*, 41(3):1–58, 2009.
- Ching Chang, Yidan Shi, Defu Cao, Wei Yang, Jeehyun Hwang, Haixin Wang, Jiacheng Pang, Wei Wang, Yan Liu, Wen-Chih Peng, et al. A survey of reasoning and agentic systems in time series with large language models. *arXiv preprint arXiv:2509.11575*, 2025.
- Ching Chang, Jeehyun Hwang, Yidan Shi, Haixin Wang, Wei Wang, Wen-Chih Peng, and Tien-Fu Chen. Time-imm: A dataset and benchmark for irregular multimodal multivariate time series. *Advances in Neural Information Processing Systems*, 38, 2026.
- Jialin Chen, Aosong Feng, Ziyu Zhao, Juan Garza, Gaukhar Nurbek, Cheng Qin, Ali Maatouk, Leandros Tassioulas, Yifeng Gao, and Rex Ying. MTBench: A multimodal time series benchmark for temporal reasoning and question answering. *arXiv preprint arXiv:2503.16858*, 2025a.
- Jiaxiang Chen, Mingxi Zou, Zhuo Wang, Qifan Wang, Danny Dongning Sun, Zhang Chi, and Zenglin Xu. Fin-HEAR: Human expertise and adaptive risk-aware temporal reasoning for financial decision-making. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 1648–1672, Suzhou, China, November 2025b. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.87. <https://aclanthology.org/2025.findings-emnlp.87/>.
- Peng Chen, Siyuan Wang, Shiyuan Hu, Xingjian Wu, Yang Shu, Zhongwen Rao, Meng Wang, Yijie Li, Bin Yang, and Chenjuan Guo. Towards a multimodal foundation model for time series analysis, 2026. <https://openreview.net/forum?id=dcfHcCAG3C>.
- Zihan Chen, Lei Nico Zheng, Cheng Lu, Jialu Yuan, and Di Zhu. Chatgpt informed graph neural network for stock movement prediction. *arXiv preprint arXiv:2306.03763*, 2023.
- Dawei Cheng, Fangzhou Yang, Sheng Xiang, and Jin Liu. Financial time series forecasting with multi-modality graph neural network. *Pattern Recognition*, 121:108218, 2022.
- Junyan Cheng and Peter Chin. Sociodojo: Building lifelong analytical agents with real-world text and time series. In *The Twelfth International Conference on Learning Representations*, 2024.
- Mingyue Cheng, Xiaoyu Tao, Qi Liu, Ze Guo, and Enhong Chen. Position: Beyond model-centric prediction–agentic time series forecasting. *arXiv preprint arXiv:2602.01776*, 2026a.
- Mingyue Cheng, Jiahao Wang, Daoyu Wang, Xiaoyu Tao, Qi Liu, and Enhong Chen. Can slow-thinking llms reason over time? empirical studies in time series forecasting. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining*, pages 99–110, 2026b.
- Leonardo Christino and Fernando V. Paulovich. Chatkg: Visualizing time-series patterns aided by intelligent agents and a knowledge graph. *Computers & Graphics*, 124:104092, 2024. ISSN 0097-8493. doi: <https://doi.org/10.1016/j.cag.2024.104092>. <https://www.sciencedirect.com/science/article/pii/S0097849324002279>.
- Jarosław A. Chudziak and Michał Wawer. Elliottagents: A natural language-driven multi-agent system for stock market analysis and prediction. *arXiv preprint arXiv:2507.03435*, 2025.
- Hsing-Huan Chung, Shijun Li, Yoav Wald, Xing Han, Suchi Saria, and Joydeep Ghosh. Milm: Large language models for multimodal irregular time series with informative sampling. *arXiv preprint arXiv:2605.13711*, 2026.
- Robert B Cleveland, William S Cleveland, Jean E McRae, Irma Terpenning, et al. Stl: A seasonal-trend decomposition. *J. off. Stat*, 6(1):3–73, 1990.

- Zhiqing Cui, Binwu Wang, Qingxiang Liu, Yejiang Wang, Zhengyang Zhou, Yuxuan Liang, and Yang Wang. Augur: Modeling covariate causal associations in time series via large language models. *arXiv preprint arXiv:2510.07858*, 2025.
- Longchao Da, Kuanru Liou, Tiejun Chen, Xuesong Zhou, Xiangyong Luo, Yezhou Yang, and Hua Wei. Open-ti: Open traffic intelligence with augmented language model. *International Journal of Machine Learning and Cybernetics*, 15(10):4761–4786, 2024.
- Xilin Dai, Ruidi Zhou, Jinhao Zhang, Keyi He, Fanfan Lin, and Hao Ma. Socnet: A physics-guided neural network for battery state-of-charge estimation robust to temperature variations and sensor noises. *IEEE Transactions on Transportation Electrification*, 2025a.
- Xilin Dai, Ruidi Zhou, Jinhao Zhang, Fanfan Lin, Weifeng Zhang, and Hao Ma. Socgate: Physics-gated neural network for stable multicycle estimation of battery state-of-charge. *IEEE Transactions on Industrial Electronics*, 2025b.
- Xilin Dai, Wanxu Cai, Zhijian Xu, and Qiang Xu. Position: Universal time series foundation models rest on a category error. *arXiv preprint arXiv:2602.05287*, 2026a.
- Xilin Dai, Zhijian Xu, Wanxu Cai, and Qiang Xu. From samples to scenarios: A new paradigm for probabilistic forecasting. *arXiv preprint arXiv:2509.19975*, 2026b.
- Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. *arXiv preprint arXiv:2310.10688*, 2023.
- Sarkar Snigdha Sarathi Das, Palash Goyal, Mihir Parmar, Nanyun Peng, Vishy Tirumalashetty, Chun-Liang Li, Rui Zhang, Jinsung Yoon, and Tomas Pfister. Nexus: An agentic framework for time series forecasting. *arXiv preprint arXiv:2605.14389*, 2026.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems*, 37:82895–82920, 2024.
- Ashok Devireddy and Shunping Huang. Calm: A framework for continuous, adaptive, and llm-mediated anomaly detection in time-series streams. *arXiv preprint arXiv:2508.21273*, 2025.
- Kuiye Ding, Fanda Fan, Yao Wang, Ruijie Jian, Xiaorui Wang, Luqi Gong, Yishan Jiang, Chunjie Luo, and Jianfeng Zhan. Dualsg: A dual-stream explicit semantic-guided multivariate time series forecasting framework. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 508–517, 2025.
- Yuhao Ding, Youngsuk Park, Karthick Gopalswamy, Hilaf Hasson, Bernie Wang, and Jun Huan. Dynamic ensemble for probabilistic time-series forecasting via deep reinforcement learning, 2023. <https://openreview.net/forum?id=a6NvoZ5DLoe>.
- Zihan Dong, Xinyu Fan, and Zhiyuan Peng. Fnspid: A comprehensive financial news dataset in time series. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4918–4927, 2024.
- Wei Fan, Shun Zheng, Pengyang Wang, Rui Xie, Jiang Bian, and Yanjie Fu. Addressing distribution shift in time series forecasting with instance normalization flows. *arXiv preprint arXiv:2401.16777*, 14(8), 2024.
- Syeda Sitara Wishal Fatima and Afshin Rahimi. A review of time-series forecasting algorithms for industrial manufacturing systems. *Machines*, 2024.
- Jie Feng, Tianhui Liu, Yuwei Du, Siqi Guo, Yuming Lin, and Yong Li. Citygpt: Empowering urban spatial cognition of large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 591–602, 2025.
- Kun Feng, Ziwei Shan, Yuchen Fang, Yiyang Tan, Sihan Lu, Shuqi Gu, Lintao Ma, Xingyu Lu, and Kan Ren. Kairosagent: Agentic time series forecasting with fused semantic reasoning. *arXiv preprint arXiv:2605.30002*, 2026.
- Mohamed Amine Ferrag, Norbert Tihanyi, Djallel Hamouda, Leandros Maglaras, Abderrahmane Lakas, and Merouane Debbah. From prompt injections to protocol exploits: Threats in llm-powered ai agents workflows. *ICT Express*, 2025.
- Azul Garza and Renée Rosillo. Timecopilot. *arXiv preprint arXiv:2509.00616*, 2025.
- Yunfeng Ge, Ming Jin, Yiji Zhao, Hongyan Li, Bo Du, Chang Xu, and Shirui Pan. Eventtsf: Event-aware non-stationary time series forecasting. *arXiv preprint arXiv:2508.13434*, 2025.

- Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. Moment: A family of open time-series foundation models. *arXiv preprint arXiv:2402.03885*, 2024.
- Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew G Wilson. Large language models are zero-shot time series forecasters. *Advances in neural information processing systems*, 36:19622–19635, 2023.
- Shuqi Gu, Yongxiang Zhao, Baoyu Jing, and Kan Ren. Counterfactual time series forecasting with textual conditions, 2026. <https://openreview.net/forum?id=YRp4xqTs3n>.
- Yile Gu, Yifan Xiong, Jonathan Mace, Yuting Jiang, Yigong Hu, Baris Kasikci, and Peng Cheng. Argos: Agentic time-series anomaly detection with autonomous rule generation via large language models. *arXiv preprint arXiv:2501.14170*, 2025.
- Tong Guan, Zijie Meng, Dianqi Li, Shiyu Wang, Chao-Han Huck Yang, Qingsong Wen, Zuozhu Liu, Sabato Marco Siniscalchi, Ming Jin, and Shirui Pan. Timeomni-1: Incentivizing complex reasoning with time series in large language models. *arXiv preprint arXiv:2509.24803*, 2025.
- Tong Guan, Sheng Pan, Johan Barthelemy, Zhao Li, Yujun Cai, Cesare Alippi, Ming Jin, and Shirui Pan. Timeomni-v1: Unified models for time series understanding and generation. *arXiv preprint arXiv:2602.17149*, 2026.
- Malgorzata Gwiazda, Yifu Cai, Mononito Goswami, Arjun Choudhry, and Artur Dubrawski. Timeseriesexamagent: Creating time series reasoning benchmarks at scale. In *The Fourteenth International Conference on Learning Representations*, 2026. <https://openreview.net/forum?id=DewXWSvQPH>.
- Yifan Hu, Yuante Li, Peiyuan Liu, Yuxia Zhu, Naiqi Li, Tao Dai, Shu-tao Xia, Dawei Cheng, and Changjun Jiang. Fintsb: A comprehensive and practical benchmark for financial time series forecasting. *arXiv preprint arXiv:2502.18834*, 2025a.
- Yifan Hu, Peiyuan Liu, Yuante Li, Dawei Cheng, Naiqi Li, Tao Dai, Jigang Bao, and Shu-Tao Xia. Finmamba: Market-aware graph enhanced multi-level mamba for stock movement prediction. *arXiv preprint arXiv:2502.06707*, 2025b.
- Yifan Hu, Peiyuan Liu, Peng Zhu, Dawei Cheng, and Tao Dai. Adaptive multi-scale decomposition framework for time series forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(16):17359–17367, 2025c.
- Yifan Hu, Guibin Zhang, Peiyuan Liu, Disen Lan, Naiqi Li, Dawei Cheng, Tao Dai, Shu-Tao Xia, and Shirui Pan. TimeFilter: Patch-specific spatial-temporal graph filtration for time series forecasting. In *International Conference on Machine Learning*, 2025d.
- Yifan Hu, Jie Yang, Tian Zhou, Peiyuan Liu, Yujin Tang, Rong Jin, and Liang Sun. Bridging past and future: Distribution-aware alignment for time series forecasting. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Yusha Hu and Yi Man. Energy consumption and carbon emissions forecasting for industrial processes: Status, challenges and perspectives. *Renewable and Sustainable Energy Reviews*, 2023.
- Qihe Huang, Zhengyang Zhou, Kuo Yang, Gengyu Lin, Zhongchao Yi, and Yang Wang. Leret: Language-empowered retentive network for time series forecasting. In *IJCAI*, pages 4165–4173, 2024.
- Qihe Huang, Zhengyang Zhou, Yangze Li, Kuo Yang, Binwu Wang, and Yang Wang. Many minds, one goal: Time series forecasting via sub-task specialization and inter-agent cooperation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Gunjan Jalori, Preetika Verma, and Serkan O Arik. FLAIRR-TS - forecasting LLM-agents with iterative refinement and retrieval for time series. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 15427–15437, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.834. <https://aclanthology.org/2025.findings-emnlp.834/>.
- Yushan Jiang, Wenchao Yu, Geon Lee, Dongjin Song, Kijung Shin, Wei Cheng, Yanchi Liu, and Haifeng Chen. Timexl: Explainable multi-modal time series prediction with llm-in-the-loop. *Advances in Neural Information Processing Systems*, 38:135431–135467, 2026.
- Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. Time-llm: Time series forecasting by reprogramming large language models. In *International conference on learning representations*, volume 2024, pages 23857–23880, 2024.

- Woojeong Jin, Rahul Khanna, Suji Kim, Dong-Ho Lee, Fred Morstatter, Aram Galstyan, and Xiang Ren. Forecastqa: A question answering challenge for event forecasting with temporal text data. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4636–4650, 2021.
- Baoyu Jing, Sanhorn Chen, Lecheng Zheng, Boyu Liu, Zihao Li, Jiaru Zou, Tianxin Wei, Zhining Liu, Zhichen Zeng, Ruizhong Qiu, et al. Tsaqa: Time series analysis question and answering benchmark. *arXiv preprint arXiv:2601.23204*, 2026.
- Hyeonwon Kang, Jeongseob Kim, Jinwoo Park, and Pilsung Kang. Detecting time series anomalies like an expert: A multi-agent llm framework with specialized analyzers. *arXiv preprint arXiv:2605.05725*, 2026.
- Ezra Karger, Houtan Bastani, Chen Yueh-Han, Zachary Jacobs, Danny Halawi, Fred Zhang, and Philip E Tetlock. Forecastbench: A dynamic benchmark of ai forecasting capabilities. In *International Conference on Learning Representations*, volume 2025, pages 93943–93980, 2025.
- Emaad Khwaja, Chris Lettieri, Gerald Woo, Eden Belouadah, Marc Cenac, Guillaume Jarry, Enguerrand Paquin, Xunyi Zhao, Viktoriya Zhukov, Othmane Abou-Amal, et al. Toto 2.0: Time series forecasting enters the scaling era. *arXiv preprint arXiv:2605.20119*, 2026.
- Rebecca Killick, Paul Fearnhead, and Idris A Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500):1590–1598, 2012.
- Kai Kim, Howard Tsai, Rajat Sen, Abhimanyu Das, Zihao Zhou, Abhishek Tanpure, Mathew Luo, and Rose Yu. Multi-modal forecaster: Jointly predicting time series and textual data. *arXiv preprint arXiv:2411.06735*, 2024.
- Anton Kolonin, Alexey Glushchenko, Evgeny Bochkov, and Abhishek Saxena. Time series augmented generation for financial applications. *arXiv preprint arXiv:2604.19633*, 2026.
- Yaxuan Kong, Yiyuan Yang, Yoontae Hwang, Wenjie Du, Stefan Zohren, Zhangyang Wang, Ming Jin, and Qingsong Wen. Time-mqa: Time series multi-task question answering with context enhancement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025a.
- Yaxuan Kong, Yiyuan Yang, Shiyu Wang, Chenghao Liu, Yuxuan Liang, Ming Jin, Stefan Zohren, Dan Pei, Yan Liu, and Qingsong Wen. Achieving time series reasoning requires rethinking model design, tasks formulation, and evaluation. *arXiv preprint arXiv:2502.01477*, 2025b.
- Yaxuan Kong, Qingren Yao, Yuqi Nie, Yichen Li, Yilei Shao, Stefan Zohren, Anna Vettoruzzo, Joaquin Vanschoren, Ming Jin, and Qingsong Wen. Timesage-mt: A multi-turn benchmark for evaluating agentic time series reasoning. *arXiv preprint arXiv:2606.01498*, 2026.
- Siva Rama Krishna Kottapalli, Karthik Hubli, Sandeep Chandrashekhara, Garima Jain, Sunayana Hubli, Gayathri Botla, and Ramesh Doddaiiah. Foundation models for time series: A survey. *arXiv preprint arXiv:2504.04011*, 2025.
- Jiyong Kwon, Ujin Jeon, Sooji Lee, and Guang Lin. Aivv: Neuro-symbolic llm agent-integrated verification and validation for trustworthy autonomous systems. *arXiv preprint arXiv:2604.02478*, 2026.
- Tian Lan, Hao Duong Le, Jinbo Li, Wenjun He, Meng Wang, Chenghao Liu, and Chen Zhang. Axis: Explainable time series anomaly detection with large language models. *arXiv preprint arXiv:2509.24378*, 2025.
- Xiang Lan, Feng Wu, Kai He, Qinghao Zhao, Shenda Hong, and Mengling Feng. Gem: Empowering mllm for grounded ecg understanding with time series and images. *Advances in Neural Information Processing Systems*, 38:94421–94455, 2026.
- Patrick Langer, Thomas Kaar, Max Rosenblattl, Maxwell A Xu, Winnie Chow, Martin Maritsch, Robert Jakob, Ning Wang, Juncheng Liu, Aradhana Verma, et al. Opentslm: Time-series language models for reasoning over multivariate medical text-and time-series data. *arXiv preprint arXiv:2510.02410*, 2025.
- Geon Lee, Wenchao Yu, Kijung Shin, Wei Cheng, and Haifeng Chen. Timecap: Learning to contextualize, augment, and predict time series events with large language model agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 18082–18090, 2025.
- Jaehoon Lee, Suhwan Park, Tae Yoon Lim, Seunghan Lee, Jun Seo, Dongwan Kang, Hwanil Choi, Minjae Kim, Sungdong Yoo, SoonYoung Lee, et al. Fintexts: Financial text-paired time-series dataset via semantic-based and multi-level pairing. *arXiv preprint arXiv:2603.02702*, 2026.

- Peixuan Lei, Jie Song, Yuzhe Hao, Tao Chen, Yuxuan Zhang, LEI JIA, Yuanxiang Li, et al. Itformer: Bridging time series and natural language for multi-modal qa with large-scale multitask dataset. In *Forty-second International Conference on Machine Learning*, 2025.
- Hao Li, Yu-Hao Huang, Chang Xu, Viktor Schlegel, Renhe Jiang, Riza Batista-Navarro, Goran Nenadic, and Jiang Bian. Bridge: Bootstrapping text to control time-series generation via multi-agent iterative optimization and diffusion modeling. *arXiv preprint arXiv:2503.02445*, 2025.
- Lin Li, Jiawei Huang, Qihao Quan, Dan Li, Boxin Li, Xiao Zhang, Erli Meng, Wenjie Feng, Jian Lou, and See-Kiong Ng. Empowering vlms for few-shot multimodal time series classification via tailored agentic reasoning. *arXiv preprint arXiv:2605.09395*, 2026a.
- Shuhao Li, Wanfu Zheng, Mingcheng Li, Shanshuo Xing, and Zhe Wang. Autolfin: A multi-agent llm framework for automated building load forecasting. In *Building Simulation*, pages 1–24. Springer, 2026b.
- Sirui Li, Shuhan Xiao, Mihir Joshi, Ahmed Metwally, Daniel McDuff, Wei Wang, and Yuzhe Yang. Hearts: Benchmarking llm reasoning on health time series. *arXiv preprint arXiv:2603.06638*, 2026c.
- Yuan Li, Bingqiao Luo, Qian Wang, Nuo Chen, Xu Liu, and Bingsheng He. CryptoTrade: A reflective LLM-based agent to guide zero-shot cryptocurrency trading. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024.
- Yuante Li, Xu Yang, Xiao Yang, Xisen Wang, Weiqing Liu, and Jiang Bian. R&d-agent-quant: a multi-agent framework for data-centric factors and model joint optimization. *Advances in Neural Information Processing Systems*, 38, 2026d.
- Zechen Li, Baiyu Chen, Hao Xue, and Flora D. Salim. Zara: Training-free motion time-series reasoning via evidence-grounded llm agents. *arXiv preprint arXiv:2508.04038*, 2026e.
- Minhua Lin, Zhengzhang Chen, Yanchi Liu, Xujiang Zhao, Zongyu Wu, Junxiang Wang, Xiang Zhang, Suhang Wang, and Haifeng Chen. Decoding time series with llms: A multi-agent framework for cross-domain annotation. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 6244–6281, 2026.
- Roderick JA Little and Donald B Rubin. *Statistical analysis with missing data*. John Wiley & Sons, 2019.
- Chenghao Liu, Taha Aksu, Juncheng Liu, Xu Liu, Hanshu Yan, Quang Pham, Silvio Savarese, Doyen Sahoo, Caiming Xiong, and Junnan Li. Moirai 2.0: When less is more for time series forecasting. *arXiv preprint arXiv:2511.11698*, 2025a.
- Haixin Liu, Harshavardhan Kamarthi, Lingkai Kong, Zhiyuan Zhao, Chao Zhang, and B Aditya Prakash. Time-series forecasting for out-of-distribution generalization using invariant learning. In *International Conference on Machine Learning*, pages 31312–31325. PMLR, 2024a.
- Haixin Liu, Shangqing Xu, Zhiyuan Zhao, Lingkai Kong, Harshavardhan Kamarthi, Aditya B Sasanur, Megha Sharma, Jiaming Cui, Qingsong Wen, Chao Zhang, et al. Time-mmd: Multi-domain multimodal dataset for time series analysis. *Advances in Neural Information Processing Systems*, 2024b.
- Jiashuo Liu, Siyuan Chen, Zaiyuan Wang, Zhiyuan Zeng, Jiacheng Guo, Liang Hu, Lingyue Yin, Suozhi Huang, Wenxin Hao, Yang Yang, et al. Futurex-pro: Extending future prediction to high-value vertical domains. *arXiv preprint arXiv:2601.12259*, 2026a.
- Lei Liu, Yuxing Tian, Chinmay Chakraborty, Jie Feng, Qingqi Pei, Li Zhen, and Keping Yu. Multilevel federated learning-based intelligent traffic flow forecasting for transportation network management. *IEEE Transactions on Network and Service Management*, 2023.
- Peiyuan Liu, Hang Guo, Tao Dai, Naiqi Li, Jigang Bao, Xudong Ren, Yong Jiang, and Shu-Tao Xia. Calf: Aligning llms for time series forecasting via cross-modal fine-tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025b.
- Peiyuan Liu, Beiliang Wu, Yifan Hu, Naiqi Li, Tao Dai, Jigang Bao, and Shu-Tao Xia. Timebridge: Non-stationarity matters for long-term time series forecasting. In *International Conference on Machine Learning*. PMLR, 2025c.
- Penghang Liu, Elizabeth Fons, Svitlana Vyetenko, Daniel Borrajo, Vamsi K Potluru, and Manuela Veloso. Ts-agent: A time series reasoning agent with iterative statistical insight gathering. In *First Workshop on Foundations of Reasoning in Language Models*, 2025d.

- Qingxiang Liu, Zhiqing Cui, Xiaoliang Luo, Yuqian Wu, Zhuoyang Jiang, Huaiyu Wan, Sheng Sun, Lvchun Wang, Wei Yu, and Yuxuan Liang. Rationale-grounded in-context learning for time series reasoning with multimodal large language models. *arXiv preprint arXiv:2601.02968*, 2026b.
- Shixu Liu. Optimizing multi-agent weather captioning via text gradient descent: A training-free approach with consensus-aware gradient fusion. *arXiv preprint arXiv:2603.21673*, 2026.
- Yiding Liu, Yifan Hu, Hongjie Xia, Peiyuan Liu, Hongzhou Chen, Xilin Dai, Zewei Dong, and Jiang-Ming Yang. Falcon-x: A time series foundation model for heterogeneous multivariate modeling. *arXiv preprint arXiv:2605.27286*, 2026c.
- Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. iTransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations*, 2024c.
- Yong Liu, Guo Qin, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Autotimes: Autoregressive time series forecasters via large language models. *Advances in Neural Information Processing Systems*, 37:122154–122184, 2024d.
- Yong Liu, Haoran Zhang, Chenyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Timer: Generative pre-trained transformers are large time series models. *arXiv preprint arXiv:2402.02368*, 2024e.
- Qingqing Long, Zheng Fang, Chen Fang, Chong Chen, Pengfei Wang, and Yuanchun Zhou. Unveiling delay effects in traffic forecasting: a perspective from spatial-temporal delay differential equations. In *Proceedings of the ACM web conference 2024*, pages 1035–1044, 2024.
- Junkai Lu, Peng Chen, Xingjian Wu, Yang Shu, Chenjuan Guo, Christian S Jensen, and Bin Yang. PATRA: Pattern-aware alignment and balanced reasoning for time series question answering. *arXiv preprint arXiv:2602.23161*, 2026.
- Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, et al. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025a.
- Yucong Luo, Yitong Zhou, Mingyue Cheng, Jiahao Wang, Daoyu Wang, Tingyue Pan, and Jintao Zhang. Time series forecasting as reasoning: A slow-thinking approach with reinforced LLMs. *arXiv preprint arXiv:2506.10630*, 2025b.
- Sisuo Lyu, Siru Zhong, Tiegang Chen, Weilin Ruan, Qingxiang Liu, Taiqiang Lv, Qingsong Wen, Raymond Chi-Wing Wong, and Yuxuan Liang. Ts-memory: Plug-and-play memory for time series foundation models. *arXiv preprint arXiv:2602.11550*, 2026.
- Mike A Merrill, Mingtian Tan, Vinayak Gupta, Thomas Hartvigsen, and Tim Althoff. Language models still struggle to zero-shot reason about time series. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3512–3533, 2024.
- Shvat Messica, Jiawen Zhang, Kevin Li, Theodoros Tsiligkaridis, and Marinka Zitnik. Adaptive time series reasoning via segment selection. *arXiv preprint arXiv:2602.18645*, 2026.
- Hamza Mostafa, Om Shastri, and Dennis Lee. Timeseek: Temporal reliability of agentic forecasters. *arXiv preprint arXiv:2604.04220*, 2026.
- Maksym Nechepurenko and Pavel Shuvalov. Foresight arena: An on-chain benchmark for evaluating ai forecasting agents. *arXiv preprint arXiv:2605.00420*, 2026.
- Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. *arXiv preprint arXiv:2211.14730*, 2022.
- Kanghui Ning, Zijie Pan, Yushan Jiang, Anderson Schneider, Yuriy Nevmyvaka, and Dongjin Song. Towards interpretable and trustworthy time series reasoning: A bluesky vision. *arXiv preprint arXiv:2510.16980*, 2025a.
- Kanghui Ning, Zijie Pan, Yu Liu, Yushan Jiang, James Yiming Zhang, Kashif Rasul, Anderson Schneider, Lintao Ma, Yuriy Nevmyvaka, and Dongjin Song. Ts-rag: Retrieval-augmented generation based time series foundation models are stronger zero-shot forecaster. *arXiv preprint arXiv:2503.07649*, 2025b.
- Peisong Niu, Ziqing Ma, Tian Zhou, Weiqi Chen, Lefei Shen, Rong Jin, and Liang Sun. Utilizing strategic pre-training to reduce overfitting: Baguan-a pre-trained weather forecasting model. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025.

- NVIDIA, :, Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, Daniel Dworakowski, Jiaojiao Fan, Michele Fenzi, Francesco Ferroni, Sanja Fidler, Dieter Fox, Songwei Ge, Yunhao Ge, Jinwei Gu, Siddharth Gururani, Ethan He, Jiahui Huang, Jacob Huffman, Pooya Jannaty, Jingyi Jin, Seung Wook Kim, Gergely Klár, Grace Lam, Shiyi Lan, Laura Leal-Taixe, Anqi Li, Zhaoshuo Li, Chen-Hsuan Lin, Tsung-Yi Lin, Huan Ling, Ming-Yu Liu, Xian Liu, Alice Luo, Qianli Ma, Hanzi Mao, Kaichun Mo, Arsalan Mousavian, Seungjun Nah, Sriharsha Niverty, David Page, Despoina Paschalidou, Zeeshan Patel, Lindsey Pavao, Morteza Ramezani, Fitsum Reda, Xiaowei Ren, Vasanth Rao Naik Sabavat, Ed Schmerling, Stella Shi, Bartosz Stefaniak, Shitao Tang, Lyne Tchapmi, Przemek Tredak, Wei-Cheng Tseng, Jibin Varghese, Hao Wang, Haoxiang Wang, Heng Wang, Ting-Chun Wang, Fangyin Wei, Xinyue Wei, Jay Zhangjie Wu, Jiashu Xu, Wei Yang, Lin Yen-Chen, Xiaohui Zeng, Yu Zeng, Jing Zhang, Qinsheng Zhang, Yuxuan Zhang, Qingqing Zhao, and Artur Zolkowski. Cosmos world foundation model platform for physical ai. 2025. <https://arxiv.org/abs/2501.03575>.
- Bokai Pan, Mingyue Cheng, Zhiding Liu, Shuo Yu, Xiaoyu Tao, Yuchong Wu, Qi Liu, Defu Lian, and Enhong Chen. Castflow: Learning role-specialized agentic workflows for time series forecasting. *arXiv preprint arXiv:2604.27840*, 2026.
- Charidimos Papadakis, Angeliki Dimitriou, Giorgos Filandrianos, Maria Lymperaioi, Konstantinos Thomas, and Giorgos Stamou. Atlas: Adaptive trading with llm agents through dynamic prompt optimization and multi-agent coordination. *arXiv preprint arXiv:2510.15949*, 2025.
- Gijun Park. Bridging temporal and textual modalities: A multimodal framework for automated cloud failure root cause analysis. *arXiv preprint arXiv:2601.04709*, 2026.
- Felix Parker, Nimeesha Chan, Chi Zhang, and Kimia Ghobadi. Eliciting chain-of-thought reasoning for time series analysis using reinforcement learning. *arXiv preprint arXiv:2510.01116*, 2025.
- Isha Patel and Iman Rahimi. Ai-based demand forecasting and load balancing for optimising energy use in healthcare systems: A real case study. *Systems*, 2026.
- Oleeviya Babu Poikarayil, Cédric Schockaert, Abdulrahman Nahhas, Christian Daase, Mursal Dawodi, and Jawid Ahmad Baktash. Genautoml: An agentic framework for dynamic architecture generation and optimization in time-series analysis. *arXiv preprint arXiv:2606.05860*, 2026.
- Willa Potosnak, Cristian Challu, Mononito Goswami, Kin G Olivares, Michał Wiliński, Nina Żukowska, and Artur Dubrawski. Investigating compositional reasoning in time series foundation models. *arXiv preprint arXiv:2502.06037*, 2025.
- Ye Qiao, Haoxiang Zhang, Linfeng Wang, Cheng Chen, Tong Niu, Xueru Bai, and Yang Xiang. Multi-agent adversarial time series forecasting, 2025. <https://openreview.net/forum?id=lj1qwU6Gxs>.
- Zhongzheng Qiao, Sheng Pan, Anni Wang, Viktoriya Zhukova, Yong Liu, Xudong Jiang, Qingsong Wen, Mingsheng Long, Ming Jin, and Chenghao Liu. It’s time: Towards the next generation of time series forecasting benchmarks. *arXiv preprint arXiv:2602.12147*, 2026.
- Zhenkai Qin, Qining Luo, Xunyi Nong, Xiaolong Chen, Hongfeng Zhang, and Cora Un In Wong. Mas- lstm: A multi-agent lstm-based approach for scalable anomaly detection in iiot networks. *Processes*, 13(3):753, 2025.
- Paul Quinlan, Jeremy Levasseur, Qingguo Li, and Xiaodan Zhu. Chronicle: A multimodal foundation model for joint language and time series understanding. *arXiv preprint arXiv:2605.20268*, 2026.
- Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Hena Ghonia, Rishika Bhagwatkar, Arian Khorasani, Mohammad Javad Darvishi Bayazi, George Adamopoulos, Roland Riachi, Nadhir Hassen, et al. Lag-llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*, 2023.
- Chidaksh Ravuru, Sagar Srinivas Sakhinana, and Venkataramana Runkana. Agentic retrieval-augmented generation for time series analysis. *arXiv preprint arXiv:2408.14484*, 2024.
- Weilin Ruan and Yuxuan Liang. Visual reasoning over time series via multi-agent system. *arXiv preprint arXiv:2602.03026*, 2026.
- Donald B Rubin. Inference and missing data. *Biometrika*, 63(3):581–592, 1976.
- Oisín Ryan, Jonas MB Haslbeck, and Lourens J Waldorp. Non-stationarity in time-series analysis: Modeling stochastic and deterministic trends. *Multivariate Behavioral Research*, 2025.
- Hoon Sagong, Heesu Kim, and Hanbeen Hong. Hi-darts: Hierarchical dynamically adapting reinforcement trading system. *arXiv preprint arXiv:2509.12048*, 2025.

- Oleksandr Shchur, Abdul Fatir Ansari, Caner Turkmen, Lorenzo Stella, Nick Erickson, Pablo Guerron, Michael Bohlke-Schneider, and Yuyang Wang. fev-bench: A realistic benchmark for time series forecasting. *arXiv preprint arXiv:2509.26468*, 2025.
- Feifei Shi, Xueyan Yin, Kang Wang, Wanyu Tu, Qifu Sun, and Huansheng Ning. Large language models for time series analysis: Techniques, applications, and challenges. *arXiv preprint arXiv:2506.11040*, 2025a.
- Haojie Shi, Yong Wang, and Lin Zhou. Sage: Spatiotemporal feature fusion for anomaly detection in multivariate time series. In *International Conference on Information Security and Cryptology*, pages 143–156. Springer, 2025b.
- Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt injection attack to tool selection in llm agents. *arXiv preprint arXiv:2504.19793*, 2025c.
- Jingzhe Shi, Qinwei Ma, Huan Ma, and Lei Li. Scaling law for time series forecasting. *Advances in Neural Information Processing Systems*, 37:83314–83344, 2024.
- Runze Shi, Shengyu Yan, Yuecheng Cai, and Chengxi Lv. Hubble: An llm-driven agentic framework for safe, diverse, and reproducible alpha factor discovery. *arXiv preprint arXiv:2604.09601*, 2026a.
- Yuhan Shi, Yuanyuan Yao, Lu Chen, Mourad Khayati, and Tianyi Li. A hierarchical agent system with reinforcement learning for multivariate time series data cleaning. *arXiv preprint arXiv:2605.04902*, 2026b.
- Geri Skenderi, Christian Joppi, Matteo Denitto, and Marco Cristani. Well googled is half done: Multimodal forecasting of new fashion product sales with image-based google trends. *Journal of Forecasting*, 43(6):1982–1997, 2024.
- Ireneous N Soyiri and Daniel D Reidpath. An overview of health forecasting. *Environmental health and preventive medicine*, 18(1):1–9, 2013.
- Chen Su, Yuanhe Tian, and Yan Song. Chain-of-thought reviewing and correction for time series question answering. *arXiv preprint arXiv:2512.22627*, 2025.
- Songyuan Sui, Zihang Xu, Yu-Neng Chuang, Kwei-Herng Lai, and Xia Hu. Training-free time series classification via in-context reasoning with llm agents. *arXiv preprint arXiv:2510.05950*, 2025.
- Daniil Sukhorukov, Andrei Zakharov, Nikita Glazkov, Katsiaryna Yanchanka, Vladimir Kirilin, Maxim Dubovitsky, Roman Sultimov, Yury Maximov, and Ilya Makarov. Hierarchical ai-meteorologist: Llm-agent system for multi-scale and explainable weather forecast reporting. In *AAAI-26 AI for Environmental Science Workshop*, 2026.
- Chenxi Sun, Hongyan Li, Yaliang Li, and Shenda Hong. Test: Text prototype aligned embedding to activate llm’s ability for time series. In *International Conference on Learning Representations*, volume 2024, pages 37854–37881, 2024.
- Hui Sun, Chang Xu, Haonan Xie, Hao Li, Yuhao Huang, Chuheng Zhang, Ming Jin, Xiaoguang Liu, Gang Wang, and Jiang Bian. Chatad: Reasoning-enhanced time-series anomaly detection with multi-turn instruction evolution. *arXiv preprint arXiv:2601.13546*, 2026.
- Siming Sun, Kai Zhang, Xuejun Jiang, Wenchao Meng, and Qinmin Yang. Enhancing llms for time series forecasting via structure-guided cross-modal alignment. *arXiv preprint arXiv:2505.13175*, 2025.
- Zhao Tan, Yiji Zhao, Shiyu Wang, Chang Xu, Yuxuan Liang, Xiping Liu, Shirui Pan, and Ming Jin. Sonar-ts: Search-then-verify natural language querying for time series databases. *arXiv preprint arXiv:2602.17001*, 2026.
- Yihong Tang, Andrew Robert Williams, Arjun Ashok, Vincent Zhihao Zheng, Lijun Sun, Alexandre Drouin, Issam H Laradji, Étienne Marcotte, and Valentina Zantedeschi. Dr-cik: A testbed for foresight-driven agents. *arXiv preprint arXiv:2605.27904*, 2026.
- Xiaoyu Tao, Mingyue Cheng, Ze Guo, Shuo Yu, Yaguo Liu, Qi Liu, and Shijin Wang. Memcast: Memory-driven time series forecasting with experience-conditioned reasoning. *arXiv preprint arXiv:2602.03164*, 2026a.
- Xiaoyu Tao, Mingyue Cheng, Chuang Jiang, Tian Gao, Huanjian Zhang, and Yaguo Liu. Cast-r1: Learning tool-augmented sequential decision policies for time series forecasting. *arXiv preprint arXiv:2602.13802*, 2026b.
- Xiaoyu Tao, Yuchong Wu, Mingyue Cheng, Ze Guo, and Tian Gao. Anomamind: Agentic time series anomaly detection with tool-augmented reasoning. *arXiv preprint arXiv:2602.13807*, 2026c.
- PAN Team, Jiannan Xiang, Yi Gu, Zihan Liu, Zeyu Feng, Qiyue Gao, Yiyan Hu, Benhao Huang, Guangyi Liu, Yichi Yang, Kun Zhou, Davit Abrahamyan, Arif Ahmad, Ganesh Bannur, Junrong Chen, Kimi Chen, Mingkai Deng, Ruobing Han, Xinqi Huang, Haoqiang Kang, Zheqi Liu, Enze Ma, Hector Ren, Yashowardhan Shinde, Rohan Shingre, Ramsundar Tanikella, Kaiming Tao, Dequan Yang, Xinle Yu, Cong Zeng, Binglin Zhou, Zhengzhong Liu,

- Zhiting Hu, and Eric P. Xing. Pan: A world model for general, interactable, and long-horizon world simulation. 2025. <https://arxiv.org/abs/2511.09057>.
- Malik Tiomoko, Hamza Cherkaoui, Giuseppe Paolo, Zhang Yili, Yu Meng, Zhang Keli, and Hafiz Tiomoko Ali. Human in the loop adaptive optimization for improved time series forecasting. *arXiv preprint arXiv:2505.15354*, 2025.
- Patara Trirat and Jae-Gil Lee. Monaq: Multi-objective neural architecture querying for time-series analysis on resource-constrained devices. In *The 2025 Conference on Empirical Methods in Natural Language Processing*, pages 16922–16950. Association for Computational Linguistics (ACL), 2025.
- Patara Trirat, Jin Myung Kwak, Jay Heo, Heejun Lee, and Sung Ju Hwang. Ts-debate: Multimodal collaborative debate for zero-shot time series reasoning. *arXiv preprint arXiv:2601.19151*, 2026.
- Mariano Vargas-Santiago, Diana Assaely León-Velasco, Christian Efraín Maldonado-Sifuentes, and Liliana Chanona-Hernandez. A state-of-the-art review of artificial intelligence (ai) applications in healthcare: advances in diabetes, cancer, epidemiology, and mortality prediction. *Computers*, 2025.
- Sky Chenwei Wan, Tianjun Hou, Yifei Wang, Xiqing Chang, and Aymeric Jan. Grammar of the wave: Towards explainable multivariate time series event detection via neuro-symbolic vlm agents. *arXiv preprint arXiv:2603.11479*, 2026.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18 (6):186345, 2024a.
- Xinlei Wang, Maike Feng, Jing Qiu, Jinjin Gu, and Junhua Zhao. From news to forecast: Integrating event analysis in llm-based time series forecasting with reflection. *Advances in Neural Information Processing Systems*, 37:58118–58153, 2024b.
- Yuxuan Wang, Haixu Wu, Jiayang Dong, Yong Liu, Chen Wang, Mingsheng Long, and Jianmin Wang. Deep time series models: A comprehensive survey and benchmark. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026a.
- Zhen Wang, Xi Zhou, Yating Yang, Bo Ma, Lei Wang, Rui Dong, and Azmat Anwar. Openforecast: A large-scale open-ended event forecasting dataset. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 5273–5294, 2025a.
- Zhensheng Wang, Xiaole Liu, Wenmian Yang, Kun Zhou, Yiquan Zhang, and Weijia Jia. Odtqa-fore: An open-domain tabular question answering dataset for future data forecasting and reasoning. *arXiv preprint arXiv:2606.02433*, 2026b.
- Zixiang Wang, Yinghao Zhu, Huiya Zhao, Xiaochen Zheng, Dehao Sui, Tianlong Wang, Wen Tang, Yasha Wang, Ewen Harrison, Chengwei Pan, et al. Colacare: Enhancing electronic health record modeling through large language model-driven multi-agent collaboration. In *Proceedings of the ACM on Web Conference 2025*, pages 2250–2261, 2025b.
- Michał Wawer and Jarosław Chudziak. Integrating traditional technical analysis with ai: A multi-agent llm-based approach to stock market forecasting. In *Proceedings of the 17th International Conference on Agents and Artificial Intelligence*, page 100–111. SCITEPRESS - Science and Technology Publications, 2025. doi: 10.5220/0013191200003890. <http://dx.doi.org/10.5220/0013191200003890>.
- Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. Transformers in time series: A survey. *arXiv preprint arXiv:2202.07125*, 2022.
- Muyan Weng, Defu Cao, Wei Yang, Yashaswi Sharma, and Yan Liu. Temporalbench: A benchmark for evaluating llm-based agents on contextual and event-informed time series tasks. *arXiv preprint arXiv:2602.13272*, 2026.
- Andrew Robert Williams, Arjun Ashok, Étienne Marcotte, Valentina Zantedeschi, Jithendaraa Subramanian, Roland Riachi, James Requeima, Alexandre Lacoste, Irina Rish, Nicolas Chapados, et al. Context is key: A benchmark for forecasting with essential textual information. *arXiv preprint arXiv:2410.18959*, 2024.
- Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *Forty-first International Conference on Machine Learning*, 2024.
- Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, volume 34, pages 22419–22430, 2021.

- Huizhe Wu, Wei Zhang, Weiwei Shen, and Jun Wang. Hybrid deep sequential modeling for social text-driven stock prediction. In *Proceedings of the 27th ACM international conference on information and knowledge management*, pages 1627–1630, 2018.
- Jing Wu, Yang Liu, Lin Zhang, Junbo Zeng, Jiabin Wang, Zi Ye, Guowen Li, Shilei Cao, Jiashun Cheng, Fang Wang, et al. Agcd: Agent-guided cross-modal decoding for weather forecasting. *arXiv preprint arXiv:2603.15260*, 2026a.
- Xingjian Wu, Junkai Lu, Zhengyu Li, Xiangfei Qiu, Jilin Hu, Chenjuan Guo, Christian S Jensen, and Bin Yang. Timeart: Towards agentic time series reasoning via tool-augmentation. *arXiv preprint arXiv:2601.13653*, 2026b.
- Yihan Xia, Panpan You, Taotao Wang, Fang Liu, Han Qi, Xiaoxiao Wu, and Shengli Zhang. Agentic trading: When llm agents meet financial markets. *arXiv preprint arXiv:2605.19337*, 2026.
- Mengxi Xiao, Zihao Jiang, Lingfei Qian, Zhengyu Chen, Yueru He, Yijing Xu, Yuecheng Jiang, Dong Li, Ruey-Ling Weng, Min Peng, Jimin Huang, Sophia Ananiadou, and Qianqian Xie. Retrieval-augmented large language models for financial time series forecasting. *arXiv preprint arXiv:2502.05878*, 2025a.
- Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. Tradingagents: Multi-agents llm financial trading framework. *arXiv preprint arXiv:2412.20138*, 2024.
- Yijia Xiao, Edward Sun, Tong Chen, Fang Wu, Di Luo, and Wei Wang. Trading-r1: Financial trading with llm reasoning via reinforcement learning. *arXiv preprint arXiv:2509.11420*, 2025b.
- Stephan Xie, Ben Cohen, Mononito Goswami, Junhong Shen, Emaad Khwaja, Chenghao Liu, David Asker, Othmane Abou-Amal, and Ameet Talwalkar. Arfbench: Benchmarking time series question answering ability for software incident response. *arXiv preprint arXiv:2604.21199*, 2026.
- Zhe Xie, Zeyan Li, Xiao He, Longlong Xu, Xidao Wen, Tieying Zhang, Jianjun Chen, Rui Shi, and Dan Pei. Chatts: Aligning time series with llms via synthetic data for enhanced understanding and reasoning. *arXiv preprint arXiv:2412.03104*, 2024.
- Binyan Xu, Xilin Dai, Di Tang, and Kehuan Zhang. One surrogate to fool them all: Universal, transferable, and targeted adversarial attacks with clip. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 3087–3101, 2025a.
- Binyan Xu, Fan Yang, Xilin Dai, Di Tang, and Kehuan Zhang. Clip-guided backdoor defense through entropy-based poisoned dataset separation. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 7415–7423, 2025b.
- Binyan Xu, Xilin Dai, and Kehuan Zhang. Contextual agentic memory is a memo, not true memory. *arXiv preprint arXiv:2604.27707*, 2026a.
- Binyan Xu, Dong Fang, Haitao Li, and Kehuan Zhang. From multi-agent to single-agent: When is skill distillation beneficial? *arXiv preprint arXiv:2604.01608*, 2026b.
- Binyan Xu, Fan Yang, Xilin Dai, Di Tang, and Kehuan Zhang. From internal diagnosis to external auditing: A vlm-driven paradigm for online test-time backdoor defense. *arXiv preprint arXiv:2601.19448*, 2026c.
- Binyan Xu, Fan Yang, Di Tang, Xilin Dai, and Kehuan Zhang. Breaking the stealth-potency trade-off in clean-image backdoors with generative trigger optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 27197–27205, 2026d.
- Congluo Xu, Zhaobin Liu, and Ziyang Li. Finarena: A human-agent collaboration framework for financial market analysis and forecasting. *arXiv preprint arXiv:2503.02692*, 2025c.
- Longkun Xu, Xiaochun Zhang, Qiantu Tuo, and Rui Li. Sea-ts: Self-evolving agent for autonomous code generation of time series forecasting algorithms. *arXiv preprint arXiv:2603.04873*, 2026e.
- Wenyan Xu, Dawei Xiang, Yue Liu, Xiyu Wang, Yanxiang Ma, Liang Zhang, Shu Hu, Chang Xu, and Jiaheng Zhang. Finmultitime: A four-modal bilingual dataset for financial time-series analysis. *arXiv preprint arXiv:2506.05019*, 2025d.
- Xiong Xiao Xu, Haoran Wang, Yueqing Liang, Philip S Yu, Yue Zhao, and Kai Shu. Can multimodal llms perform time series anomaly detection? In *Proceedings of the ACM Web Conference 2026*, pages 5392–5403, 2026f.
- Yumo Xu and Shay B Cohen. Stock movement prediction from tweets and historical prices. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1970–1979, 2018.

- Zhijian Xu, Wanxu Cai, Xilin Dai, Zhaorong Deng, and Qiang Xu. Fidel-ts: A high-fidelity benchmark for multimodal time series forecasting. *arXiv preprint arXiv:2509.24789*, 2025e.
- Hao Xue and Flora D Salim. Promptcast: A new prompt-based learning paradigm for time series forecasting. *IEEE Transactions on Knowledge and Data Engineering*, 36(11):6851–6864, 2023.
- Qi Yan, Raihan Seraj, Jiawei He, Lili Meng, and Tristan Sylvain. Autocast++: Enhancing world event prediction with zero-shot ranking-based context retrieval. In *International Conference on Learning Representations*, volume 2024, pages 34523–34539, 2024.
- Jie Yang, Yifan Hu, Kexin Zhang, Luyang Niu, Philip S Yu, and Kaize Ding. Revisiting multivariate time series forecasting with missing values. *arXiv preprint arXiv:2509.23494*, 2025a.
- Jie Yang, Rui Zhang, Ziyang Cheng, Dawei Cheng, Guang Yang, and Bo Wang. Grad: Guided relation diffusion generation for graph augmentation in graph fraud detection. In *Proceedings of the ACM on Web Conference 2025*, pages 5308–5319, 2025b.
- Jie Yang, Yifan Hu, Yuante Li, Kexin Zhang, Kaize Ding, and Philip S Yu. From observations to states: Latent time series forecasting. *arXiv preprint arXiv:2602.00297*, 2026a.
- Jie Yang, Kexin Zhang, Guibin Zhang, Philip S Yu, and Kaize Ding. Glocal information bottleneck for time series imputation. *Advances in Neural Information Processing Systems*, 38:104452–104484, 2026b.
- Qingchuan Yang, Simon Mahns, Sida Li, Anri Gu, Jibang Wu, and Haifeng Xu. Llm-as-a-prophet: Understanding predictive intelligence with prophet arena. *arXiv preprint arXiv:2510.17638*, 2025c.
- Tiankai Yang, Junjun Liu, Michael Siu, Jiahang Wang, Zhuangzhuang Qian, Chanjuan Song, Cheng Cheng, Xiyang Hu, and Yue Zhao. Ad-agent: A multi-agent framework for end-to-end anomaly detection. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 191–205, 2025d.
- Xu Yang, Xiao Yang, Shikai Fang, Yifei Zhang, Jian Wang, Bowen Xian, Qizheng Li, Jingyuan Li, Minrui Xu, Yuante Li, et al. R&d-agent: An llm-agent framework towards autonomous data science. *arXiv preprint arXiv:2505.14738*, 2025e.
- Yiyuan Yang, Zichuan Liu, Lei Song, Kai Ying, Zhiguang Wang, Tom Bamford, Svitlana Vyetenko, Jiang Bian, and Qingsong Wen. Time-ra: Towards time series reasoning for anomaly diagnosis with llm feedback. *arXiv preprint arXiv:2507.15066*, 2025f.
- Yongbin Yang, Mengdie Wang, Jiyuan Wang, Pan Li, and Mengjie Zhou. Multi-agent deep reinforcement learning for integrated demand forecasting and inventory optimization in sensor-enabled retail supply chains. *Sensors*, 25(8): 2428, 2025g.
- Wen Ye, Jinbo Liu, Defu Cao, Wei Yang, and Yan Liu. When llm meets time series: Can llms perform multi-step time series reasoning and inference. *arXiv preprint arXiv:2509.01822*, 2025.
- Wen Ye, Wei Yang, Defu Cao, Yizhou Zhang, Luminyuan Tang, Jie Cai, and Yan Liu. Ts-reasoner: Domain-oriented time series inference agents for reasoning and automated analysis. *Transactions on Machine Learning Research*, 2026.
- Chin-Chia Michael Yeh, Vivian Lai, Uday Singh Saini, Xiran Fan, Yujie Fan, Junpeng Wang, Xin Dai, and Yan Zheng. Dcats: Empowering time series forecasting with LLM-agents. *arXiv preprint arXiv:2508.04231*, 2025.
- Yao Yin, Zhenyu Xiao, Musheng Li, Yiwen Liu, Sutong Nan, Yiting He, Ruiqi Wang, Zhenwei Zhang, Qingmin Liao, and Yuantao Gu. Mmts-bench: A comprehensive benchmark for time series understanding and reasoning. *arXiv preprint arXiv:2602.08588*, 2026.
- Fangxu Yu, Hongyu Zhao, and Tianyi Zhou. Ts-reasoner: Aligning time series foundation models with llm reasoning. *arXiv preprint arXiv:2510.03519*, 2025a.
- Fangxu Yu, Xingang Guo, Lingzhi Yuan, Haoqiang Kang, Hongyu Zhao, Lianhui Qin, Furong Huang, Bin Hu, and Tianyi Zhou. Tsrbench: A comprehensive multi-task multi-modal time series reasoning benchmark for generalist models. *arXiv preprint arXiv:2601.18744*, 2026a.
- Xiaoyun Yu, Li fan, Xiangfei Qiu, Nanqing Dong, Yonggui Huang, Honggang Qi, Geguang Pu, Wanli Ouyang, Xi Chen, and Jilin Hu. Memts: Internalizing domain knowledge via parameterized memory for retrieval-free domain adaptation of time series foundation models. *arXiv preprint arXiv:2602.13783*, 2026b.

- Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yuechen Jiang, Yupeng Cao, Zhi Chen, Jordan W Suchow, Zhenyu Cui, Rong Liu, et al. Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *Advances in Neural Information Processing Systems*, 37:137010–137045, 2024.
- Yangyang Yu, Haohang Li, Zhi Chen, Yuechen Jiang, Yang Li, Jordan W Suchow, Denghui Zhang, and Khaldoun Khashanah. Finnem: A performance-enhanced llm trading agent with layered memory and character design. *IEEE Transactions on Big Data*, 2025b.
- Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 11121–11128, 2023.
- Zhiyuan Zeng, Jiashuo Liu, Siyuan Chen, Tianci He, Yali Liao, Yixiao Tian, Jinpeng Wang, Zaiyuan Wang, Yang Yang, Lingyue Yin, et al. Futurex: An advanced live benchmark for llm agents in future prediction. *arXiv preprint arXiv:2508.11987*, 2025.
- Tianxiang Zhan, Xiaobao Song, Tong Guan, Shirui Pan, and Ming Jin. Aion: Next-generation tasks and practical harness for time series. *arXiv preprint arXiv:2605.25045*, 2026.
- Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *The Thirteenth International Conference on Learning Representations*, 2025a. <https://openreview.net/forum?id=V4y0CpX4hK>.
- Peng Zhang, Yapeng Ji, Qi Cheng, Yunji Liang, and Zhiwen Yu. Stockagent: A multi-agent collaborative framework for financial time series prediction. *2025 IEEE International Conference on Social Computing and Networking (SocialCom)*, pages 8–17, 2025b.
- Regina Zhang, Si Qi Goh, Xingsheng Chen, Zongru Li, Honggang Wen, Jiale Guo, Menglin Yang, Hongzhi Yin, Qiang Yang, SM Yiu, et al. From prompts to agents: A comprehensive survey of llm-driven time series analysis. *Available at SSRN 6614598*, 2025c.
- Xiaohan Zhang, Tian Gao, Mingyue Cheng, Bokai Pan, Ze Guo, Yaguo Liu, Xiaoyu Tao, and Qi Liu. Alphacast: A human wisdom-llm intelligence co-reasoning framework for interactive time series forecasting. *arXiv preprint arXiv:2511.08947*, 2026a.
- Xiyuan Zhang, Ranak Roy Chowdhury, Rajesh K Gupta, and Jingbo Shang. Large language models for time series: A survey. *arXiv preprint arXiv:2402.01801*, 2024.
- Zhixing Zhang, Jesen Zhang, Hao Liu, Qinhan Lv, Jing Yang, Kaitong Cai, and Keze Wang. Agriworld: A world tools protocol framework for verifiable agricultural reasoning with code-executing llm agents. *arXiv preprint arXiv:2602.15325*, 2026b.
- Zijian Zhang, Yujie Sun, Zepu Wang, Yuqi Nie, Xiaobo Ma, Ruolin Li, Peng Sun, and Xuegang Ban. Large language models for mobility analysis in transportation systems: A survey on forecasting tasks. *Transportation Research Record*, 2026c.
- Haokun Zhao, Xiang Zhang, Jiaqi Wei, Yiwei Xu, Yuting He, Siqi Sun, and Chenyu You. Timeseriesscientist: A general-purpose ai agent for time series analysis. *arXiv preprint arXiv:2510.01538*, 2025.
- Lecheng Zheng, Zhengzhang Chen, Dongjie Wang, Chengyuan Deng, Reon Matsuoka, and Haifeng Chen. Lemma-rca: A large multi-modal multi-domain dataset for root cause analysis. *arXiv preprint arXiv:2406.05375*, 2024.
- Siru Zhong, Yuxuan Liang, Weilin Ruan, Ming Jin, Huan Li, and Qingsong Wen. Time-vlm: Exploring multimodal vision-language models for augmented time series forecasting. In *AI for Time Series*, pages 173–203. CRC Press, 2025.
- Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11106–11115, 2021.
- Jiahui Zhou, Dan Li, Boxin Li, Xiao Zhang, Erli Meng, Lin Li, Zhuomin Chen, Jian Lou, and See-Kiong Ng. Time series reasoning via process-verifiable thinking data synthesis and scheduling for tailored llm reasoning. *arXiv preprint arXiv:2602.07830*, 2026.
- Luca Zhou, Pratham Yashwante, Marshall Fisher, Alessio Sampieri, Zihao Zhou, Fabio Galasso, and Rose Yu. Cats-bench: Can language models describe numeric time series? *arXiv preprint arXiv:2509.20823*, 2025a.

- Shu Zhou, Yunyang Xuan, Yuxuan Ao, Xin Wang, Tao Fan, and Hao Wang. MERIT: Multi-agent collaboration for unsupervised time series representation learning. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 2025b.
- Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning*, pages 27268–27286. PMLR, 2022.
- Tian Zhou, Peisong Niu, Liang Sun, Rong Jin, et al. One fits all: Power general time series analysis by pretrained lm. *Advances in neural information processing systems*, 36:43322–43355, 2023.
- Xin Zhou, Weiqing Wang, Francisco J Baldán, Wray Buntine, and Christoph Bergmeir. Motime: A dataset suite for multimodal time series forecasting. *arXiv preprint arXiv:2505.15072*, 2025c.
- Zihao Zhou and Rose Yu. Can llms understand time series anomalies? In *International Conference on Learning Representations*, volume 2025, pages 1858–1896, 2025.
- Peng Zhu, Yuante Li, Yifan Hu, Qinyuan Liu, Dawei Cheng, and Yuqi Liang. Lsr-igru: Stock trend prediction based on long short-term relationships and improved gru. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 5135–5142, 2024.
- Andy Zou, Tristan Xiao, Ryan Jia, Joe Kwon, Mantas Mazeika, Richard Li, Dawn Song, Jacob Steinhardt, Owain Evans, and Dan Hendrycks. Forecasting future world events with neural networks. *Advances in Neural Information Processing Systems*, 35:27293–27305, 2022.